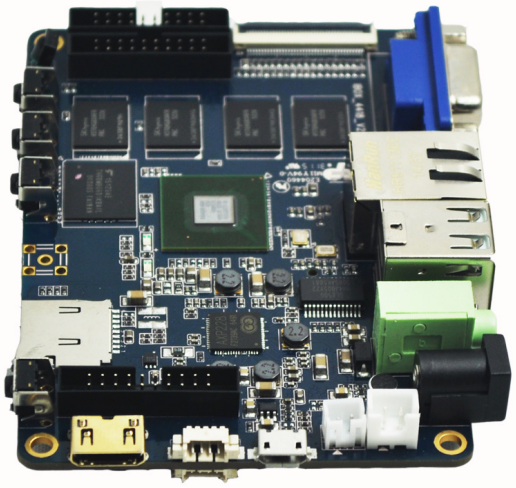
X6818 Linux平台

用户手册



卡片电脑



开发板

深圳市九鼎创展科技有限公司

www.9tripod.com

版权声明

本手册版权归属深圳市九鼎创展科技有限公司所有，并保留一切权力。非经九鼎创展同意(书面形式)，任何单位及个人不得擅自摘录本手册部分或全部，违者我们将追究其法律责任。

敬告：

在售开发板的手册会经常更新，请在 http://www.9tripod.com 网站下载最新手册，不再另行通知。

版本说明

版本号	日期	作者	描述
Rev.01	2015/8/23	lqm	原始版本
Rev.02	2016/1/22	lqm	勘误
Rev.03	2016/7/12	Jason Shi	修正，更新

技术支持

如果您对文档有所疑问，您可以在办公时间（星期一至星期五上午 9:00~12:00；下午 1:30~6:00）通过拨打技术支持电话、E-mail、留言到 BBS 论坛（http://www.xboot.org）。

网　　址：　www.9tripod.com
联系电话：　销售 0755-33121205（400-003-3436）　0755-61952310
　　技术支持专线：0755-61952306
　　软件专线：0755-61952309
　　硬件专线：0755-61952307
E - mail：　supports@9tripod.com

销售与服务网络

公司：深圳市九鼎创展科技有限公司
地址：深圳市宝安中心区兴业路宝安互联网产业基地 B 区 3003B 室
邮编：518101
电话：0755-33121205（400-003-3436）　0755-61952310
网址：http://www.9tripod.com
论坛：http://www.xboot.org
淘宝：http://armeasy.taobao.com

QQ 群：
x210/i210 一群：　【 23831259】
x210/i210 二群：　【211127570】
x4412/ibox4412一群：　【16073601】
x4412/ibox4412 二群：　【211128231】
X4418/ibox4418 论坛：　【199358213】
x6818/ibox6818 论坛：　【189920370】
x3288 技术论坛：　【159144256】

热烈欢迎广大同仁扫描右侧九鼎创展官方公众微信号，关注有礼，您将优先得知九鼎创展最新动态！

目录

版权声明.....1
第 1 章 使用虚拟机安装 ubuntu 系统.....7
1.1 VMware 虚拟机的安装.....7
1.2 在虚拟机上安装 ubuntu 系统.....14
1.3 VMware Tools 的安装.....22
第 2 章 编译 linux+QT.....27
2.1 安装 uboot，内核源码包.....27
2.2 编译 uboot.....27
2.3 编译内核.....27
2.4 编译文件系统.....28
第 3 章 烧写 linux QT 映像.....30
3.1 ubuntu 下 fastboot 的安装.....30
3.1.1 安装 fastboot.....30
3.1.2 新建 51-android.rules.....30
3.2 ubuntu 下制作启动卡.....30
3.3 ubuntu 下烧写映像文件.....33
3.3.1 emmc 中无固件.....33
3.3.2 emmc 有正常启动的映像.....34
3.4 windows 下 fastboot 的安装.....34
3.5 windows 下制作启动卡.....34
3.6 windows 下 fastboot 烧写映像文件.....39
3.7 使用 TF 卡脱机更新映像文件.....40
3.8 emmc 中不存在 bootloader 时更新 uboot.....41
3.9 emmc 中存在 bootloader 时更新 uboot.....41
3.10 uboot 环境变量设置.....41
第 4 章 QT 文件系统的搭建.....43
4.1 下载 buildroot.....43
4.2 配置 buildroot.....43
4.3 编译 buildroot.....48
4.4 测试 QT5.4 默认示例.....48
4.5 安装 QT Creator.....51
4.6 编译 QT Creator 默认示例.....51
第 5 章 qttest 测试程序.....62
5.1 使用 QT_demo 测试 LED.....62
5.2 使用 QT_demo 测试蜂鸣器.....62
5.3 使用 QT_demo 调节背光.....63
5.4 使用 QT_demo 测试按键.....63
5.5 使用 QT_demo 测试 ADC 电压.....64
5.6 使用 QT_demo 测试音频.....64
5.7 使用 QT_demo 测试触摸屏.....64
5.8 使用 QT_demo 测试串口.....66

5.9 使用 QT_demo 测试网络 66
5.10 使用 QT_demo 测试 TF 卡 67
5.11 使用 QT_demo 测试 U 盘 67
5.12 使用 QT_demo 测试休眠唤醒 68
5.13 使用 QT_demo 测试重启 68
5.14 使用 QT_demo 测试关机 69
第 6 章 linux 底层开发示例 70
6.1 播放 mp3 70
6.2 在后台运行程序 70
6.3 中止程序的运行 70
6.4 屏幕抓图 70
6.5 挂载 TF 卡 70
6.6 挂载 U 盘 71
6.7 保存系统时钟 72
6.8 掉电保存数据到 flash 72
6.9 设置开机自动运行程序 73
6.10 查看开发板内存信息 73
6.11 uboot 使用 tftp 下载内核 73
6.12 挂载 nfs 网络文件系统 74
第 7 章 使用 ramdisk 文件系统 76
第 8 章 linux 应用开发示例 77
8.1 Hello World 77
8.2 LED 测试程序 78
8.3 数学函数库调用 79
8.4 多线程编程示例 79
8.5 多进程编程示例 81
8.6 makefile 编程示例 83
8.7 mplayer 移植 86
8.8 madplayer 移植 86
8.9 屏幕抓图工具 gsnap 移植 86
第 9 章 安装 ubuntu12.04 系统 88
9.1 ubuntu 烧写方法一 88
9.1.1 将 uboot 和 boot.img 烧写到 emmc 88
9.1.2 将 ubuntu 文件系统烧到到 TF 卡 88
9.1.3 设置 uboot 环境变量 90
9.2 ubuntu 烧写方法二 91
9.2.1 制作 ubuntu 升级卡 91
9.2.2 使用升级卡一键升级 ubuntu 系统 94
9.2.3 设置 uboot 环境变量 94
第 10 章 体验 ubuntu12.04 系统 95
10.1 ubuntu 系统界面 95
10.2 使用 USB 鼠标键盘 95
10.3 使用超级终端 95

10.4 使用 scrot 工具抓图.....96
10.5 使用有线上网.....96
10.6 使用 WIFI 上网.....96
10.7 使用 3G 上网.....97
10.8 使用蓝牙传输数据.....97
10.9 使用 U 盘.....97
10.10 播放音乐.....97
10.11 播放视频.....98
10.12 使用 USB 摄相头.....98
第 11 章 其他产品介绍.....100
11.1 核心板系列.....100
11.2 开发板系列.....100
11.3 卡片电脑系列.....100

第 1 章 使用虚拟机安装 ubuntu 系统

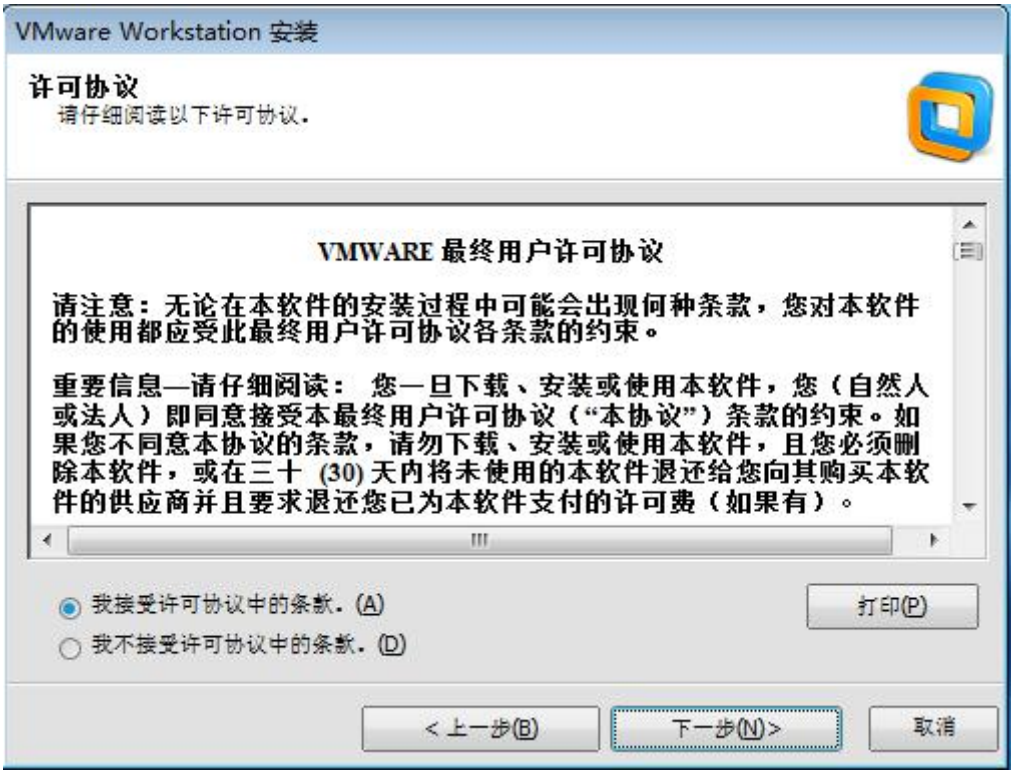
在进行 linux 开发时，并不像 android 那样，对 PC 机性能要求很高，因此我们完全可以采用虚拟机来开发 linux。本章节讲述在虚拟机下安装 ubuntu15.04 操作系统，当然您也可以直接安装纯净的 ubuntu 系统，而不使用虚拟机，这样编译效率会高出很多。在 ubuntu 下，也可以使用 linux 下的虚拟机再虚拟一个 windows 系统，其方式大同小异，这里就不赘述了。

1.1 VMware 虚拟机的安装

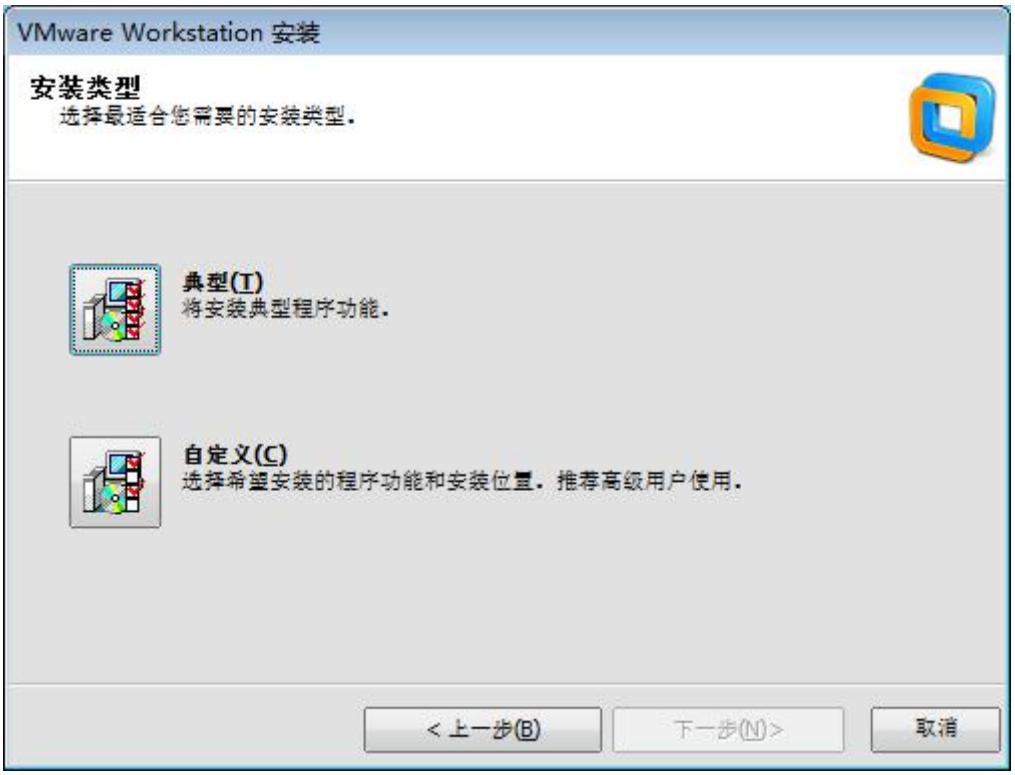
从网上下载最新的 VMware 安装包，如 VMware-workstation-full-10.0.1-1379776.exe，双击运行。



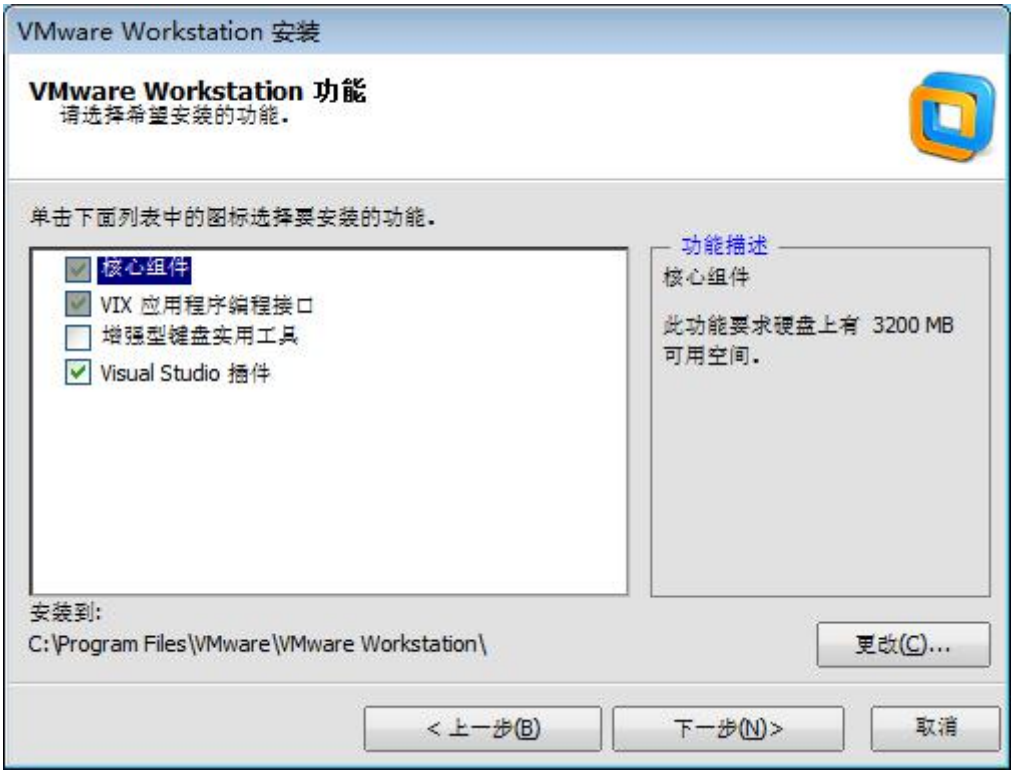
点击下一步，



选择我接受许可协议中的条款，点击下一步，



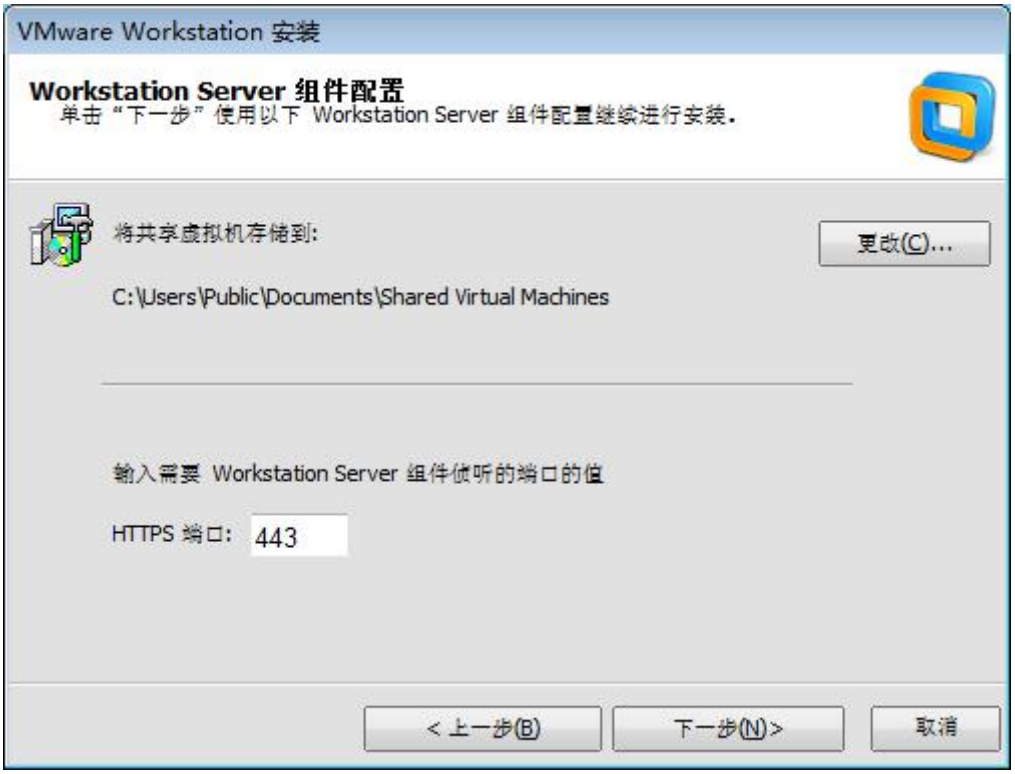
选择自定义，点击下一步：



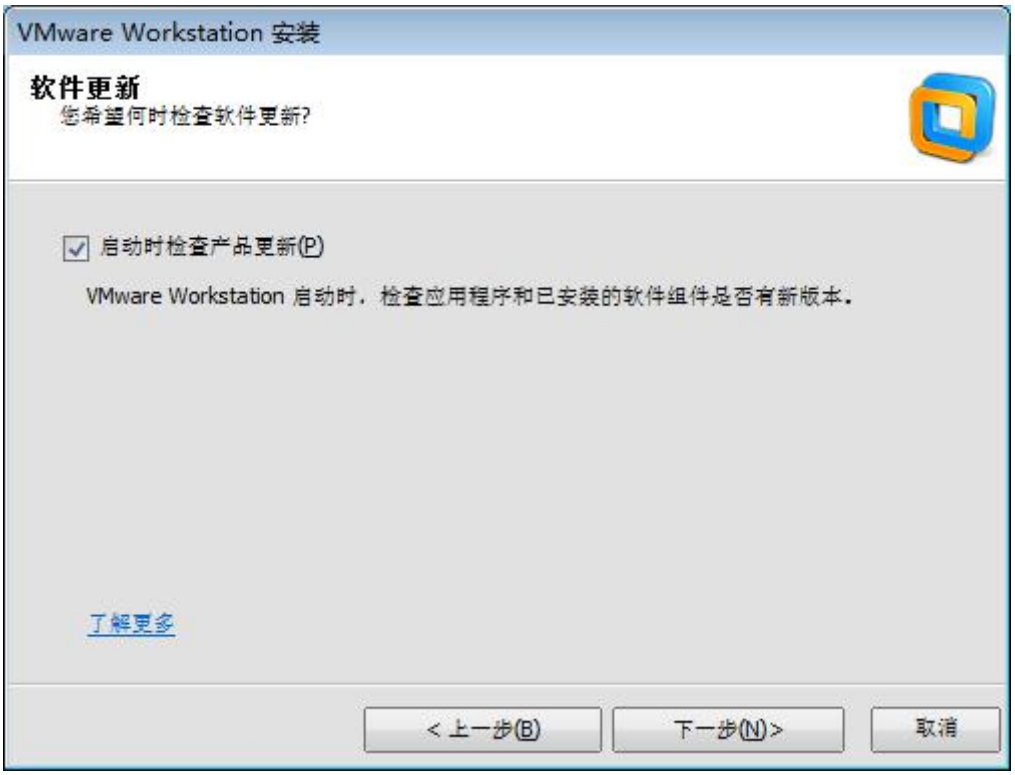
更改安装路径，通常安装到 D 盘，



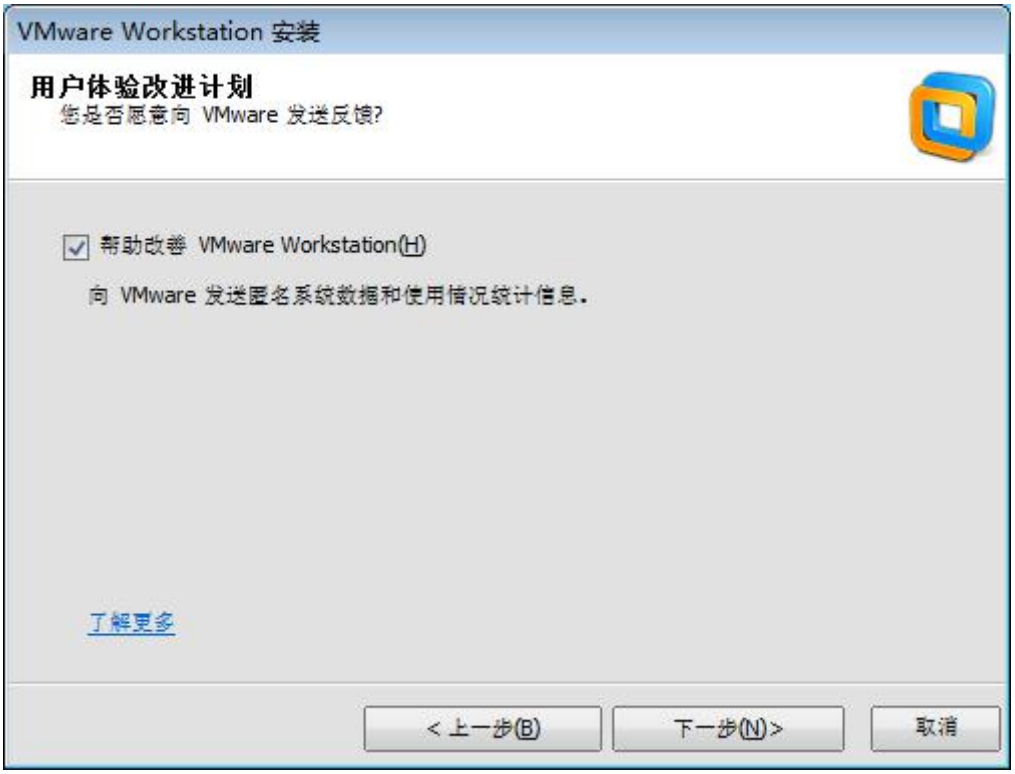
点击下一步：



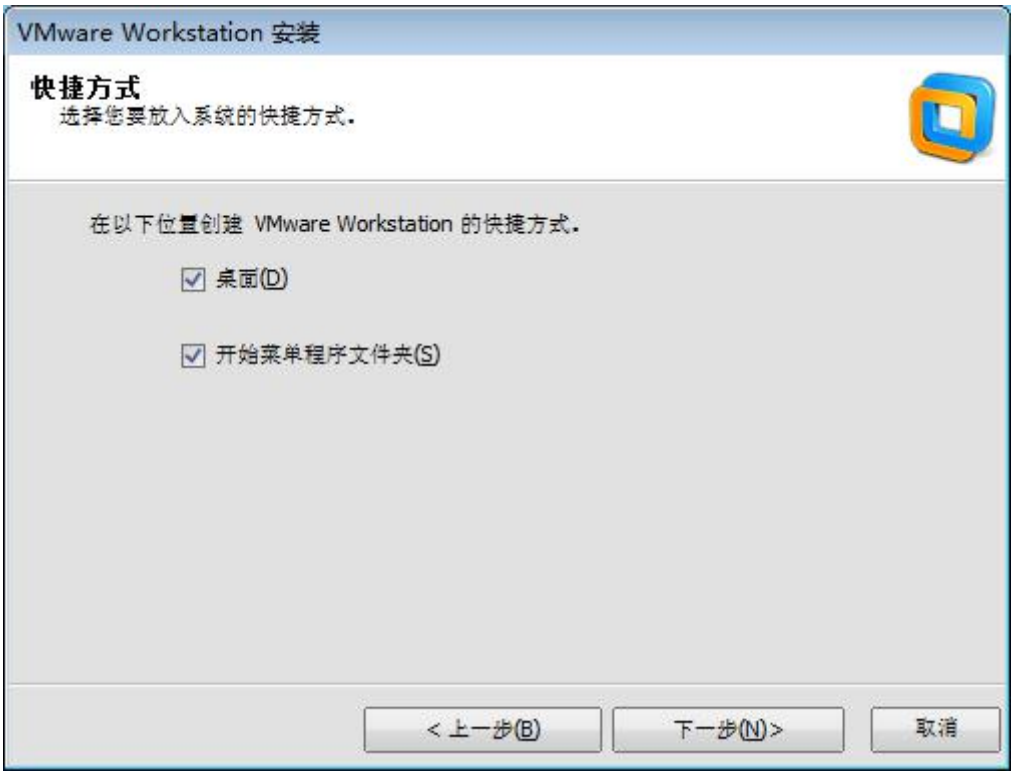
这里默认即可，直接点击下一步：



再点击下一步：



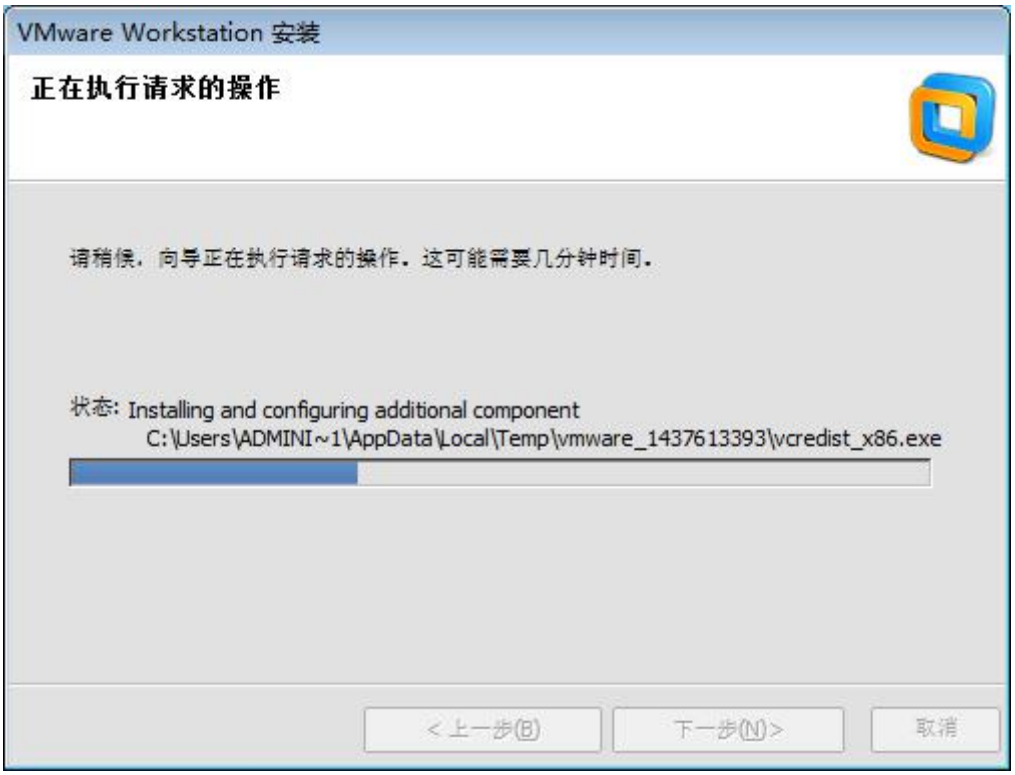
继续点击下一步：



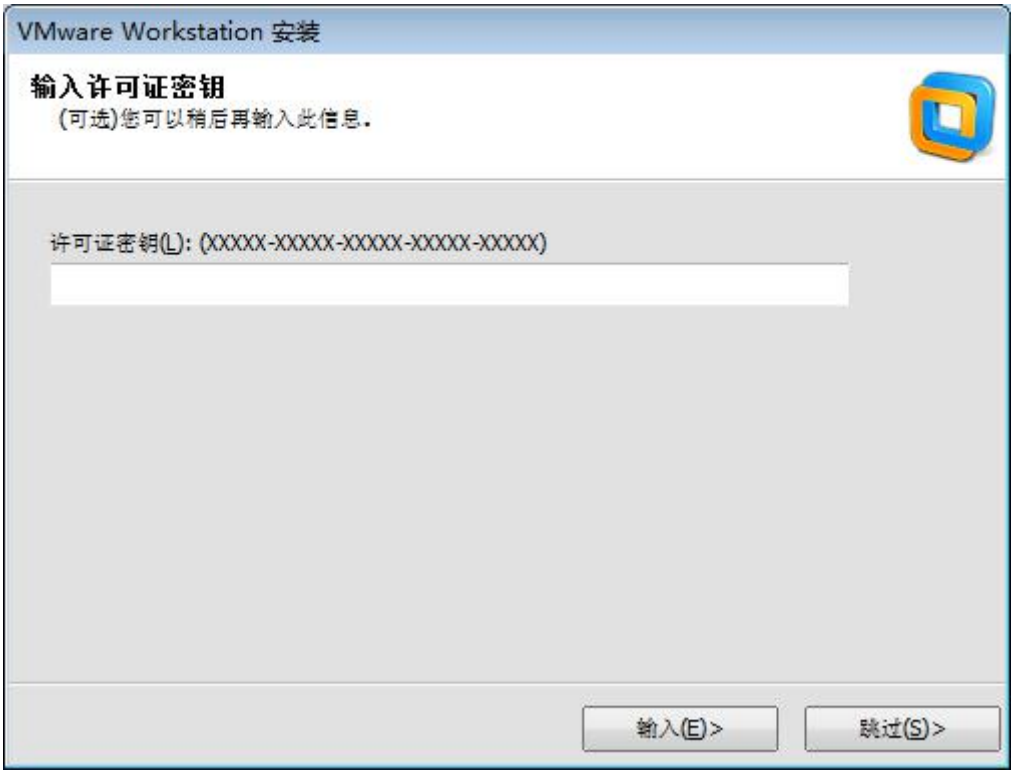
点击下一步：



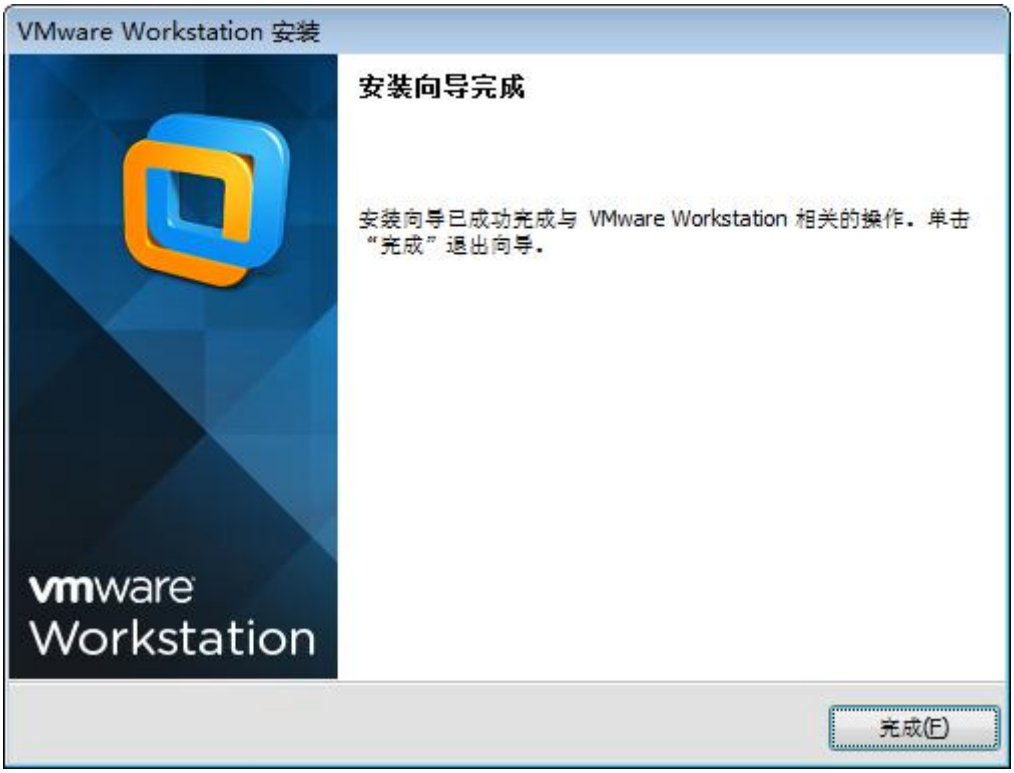
点继续：



稍等片刻，虚拟机安装完成，弹出需要注册的界面，如下图所示：



为支持正版，建议购买正版序列号，填入正确的序列号，点击输入，如果序列号填写正确，将会弹出如下界面：



点击完成，结束虚拟机的安装。

1.2 在虚拟机上安装 ubuntu 系统

使用虚拟机安装 ubuntu 系统时，无需在 windows 系统上分区，只需预留一个不小于 100G 的硬盘即可。

在 ubuntu 官网下载 ubuntu 操作系统，如 ubuntukylin-15.04-desktop-amd64.iso，打开 VMware 虚拟机：



点击创建新的虚拟机，


新建虚拟机向导
欢迎使用新建虚拟机向导
您希望使用什么类型的配置?
典型(推荐)(T)
通过几个简单的步骤创建 Workstation 10.0 虚拟机。
自定义(高级)(C)
创建带有 SCSI 控制器类型、虚拟磁盘类型以及与旧版 VMware 产品兼容性等高级选项的虚拟机。
vmware Workstation
帮助
< 上一步(B)
下一步(N) >
取消

选择典型，点击下一步，



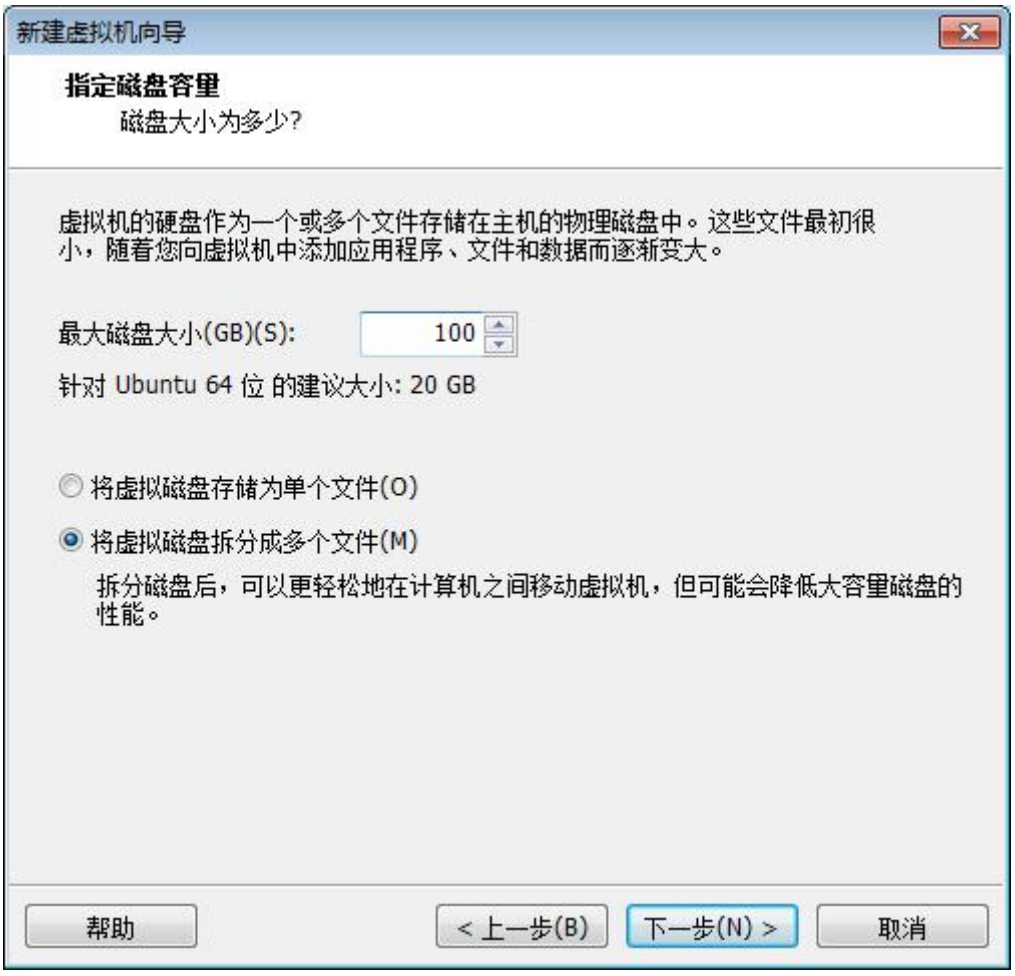
选择安装程序光盘映像文件，点击浏览，选择下载的 ubuntu 映像，虚拟机默认会识别出 ubuntu 系统的版本。点击下一步，



输入全名，用户名及密码，点击下一步，



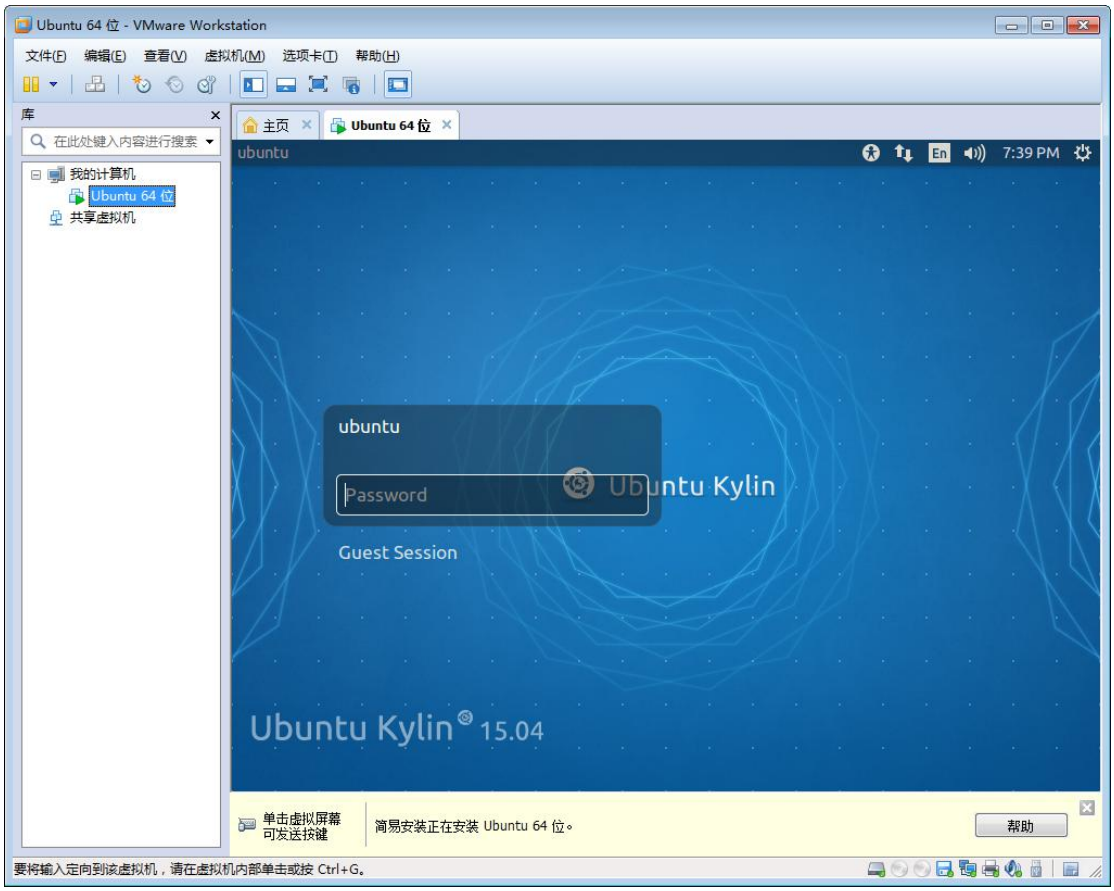
在位置中选择 ubuntu 的安装位置，注意，保证安装磁盘至少有不小于 100G 的硬盘空间。点击下一步，



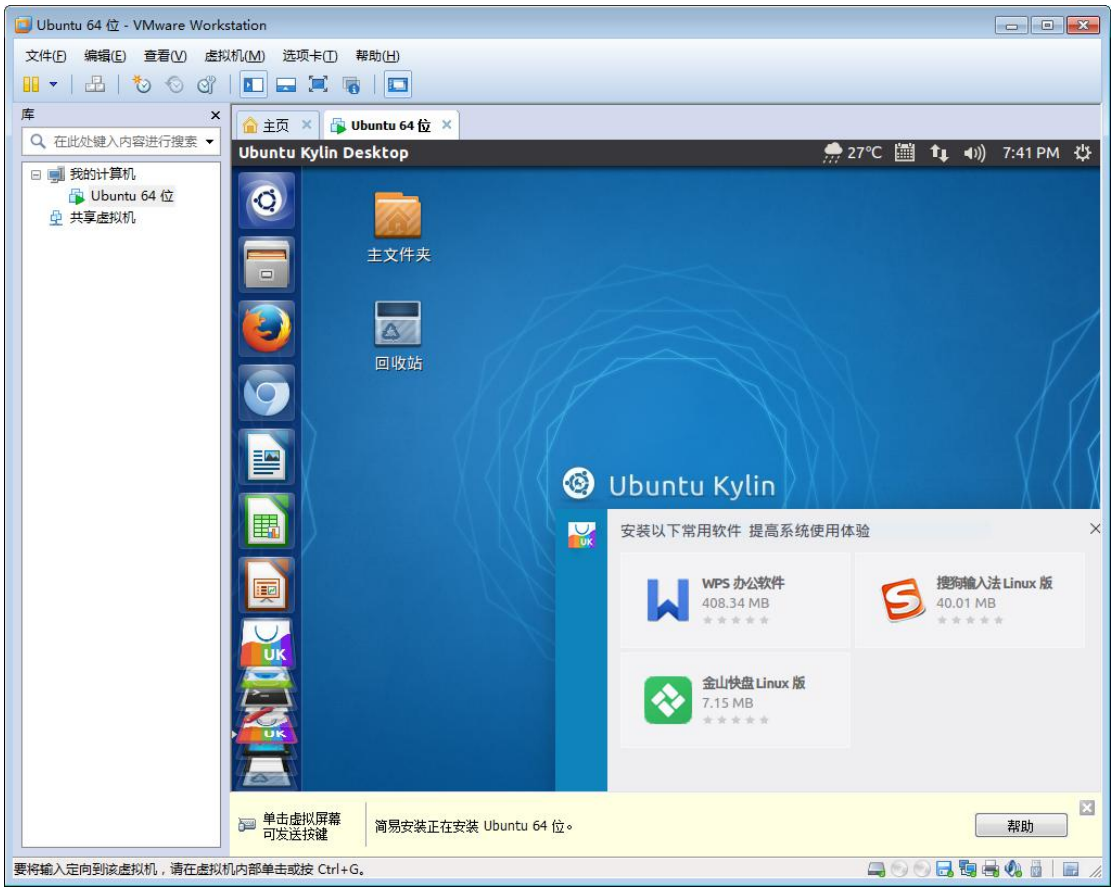
将最大磁盘大小改为 100GB，点击下一步，



点击完成，虚拟机将会自动安装 ubuntu 操作系统。安装操作系统需要等一段时间，稍安勿躁，静待系统安装完成，会自动进入 ubuntu 系统，安装完成后界面如下：



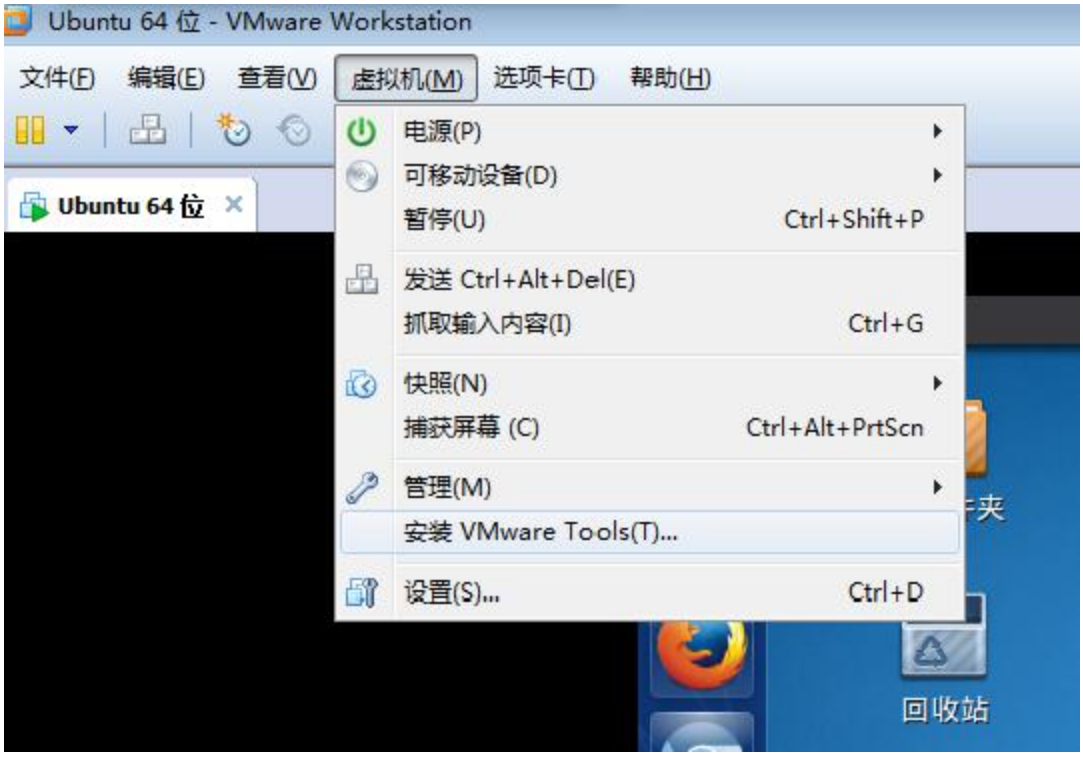
输入密码，即可登录 ubuntu 操作系统了。



值得注意的是，将虚拟机全屏，界面只能居中显示，无法全屏显示，而且虚拟机和 windows 系统之间无法相互拷贝数据。安装 VMware Tools 能够解决这个问题。

1.3 VMware Tools 的安装

点击虚拟机->安装 VMware Tools，如下图所示：



如果正常，该操作将 VMwareTools 放入虚拟光驱。进入/media 将 VMware Tools 文件拷贝到一个临时文件并解压：

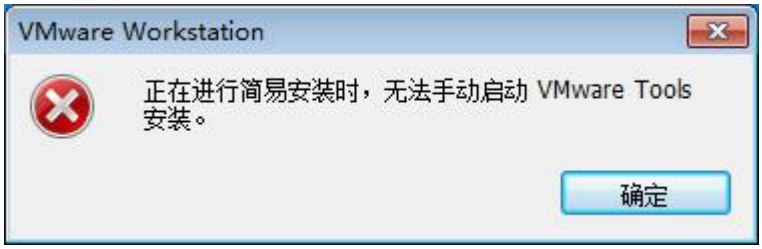
```
tar zxvf  VMwareTools-8.4.2-261024.tar.gz ./
```

发现当前目录多了 vmware-tools-distrib 目录。进入该目录，并开始安装：

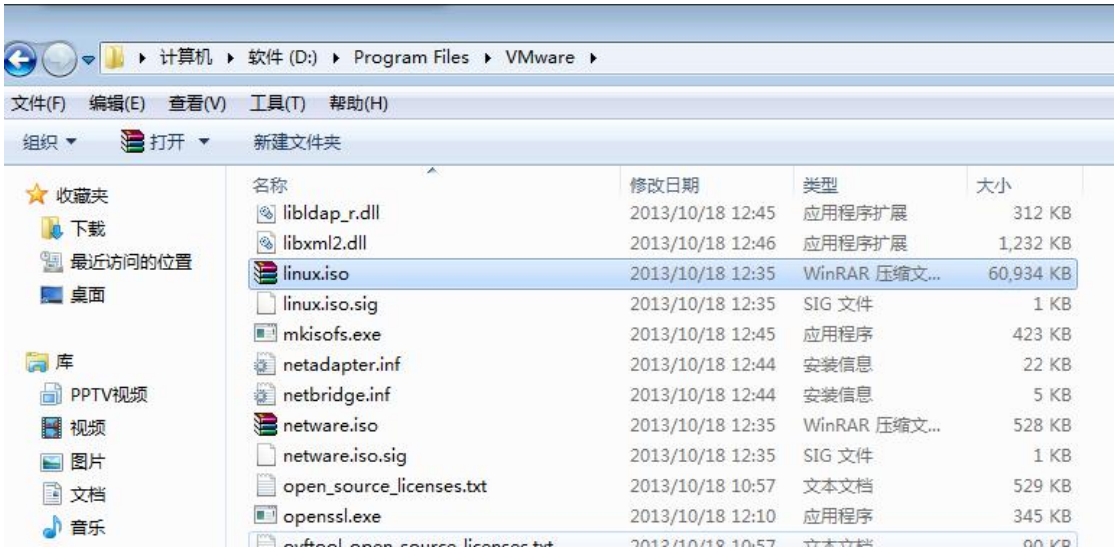
```
cd vmware-tools-distrib
./vmware-install.pl
```

在安装的过程中遇到安装的提示，按回车即可。重启虚拟机系统，安装完成。这时虚拟系统已经能和 windows 系统互相复制了。同时，屏幕也可以自适应了。

有时候，VMware-tools 安装并不是如此顺利。比如在使用简易安装 ubuntu 系统时。在点击虚拟机->安装 VMware Tools 时，很可能会弹出如下界面：



或者，安装 VMware Tools 的选项完全为灰色，无法点击。这时，我们可以在 VMware 的安装目录下找到 VMware Tools 的安装源文件 linux.iso，如下图：

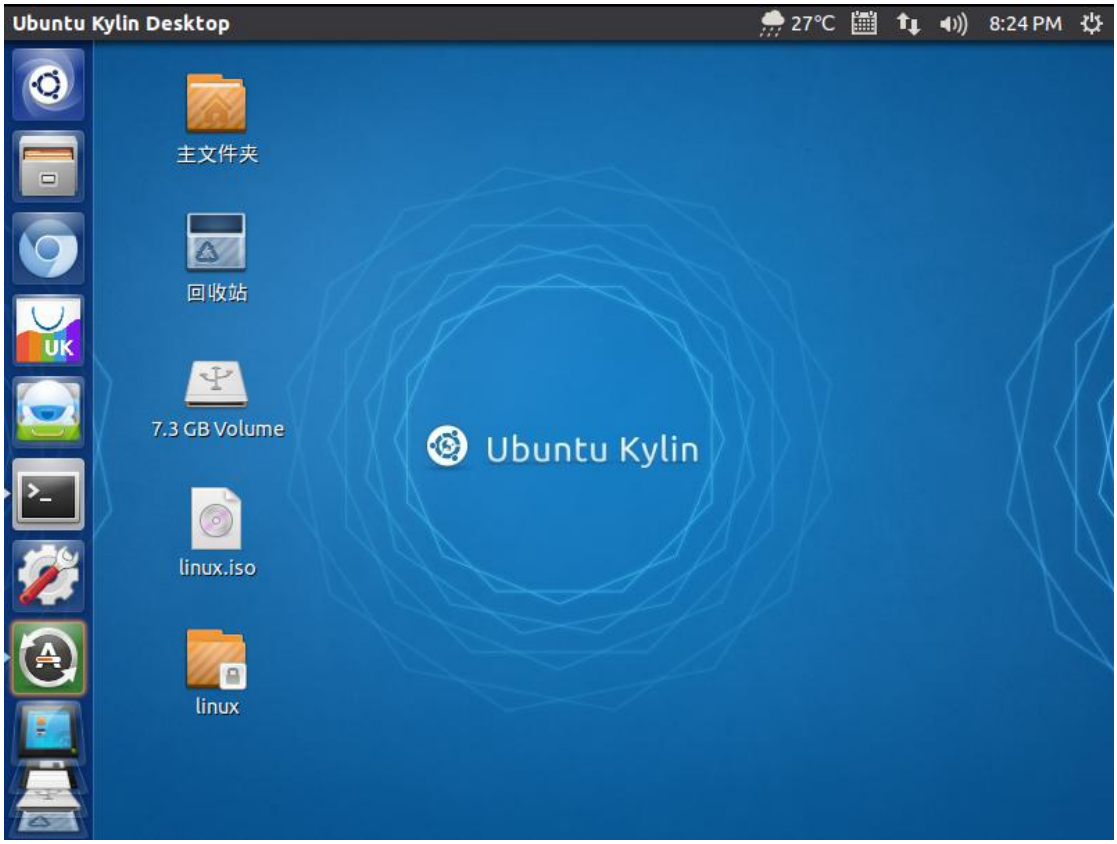


将 linux.iso 拷贝到 U 盘，再点击虚拟机->可移动设备->Kingston DataTraveler2.0，点击连接(断开与主机的连接)，如下图：

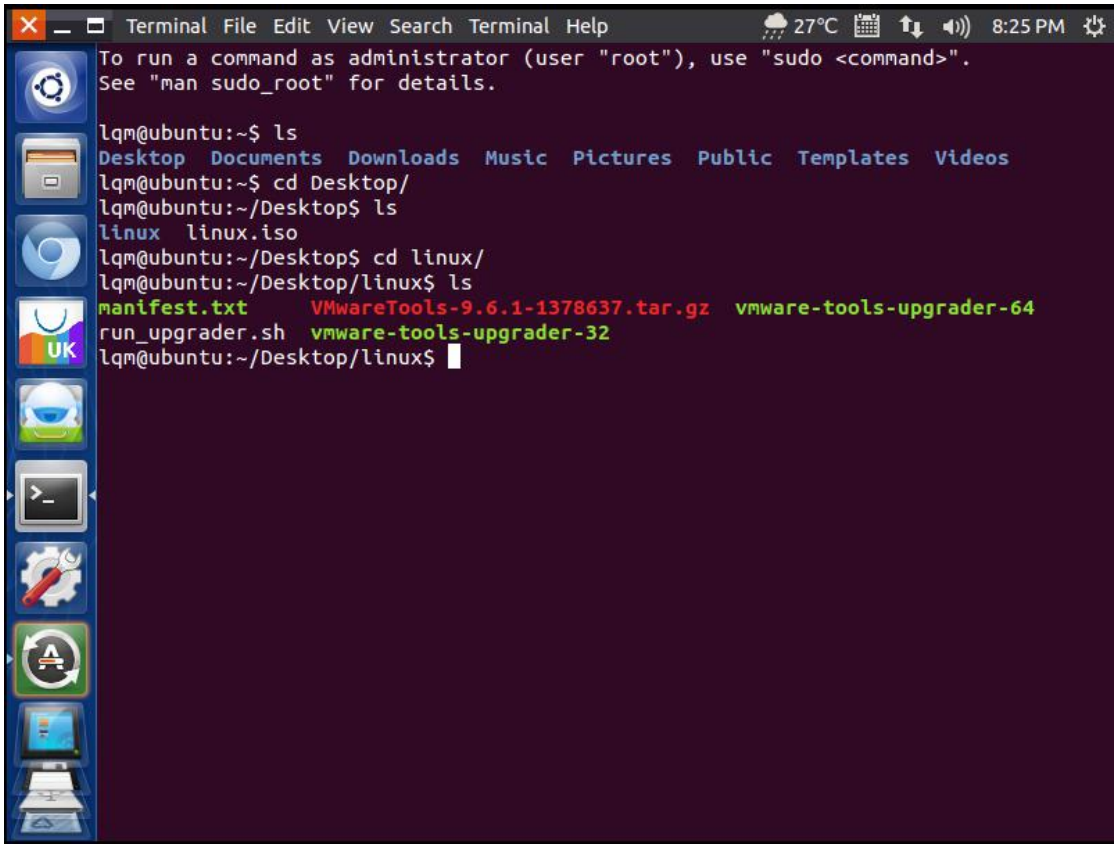


注意，Kingston DataTraveler2.0 为虚拟机发现的可以挂载的可移动设备名称，受 U 盘品牌而异。

连接成功后，在 ubuntu 下就可以识别出 U 盘了，将其中的 linux.iso 拷贝到桌面并解压，如下图：



Linux 目录即解压出来的目录，打开命令终端 ，进入该目录，



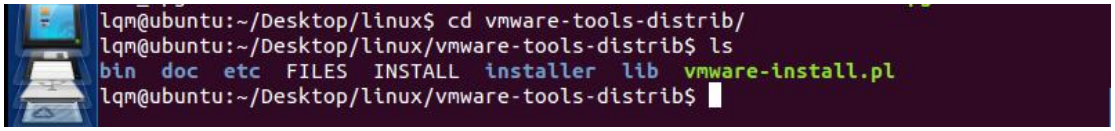
使用如下指令解压 VMwareTools：

```
sudo  tar  xvf  VMwareTools-9.6.1-1378637.tar.gz
```

解压完成后，会生成一个 vmware-tools-distrib 的目录，如下图所示：



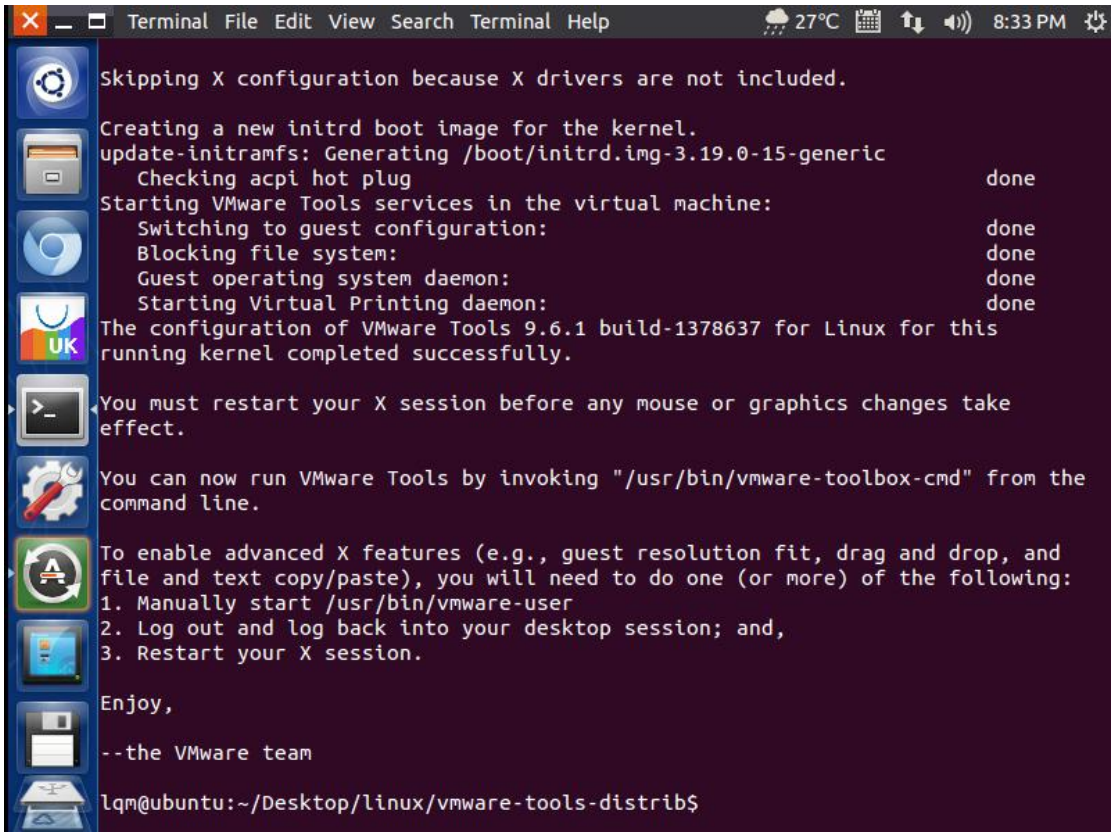
进入该目录，会发现有一个 vmware-install.pl 的文件，如下图所示：



执行如下指令即可安装 vmware-tools：

```
sudo ./ vmware-install.pl
```

这时，会弹出一系列的确认对话框，直接回车即可。安装完成后，会给出相关提示信息，如下图所示：



重启 ubuntu 系统，虚拟系统已经能和 windows 系统互相复制了。同时，屏幕也可以自适应了。

第 2 章 编译 linux+QT

2.1 安装 uboot，内核源码包

将网盘中的源码包 x6818_linux.tar.bz2 拷贝到 ubuntu 系统的用户目录并解压，会生成 x6818_linux 目录，该目录包含了 uboot，kernel 以及搭载 QT 所需要的 buildroot。

解压完成后，源码包也就安装完成了。

2.2 编译 uboot

在 x6818_linux 根目录下执行如下指令编译 uboot：

说明：默认烧写 uboot 时，需要烧写 2ndboot.bin,nsih 以及 u-boot.bin 三个文件，这里我们将三个文件打包成一个文件了，并命名为 ubootpak.bin。pak 即 package 的意思。因此，在平时调试时，我们不再需要关注 2ndboot.bin，nsih 以及 u-boot.bin 这三个文件。

```
./mk  -u
```

注意，x6818 开发板和 ibox6818 卡片电脑的 uboot 会有区别，默认 x6818 开发板使用的是 16BIT 的 DDR3，而 ibox6818 使用的是 8BIT 的 DDR3，因此 DDR 配置会有所差别。在 uboot 源码包的根目录下，存在如下几个文件，其名称及注释如下：

```
nsih.txt：uboot 编译的文件，默认为 ibox6818 卡片电脑 2GB 配置文件；
nsih-2G16b-800M.txt：x6818 开发板  2GB DDR3 的配置文件；
nsih-1G16b-800M.txt：x6818 开发板配置文件；
nsih-2G8b-800M.txt：ibox6818 2GB DDR3 的配置文件；
或者以下命名方式(ddr 芯片数量区分)：
S5P6818-CPU-800MHZ-DDR3-2PCS-1GB.txt：x6818 开发板 1G DDR 配置文件
S5P6818-CPU-800MHZ-DDR3-2PCS-2GB.txt：x6818 开发板 2G DDR 配置文件
S5P6818-CPU-800MHZ-DDR3-4PCS-1GB.txt：ibox6818 卡片电脑 1G DDR 配置文件
S5P6818-CPU-800MHZ-DDR3-4PCS-2GB.txt：ibox6818 卡片电脑 1G DDR 配置文件
备注：x6818 2 片 DDR；ibox6818 4 片 DDR。
```

默认 nsih.txt 针对 x6818 开发板，它与 nsih-1G16b-800M.txt 的内容是完全相同的。我们在编译 uboot 时，只会将 nsih.txt 编译进去，另外三个文件并不会编译进去。当编译 x6818 开发板的映像时，我们按默认参数即可；

当编译 ibox6818 的 1GB DDR3 配置套餐时，需要将 nsih-1G8b-800M.txt 替换成 nsih.txt，再编译 uboot 映像；

```
~/x6818_linux$ cp uboot/nsih-1G8b-800M.txt    uboot/nsih.txt
```

当编译 ibox6818（或者 x6818）的 2GB DDR3 配置套餐时，需要将 nsih-2G8b-800M.txt（x6818 为 nsih-2G16b-800M.txt）替换成 nsih.txt。

```
~/x6818_linux$ cp uboot/nsih-2G8b-800M.txt    uboot/nsih.txt
```

同时 uboot/board/s5p6818/x6818/include/cfg_mem.h 做如下修改

```
#define    CFG_MEM_PHY_SYSTEM_SIZE          0x80000000       /* 2GB */
/*#define    CFG_MEM_PHY_SYSTEM_SIZE          0x40000000     */   /* 1GB */ 注释本行
```

2.3 编译内核

在源码根目录下执行如下命令编译 linux 内核，编译完成后映像文件 boot.img 会释放到 out/release 目录。

```
./mk   -k
```

注意:当编译 2G 的内核时 kernel/arch/arm/plat-s5p6818/x6818/include/cfg_mem.h 做如下修改：

```
#define CFG_MEM_PHY_SYSTEM_SIZE      0x7F000000        /* 0x3F000000:Total 1GB - 2MB   0x7F000000:Total 2GB - 2MB */
/*#define CFG_MEM_PHY_SYSTEM_SIZE      0x3F000000 */       /* 0x3F000000:Total 1GB - 2MB   0x7F000000:Total 2GB - 2MB */   注释本行
```

说明：uboot 与内核的编译与 android 系统完全相同，编译时需注意 DDR 容量，x6818 开发板默认 1G，ibox6818 默认 2G。

2.4 编译文件系统

在源码根目录下执行如下命令编译包含 QT5.4 的文件系统：

```
./mk -b
```

默认编译会少一些插件或库，会弹出一些错误信息：

```
x6818_linux/buildroot/output/build/host-libtool-2.4.6/build-aux/missing: line 81: makeinfo: command not found
WARNING: 'makeinfo' is missing on your system.
```

该提示信息表明编译需要 texinfo 包，执行如下指令安装：

```
sudo   apt-get   install   texinfo
```

```
BR2_DEFCONFIG='' KCONFIG_AUTOCONFIG=/home/lqm/buildroot/output/build/buildroot-config/auto.conf KCONFIG_AUTOHEADER=/home/lqm/buildroot/output/build/buildroot-config/autoconf.h KCONFIG_TRISTATE=/home/lqm/buildroot/output/build/buildroot-config/tristate.config BR2_CONFIG=/home/lqm/buildroot/.config BR2_EXTERNAL=support/dummy-external SKIP_LEGACY= /home/lqm/buildroot/output/build/buildroot-config/conf --silentoldconfig Config.in
make[1]: Leaving directory '/home/lqm/buildroot'
You must install 'git' on your build machine
support/dependencies/dependencies.mk:28: recipe for target 'core-dependencies' failed
make: *** [core-dependencies] Error 1
```

该提示信息表明编译需要 git 包，执行如下指令安装：

```
sudo   apt-get   install   git
```

```
Your Buildroot configuration uses pre-built tools for the x86 architecture,
but your build machine uses the x86-64 architecture without the 32 bits compatibility
library.
If you're running a Debian/Ubuntu distribution, install the libc6-i386,
lib32stdc++6, and lib32z1 packages (or alternatively libc6:i386,
libstdc++6:i386, and zlib1g:i386).
For other distributions, refer to the documentation on how to install the 32 bits
compatibility libraries.
support/dependencies/dependencies.mk:28: recipe for target 'core-dependencies' failed
make: *** [core-dependencies] Error 1
lqm@ubuntu:~/buildroot$
```

该提示表明标准的 C++库没有安装，执行如下指令安装：

```
sudo apt-get install lib32stdc++-4.9-dev
```

```
lqm@ubuntu:~/buildroot$ make
gcc-linaro-arm-linux-gnueabihf-4.9-2014.09_linux.tar.xz: OK (sha256: 0cffac0caea0eb3c8bdddfa14be011ce366680f40aeddbefc7cf23cb6d4f1891)
>>> toolchain-external undefined Extracting
mkdir -p /home/lqm/buildroot/output/host/opt/ext-toolchain
xzcat /home/lqm/buildroot/dl/gcc-linaro-arm-linux-gnueabihf-4.9-2014.09_linux.tar.xz | tar --strip-components=1 --exclude='usr/lib/locale/*' -C /home/lqm/buildroot/output/host/opt/ext-toolchain  -xf -
>>> toolchain-external undefined Patching
>>> toolchain-external undefined Configuring
/home/lqm/buildroot/output/host/opt/ext-toolchain/bin/../libexec/gcc/arm-linux-gnueabihf/4.9.2/cc1: error while loading shared libraries: libz.so.1: cannot open shared object file: No such file or directory
Incorrect ABI setting: EABIhf selected, but toolchain is incompatible
package/pkg-generic.mk:146: recipe for target '/home/lqm/buildroot/output/build/toolchain-external-undefined/.stamp_configured' failed
make: *** [/home/lqm/buildroot/output/build/toolchain-external-undefined/.stamp_configured] Error 1
lqm@ubuntu:~/buildroot$
```

该提示表明缺少 libz 库，执行如下指令安装：

```
sudo apt-get install lib32z1
```

编译完成后，最终打包好的文件系统 qt-rootfs.img 存放在 out/release 目录下。它里面包含了 QT5.4 的标准库，以及一些常用的 QT 示例，在后续章节我们将会详细描述。

注意，x6818 开发板和 ibox6818 卡片电脑的文件系统源码包完全相同。

第 3 章 烧写 linux QT 映像

3.1 ubuntu 下 fastboot 的安装

3.1.1 安装 fastboot

执行如下指令安装 fastboot:

```
sudo apt-get install android-tools-fastboot
```

3.1.2 新建 51-android.rules

新建 51-android.rules 文件，内容如下：

```
# adb protocol on passion (Nexus One)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e12", MODE="0666",
OWNER="lqm"
# adb protocol on crespo/crespo4g (Nexus S)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e22", MODE="0666",
OWNER="lqm"
# fastboot protocol on crespo/crespo4g (Nexus S)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e20", MODE="0666",
OWNER="lqm"
# fastboot protocol on stingray/wingray (Xoom)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="708c", MODE="0666",
OWNER="lqm"
# fastboot protocol on maguro/toro (Galaxy Nexus)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e30", MODE="0666",
OWNER="lqm"
# fastboot protocol on x210/x4412/x4418
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="0002", MODE="0666",
OWNER="lqm"
```

注意，OWNER 里面填的”lqm”务必换成自己 ubuntu 系统的用户名。之后将 51-android.rules 文件复制到/etc/udev/rules.d/ 目录下。

到此，就可以使用 fastboot 更新映像了。如果不建 51-android.rules 这个文件，使用 fastboot 更新时需要使用 root 权限。

3.2 ubuntu 下制作启动卡

第一步：准备一张不小于 2GB 的 TF 卡，并插到装有 ubuntu 操作系统的 PC 机上；

第二步：删除 TF 卡的所有分区。

在 Linux 的终端窗口，使用 fdisk /dev/sdb 命令删除原来所有分区，sdb 为系统为 TF 卡分配的设备节点。注意，具体由节点名称而定，有可能是 sdc,sde 等。使用如下指令查询设备节点：

```
cat /proc/partitions
```

示例如下：

```
[root@lqm mass-production]# cat /proc/partitions
major minor  #blocks  name
```

```
   8        0   36700160 sda
   8        1     512000 sda1
   8        2   36187136 sda2
 253        0   34144256 dm-0
 253        1    2031616 dm-1
   8       16    3879936 sdb
   8       17    3875840 sdb1
[root@lqm mass-production]#
[root@lqm mass-production]# fdisk /dev/sdb

WARNING: DOS-compatible mode is deprecated. It's strongly recommended to
         switch off the mode (command 'c') and change display units to
         sectors (command 'u').

Command (m for help): d
Selected partition 1

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.

WARNING: Re-reading the partition table failed with error 16: 设备或资源忙.
The kernel still uses the old table. The new table will be used at
the next reboot or after you run partprobe(8) or kpartx(8)
Syncing disks.
[root@lqm mass-production]#
```

输入 d，表示删除分区，输入 w 表示保存已经修改的分区信息。至此，原/dev/sdb1 被删除。拔掉 TF 卡，再插入 PC 机上，查询设备节点：

```
[root@lqm mass-production]# cat /proc/partitions
major minor  #blocks  name

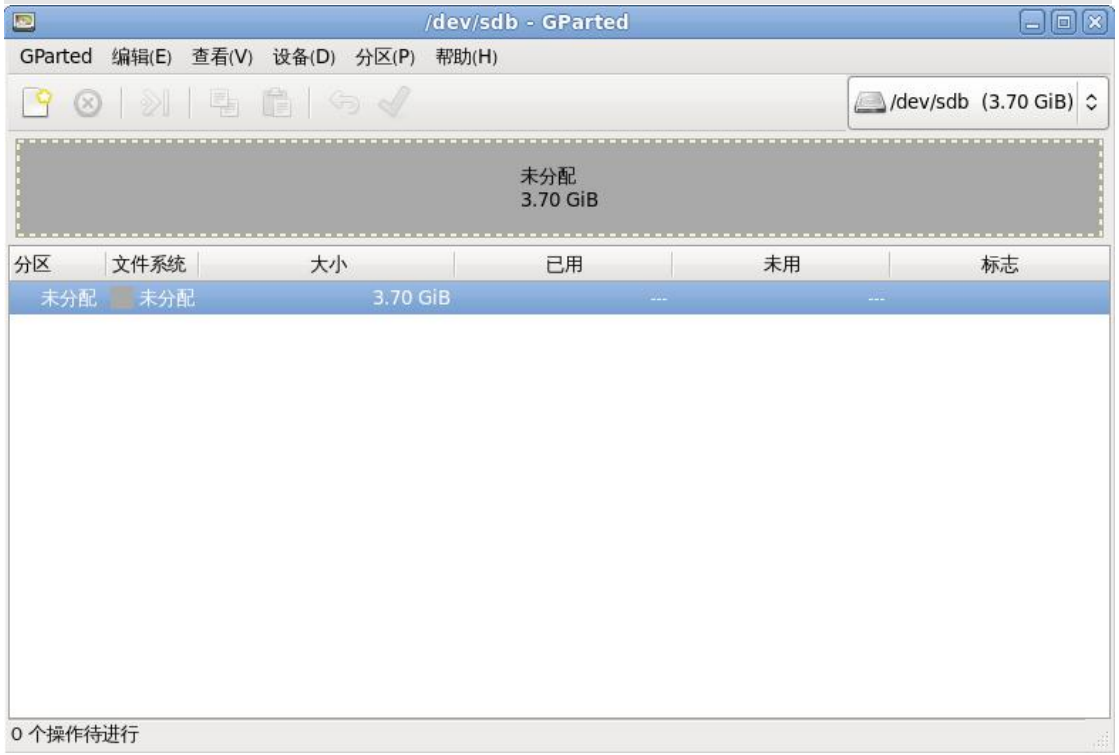
   8        0   36700160 sda
   8        1     512000 sda1
   8        2   36187136 sda2
 253        0   34144256 dm-0
 253        1    2031616 dm-1
   8       16    3879936 sdb
[root@lqm mass-production]#
```

注意必须拔掉后再插入，否则仍然会提示存在/dev/sdb1 节点，会造成出错。

第三步：使用 gparted 工具给 TF 卡预留 256M 空间，用于存放 uboot 映像。

使用如下命令打开 TF 卡分区表：

```
gparted  /dev/sdb
```



选择分区->新建，预留 256M 空间给 uboot，剩下的分区使用 fat32 格式，如下图所示：



点击添加，选择菜单中的应用全部操作，完成 TF 卡的分区。

第四步：进入映像生成目录，即 out/release 目录，执行如下指令烧写 uboot 到 TF 卡：

```
sudo  ./x6818-sdmmc.sh  /dev/sdb  ubootpak.bin
```

注意，这里/dev/sdb 为 TF 卡的节点，该节点为 linux 系统自动分配，也有可能为 sdc,sde 等，用户可查询节点名称后再执行上面的烧写脚本。

正常的执行界面如下：

```
jason@ubuntu:~$ sudo ./s5p6818-sdmmc.sh /dev/sdb ubootpak.bin
[sudo] password for jason:
594+1 records in
595+0 records out
304640 bytes (305 kB) copied, 0.485139 s, 628 kB/s
^_^ The image is fused successfully
jason@ubuntu:~$
```

这时，该 TF 卡就可以引导开发板启动 uboot 了。

注意：以上操作只是制作了一张可以引导启动的 SD 卡，如果需要制作升级卡，请查看后面 TF 卡脱机刷机章节。

3.3 ubuntu 下烧写映像文件

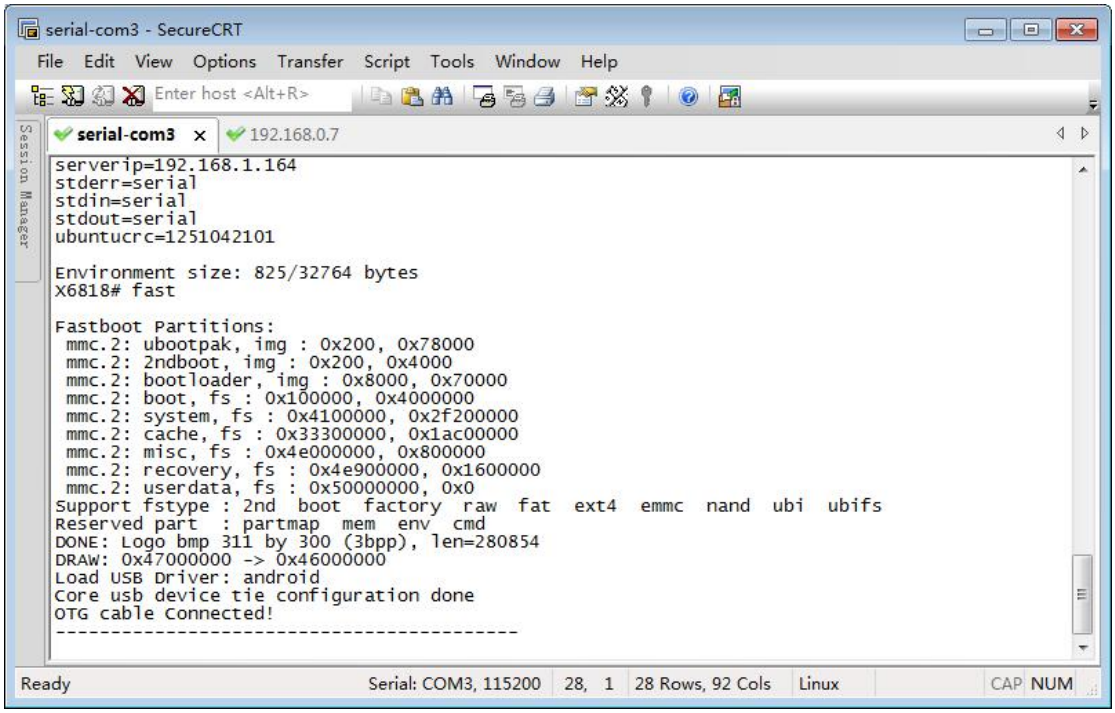
3.3.1 emmc 中无固件

在 emmc 中没有映像文件时，我们可以将 7.3 节制作的 TF 卡插到开发板的卡槽 0，开机即从 TF 卡启动 uboot，然后执行 fastboot 命令更新映像。

第一步：将 uboot 映像烧写到 TF 卡，详见上 7.3 节；

第二步：将 TF 卡插到开发板的 SD0 通道卡槽，即右侧卡槽；

第三步：连接调试串口，打开串口终端，并按住 POWER 键开机，在 uboot 三秒倒计时按下空格键进入 uboot 命令行，敲入 fastboot 指令或 fast，插入 micro USB 线连接到电脑，



这时，开发板上会显示一幅等待升级的画面，并伴随有相应的字符提示。

第六步：另外开启一个 ubuntu 终端，使用如下指令更新映像：

```
fastboot flash ubootpak ubootpak.bin        (指令一)
fastboot flash boot boot.img                (指令二)
fastboot flash system qt-rootfs.img         (指令三)
```

每执行一条指令，在液晶屏上都会有相应的界面提示，用户可以很清晰的观察升级的状

态。执行完以上指令，还不能正常启动 linux+qt 系统，必须修改完环境变量才可正常启动。

注意：

指令一等价于如下两条指令：

```
fastboot  flash  2ndboot        2ndboot.bin
fastboot  flash  bootloader     u-boot.bin
```

指令二理论上等价于如下两条指令：

```
fastboot  flash  kernel     uImage
fastboot  flash  ramdisk    ramdisk.img
```

但是实际上我们已经在 uboot 分区中去掉了 kernel 和 ramdisk，因此用户需要更新内核或 ramdisk 时，更新 boot.img 即可。

第七步：修改环境变量，键盘使用组合键“ctrl + C”取消 fastboot，执行以下指令后重启开发板即可进入 linux+qt 系统。

```
setenv bootcmd "ext4load mmc 2:1 0x48000000 uImage;bootm 0x48000000"
setenv bootargs "lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4"
save
```

3.3.2 emmc 有正常启动的映像

当开发板中有正常启动的映像时，则无需启动卡，即省略 3.3.1 中第一，第二步，从第三步开始操作。

3.4 windows 下 fastboot 的安装

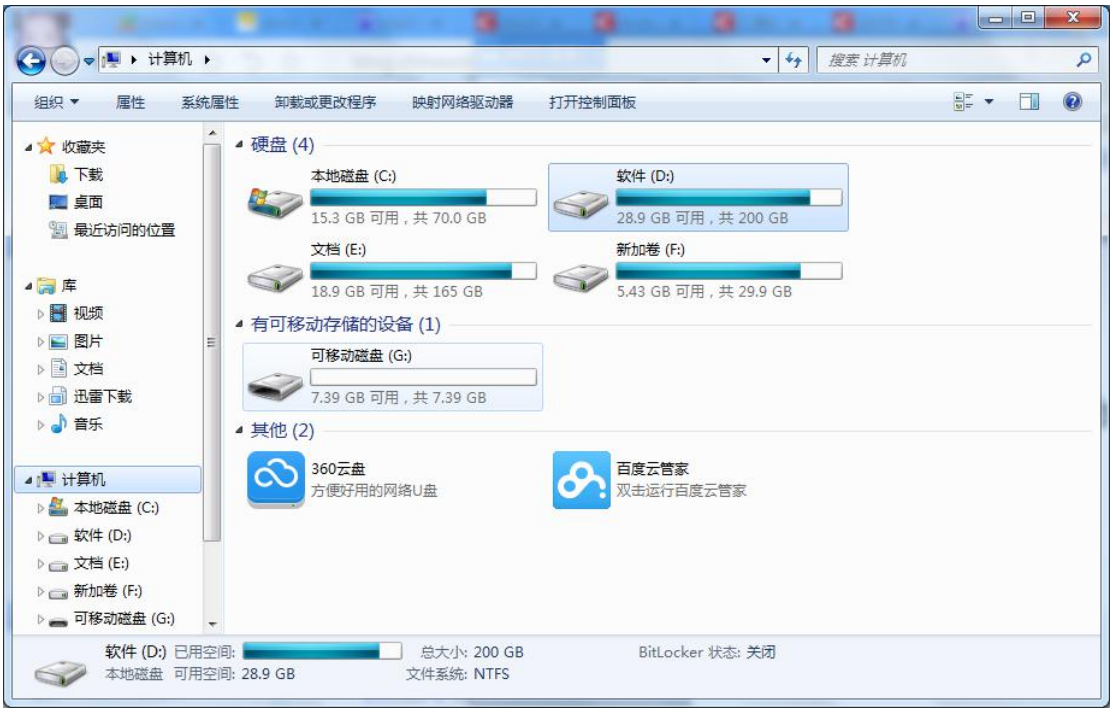
将资料 A 盘 tools 目录中的 fastboot.rar 文件解压到 C:\Windows\System32 和 C:\Windows\SysWOW64（winXP 无此目录）目录即完成安装，在命令提示符任意路径下敲 fastboot 都可以使用 fastboot。

3.5 windows 下制作启动卡

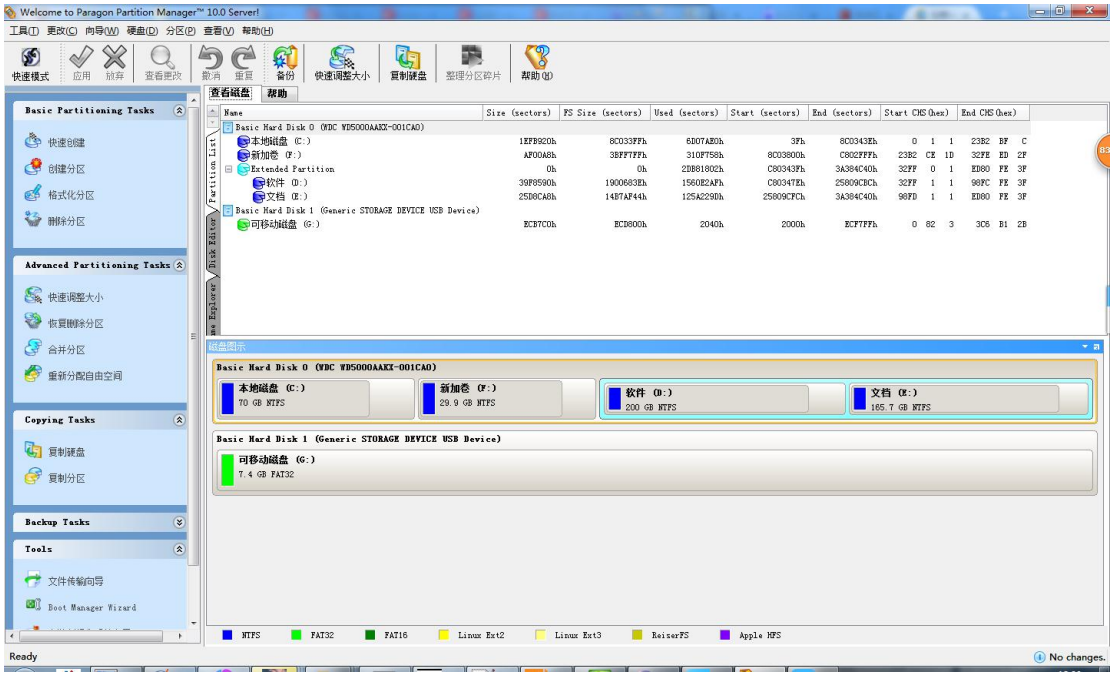
在 windows 下烧写映像文件，和在 ubuntu 下的很相似，只不过在 ubuntu 下通过 minicom 查看串口打印信息，而在 windows 下通过 secureCRT 或超级终端查看串口打印信息；在 ubuntu 下通过命令行终端敲 fastboot 指令，而在 windows 下通过 cmd 命令行敲 fastboot 指令。

在 WINDOWS 下可使用 PartitionManager 工具，详细操作步骤如下：

1.TF 卡通过读卡器连上电脑，如下图



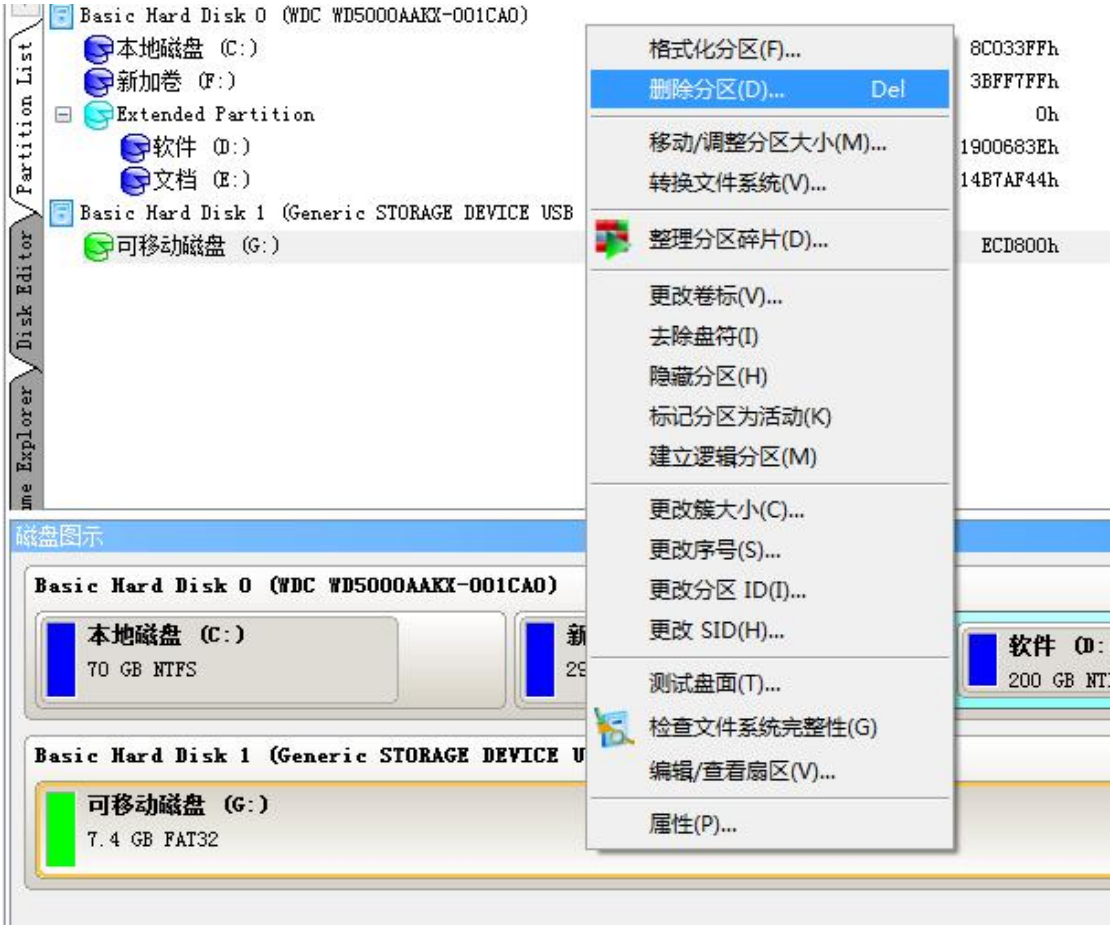
2.打开 PartitionManager.exe 分区工具(x6818_A\tools 目录)，界面如下图（也可以使用其他同类分区工具）：



没有经过任何操作的 TF 卡在工具中如下图，绿色方格（可移动磁盘 G）：



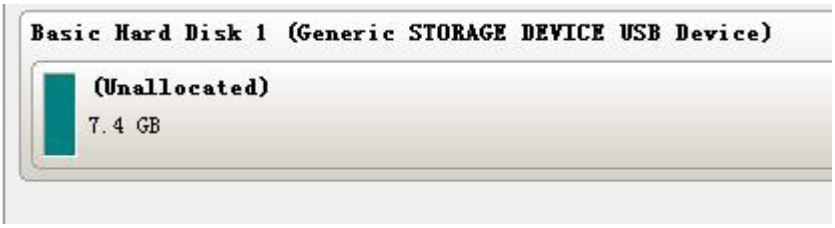
3.将 TF 卡分区删掉，鼠标右键 TF 卡的任意区域，在弹出菜单中选择删除分区：



弹出确认提示窗选择是：



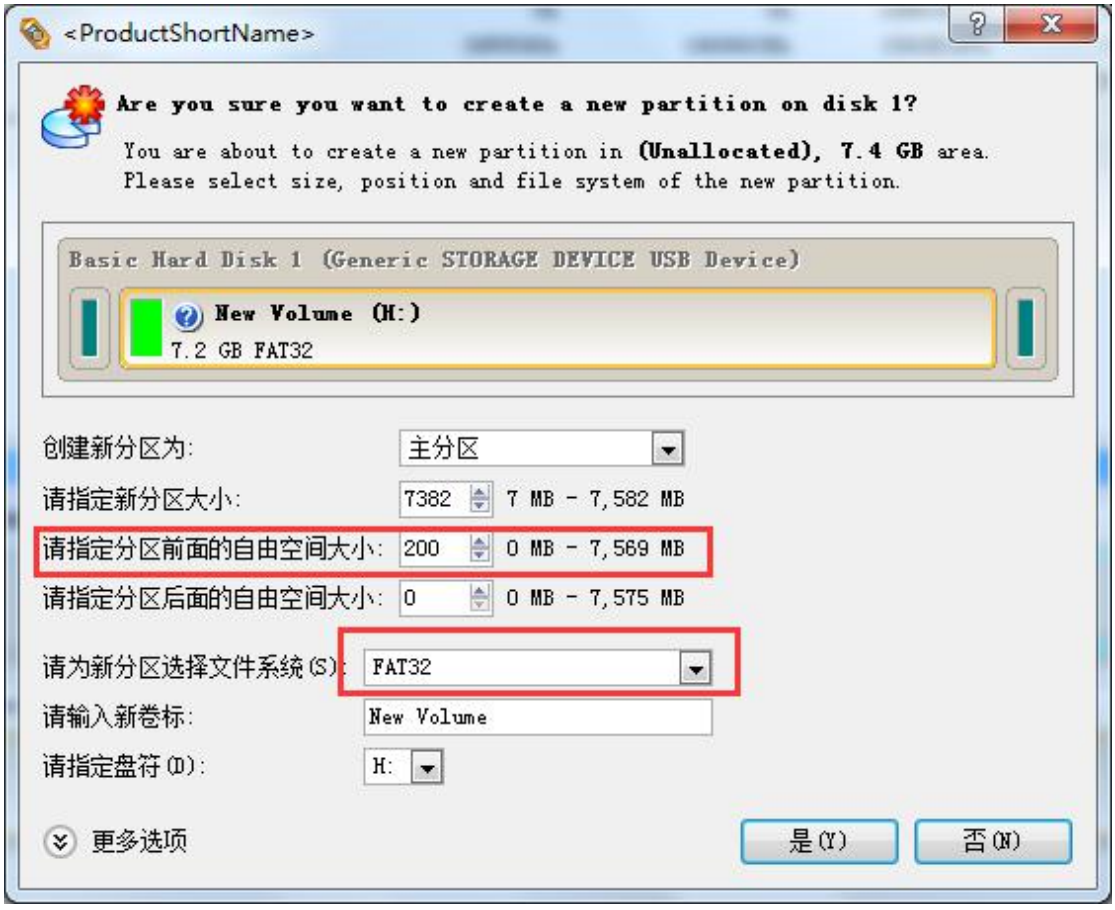
确定删除之后方格颜色变成深绿色，名称变为 Unallocated 即未分配的分区：



4.重新创建分区，鼠标右键，选择创建分区：



在弹出的窗口中改修两个地方：**分区前面自由空间的大小 100-200M，新分区的文件系统 FAT32**，点击是：

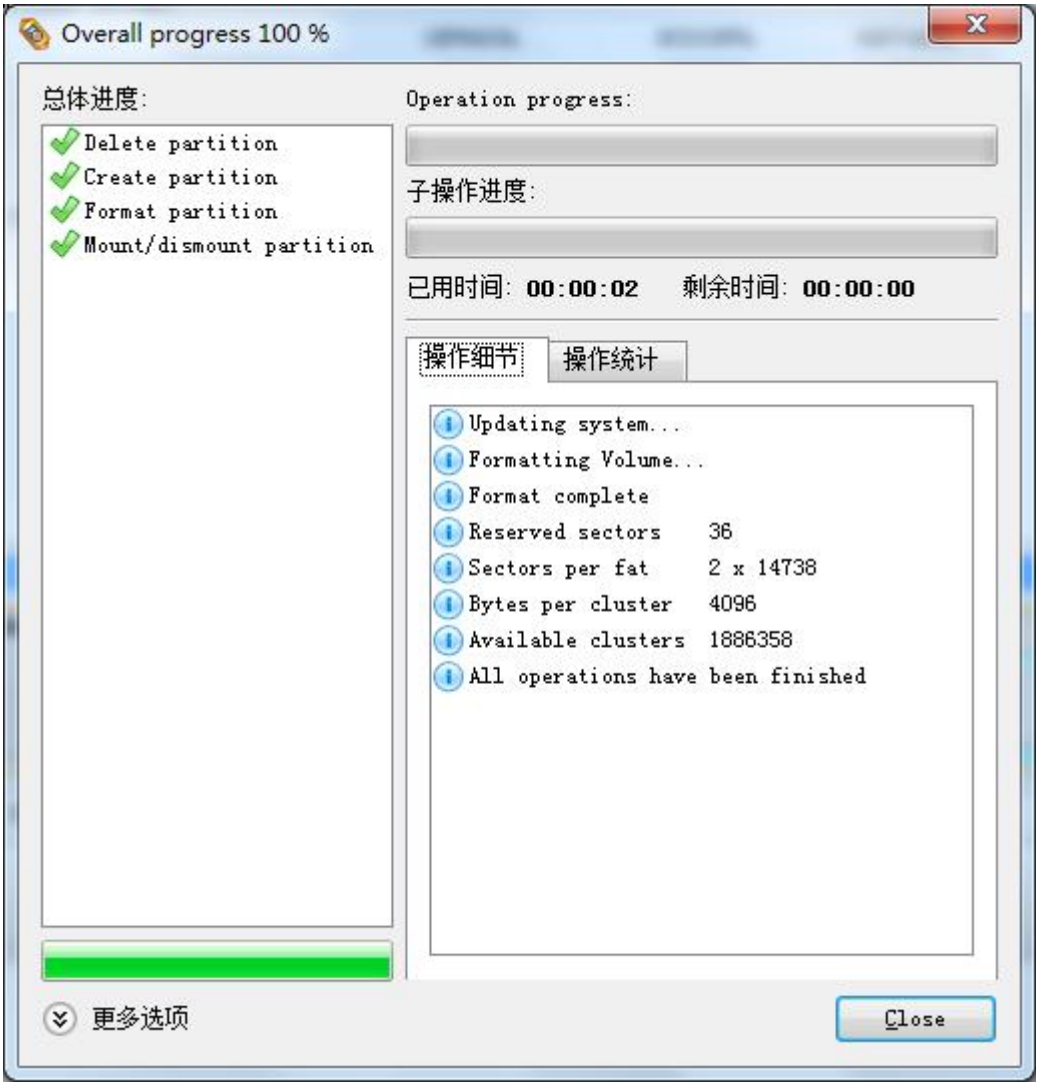


那么 U 盘的分区图示变成下面两个区域，一个 200M 未分配，一个 FAT32 的可见分区:



最后，点击软件左上角应用按钮完成分区。





注：以上的分区操作，只要不破坏 tf 卡的分区，只用做一次就好了，后面刷系统只要放入对应的系统映像就好了。

5.使用九鼎提供的 iRomfusing 工具把 uboot 烧写到预留分区（这里是把 uboot 烧写到前文所讲分区时的未分配空间，所以工具烧写完成后在系统 U 盘目录下是看到的是一个空的目录）。



工具界面如上图，选择 TF 卡盘符，点击 Browse 后在弹出窗口中选择要使用的 uboot，然后点击 ADD，在 PATH 下会增加一条记录，最后点击 START 烧写，OK 后提示 Fusing image done，则启动卡制作完成。



3.6 windows 下 fastboot 烧写映像文件

在 windows 下烧写映像文件，和在 ubuntu 下的很相似，只不过在 ubuntu 下通过 minicom 查看串口打印信息，而在 windows 下通过 secureCRT 或超级终端查看串口打印信息；在 ubuntu 下通过命令行终端敲 fastboot 指令，而在 windows 下通过 cmd 命令行敲 fastboot 指令。

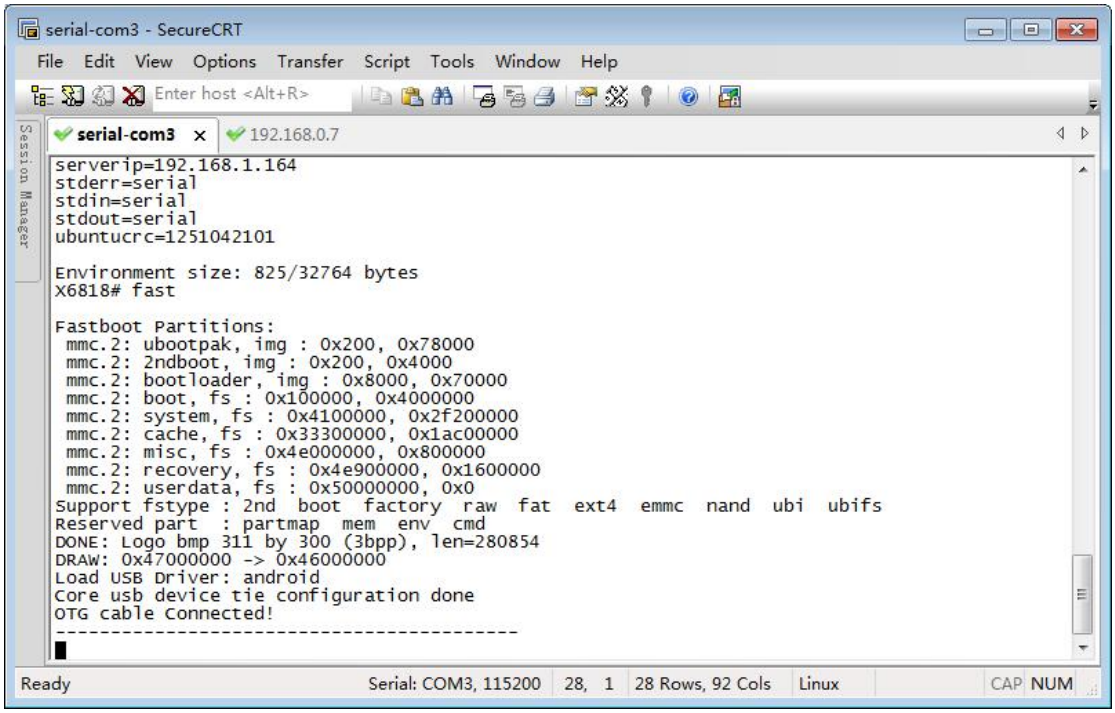
第一步：按照 3.5 讲述制作启动卡；

第二步：将 TF 卡插到开发板的 SD0 通道卡槽，即右边卡槽；

注意：如果 EMMC 中存在可以正常启动的 uboot，第一，第二步可忽略，即无需启动

卡直接上电启动。

第三步：长按开发板开机键启动，在串口终端会有打印信息，在 3 秒倒计时内按空格键，进入 uboot 命令行，敲入 fastboot 指令，会列出分区表信息：



第四步：打开 cmd 命令行终端，使用如下指令更新映像：

```
fastboot flash ubootpak ubootpak.bin (指令一)
fastboot flash boot boot.img (指令二)
fastboot flash system qt-rootfs.img (指令三)
```

第五步：修改环境变量，键盘“ctrl + C”取消 fastboot，执行以下指令后重启开发板即可进入 linux+qt 系统。

```
setenv bootcmd "ext4load mmc 2:1 0x48000000 uImage;bootm 0x48000000"
setenv bootargs "lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4"
save
```

3.7 使用 TF 卡脱机更新映像文件

x6818 linux 系统支持使用 TF 卡更新映像文件。

第一步：参考前文讲述制作启动卡章节，制作一张 TF 启动卡；

第二步：在 TF 卡根目录下建立名为 x6818-qt 的目录，并将 ubootpak.bin,boot.img，以及 qt-rootfs.img 三个文件拷贝到 x6818-qt 目录，同时在 x6818-qt 目录下新建 env.txt 文件，编辑内容如下：

```
**************************linux3.4.39 & qt5.4 ****************************
bootcmd=ext4load mmc 2:1 0x48000000 uImage;bootm 0x48000000
bootargs=lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4
ubootpak=1
boot=1
system=1
*************** web:www.9tripod.com tel:0755-33133436 ********************
```

需要更新的映像设置为 1，不需要更新的映像设置为 0 即可。

第三步：将 TF 卡插到开发板的 TF 卡槽，按下开机键，这时系统将会自动升级。

安静数秒，开发板更新完毕将会自动重启，进入 qt 系统。

注意：

1.默认 uboot 会自动判断 TF 卡的 6818-qt 目录中的映像文件是否有更新，一旦发现映像和烧写到开发板上的不同，则会自动更新映像，否则不会再次更新。

2.如果需要重复更新，上电时按下返回键（K1）即可强制进入升级。

3.在更新 linux_qt 系统时，确保在 TF 卡根目录不存在 x6818-android 和 x6818-ubuntu 两个目录，否则将会导致更新失败。

3.8 emmc 中不存在 bootloader 时更新 uboot

当 emmc 中不存在 bootloader 时，需要制作启动卡。将 uboot 烧写到 TF 卡后，将 TF 卡插到开发板的右侧卡槽，即 SD0 通道卡槽，uboot 将会从 SD 卡启动。再通过 fastboot 更新映像，uboot 将会烧写到 emmc 中。

3.9 emmc 中存在 bootloader 时更新 uboot

当 emmc 中存在 bootloader 时，无需 TF 卡，直接开机启动 uboot，再通过 fastboot 更新映像，uboot 将会烧写到 emmc 中。

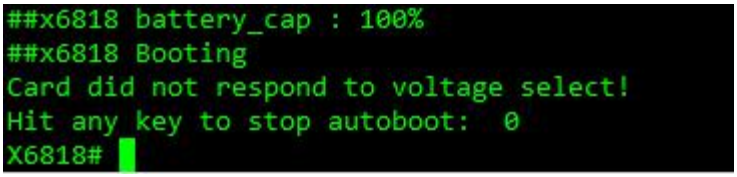
3.10 uboot 环境变量设置

针对 android，linux，ubuntu 三种不同性质的操作系统，环境变量会有所不同，它们表现在 bootargs 和 bootcmd 两个变量上。针对同一种操作系统，如 android，或 linux，或 ubuntu，如果对应不同的外设，如 LCD，触摸屏，摄相头等，则它只表现在 bootargs 这个环境变量上。

本章节仅讲述 linux_qt 系统的环境变量设置，无论外设如何变化，变量 bootcmd 都不会改变，只需做如下设置即可：

1.接上调试串口（UART0）

2.重启开发板，在 uboot 启动串口打印 3 秒倒计时时按任意键取消启动进入 uboot 命令行，如下图



3.使用如下命令修改环境变量：

Linux+qt 系统 bootcmd（其他系统环境变量查阅在网盘 x6818A\image\x6818-xxx 各系统命名目录下 env.txt 中）：

```
setenv bootcmd " ext4load mmc 2:1 0x48000000 uImage;bootm 0x48000000"
save
```

标清屏（800x480）：

```
setenv bootargs "lcd= at070tn92 tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4"
save
```

高清屏（1024x600）：

```
setenv bootargs "lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4"
save
```

MIPI 屏（1024x600）：

```
setenv bootargs "lcd=wy070ml tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4"
save
```

VGA-1024x768:

```
setenv bootargs "lcd=vga-1024x768 tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4"
save
```

VGA-1440x900:

```
setenv bootargs "lcd=vga-1440x900 tp=gslx680-linux root=/dev/mmcblk0p2 rw rootfstype=ext4"
save
```

第 4 章 QT 文件系统的搭建

前面章节的介绍，都是九鼎创展工程师已经移植的文件系统包，如果换一个平台，一切从零开始，我们如何构建 linux QT 文件系统呢？本章节将会带您一步步搭建 linux 文件系统。

4.1 下载 buildroot

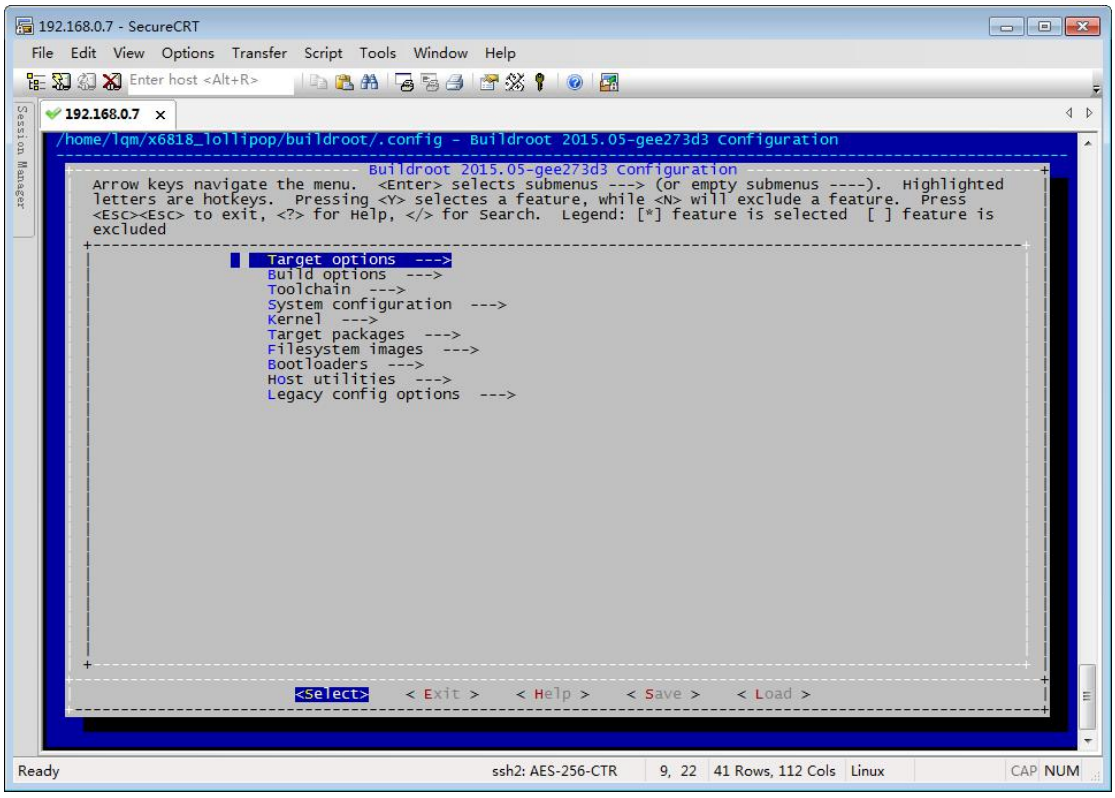
在 buildroot 官网下载 buildroot 包，下载地址如下：

http://buildroot.uclibc.org/download.html

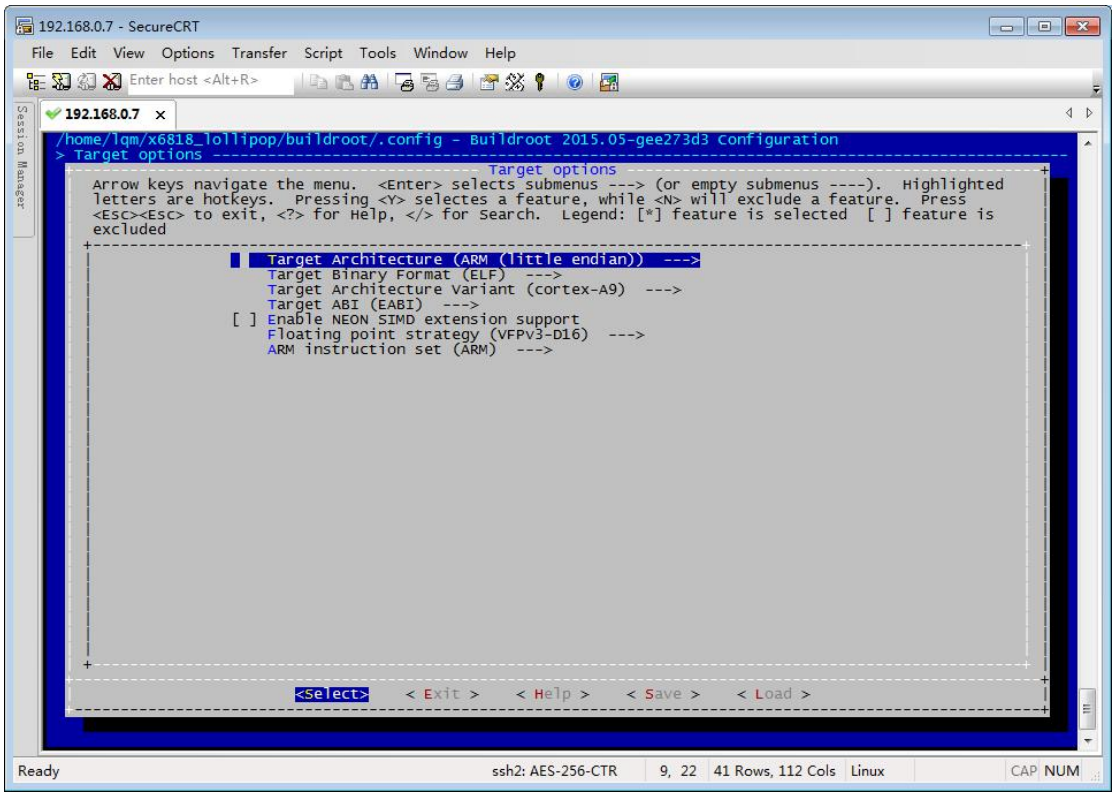
本文使用的是 buildroot 2015.05，用户下载最新版本与本文介绍版本配置可能略有不同，根据下文配置适当修改即可。

4.2 配置 buildroot

将下载的 buildroot 包拷贝到 ubuntu 系统用户目录并解压，得到 buildroot 目录，通过命令终端进入 buildroot 目录，执行 make menuconfig，进入配置界面：



进入 Target options 菜单，



在 Target Architecture 中选择 ARM(little endian)，

在 Target Binary Format 中选择 ELF，

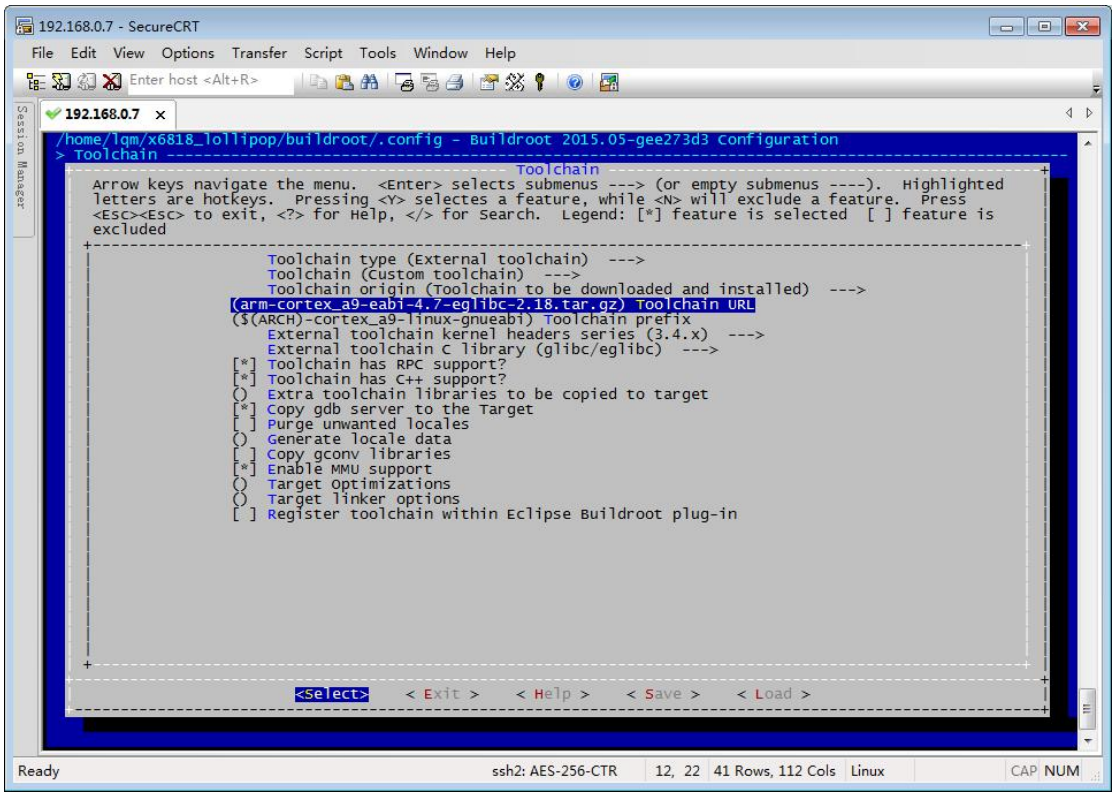
Target Architecture Variant 中选择 cortex-A9，

Target ABI 选择 EABI，

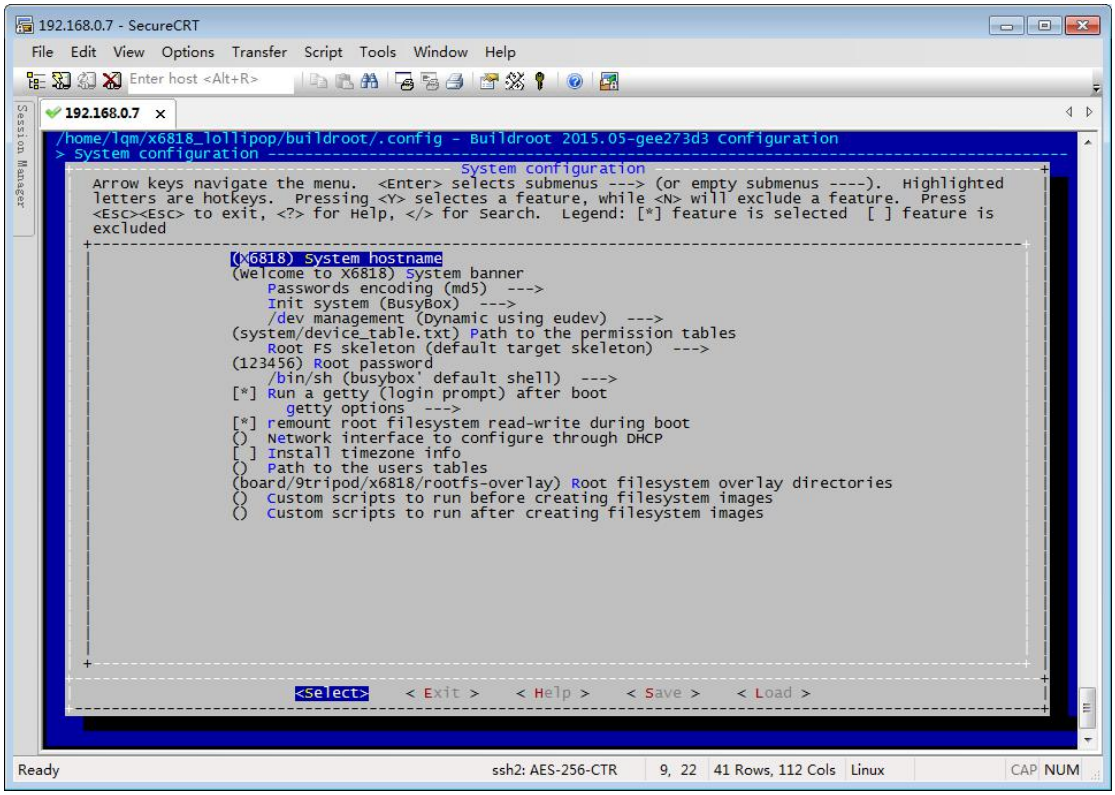
Floating point strategy 选择 VFPv3-D16，

ARM instruction set 选择 ARM。

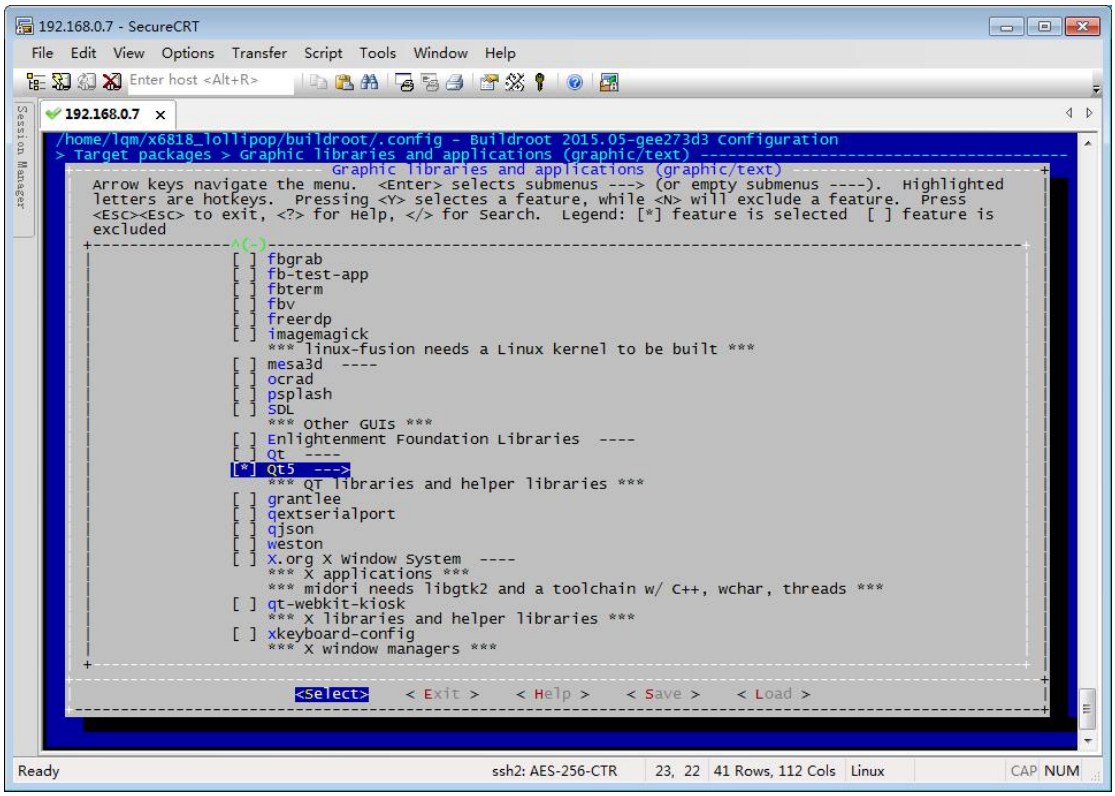
退回上一级，进入 Toolchain 目录，按下图配置：



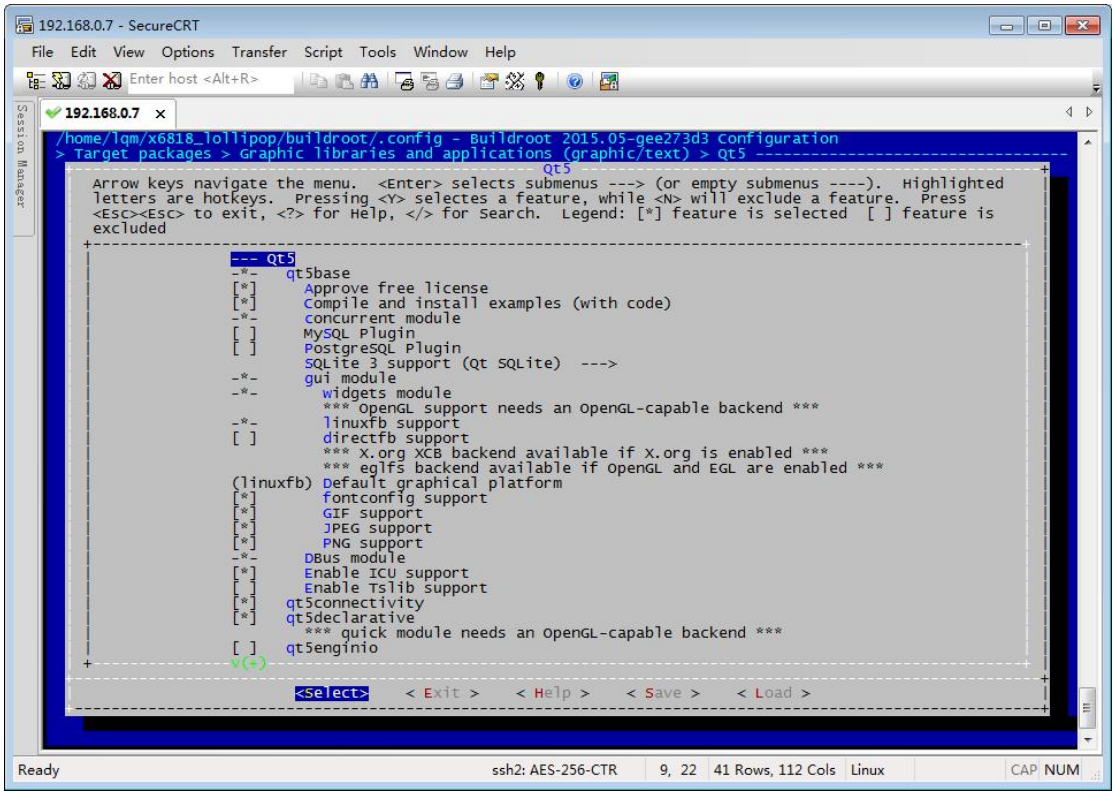
退回上一级，进入 System configuration 目录，作如下配置：

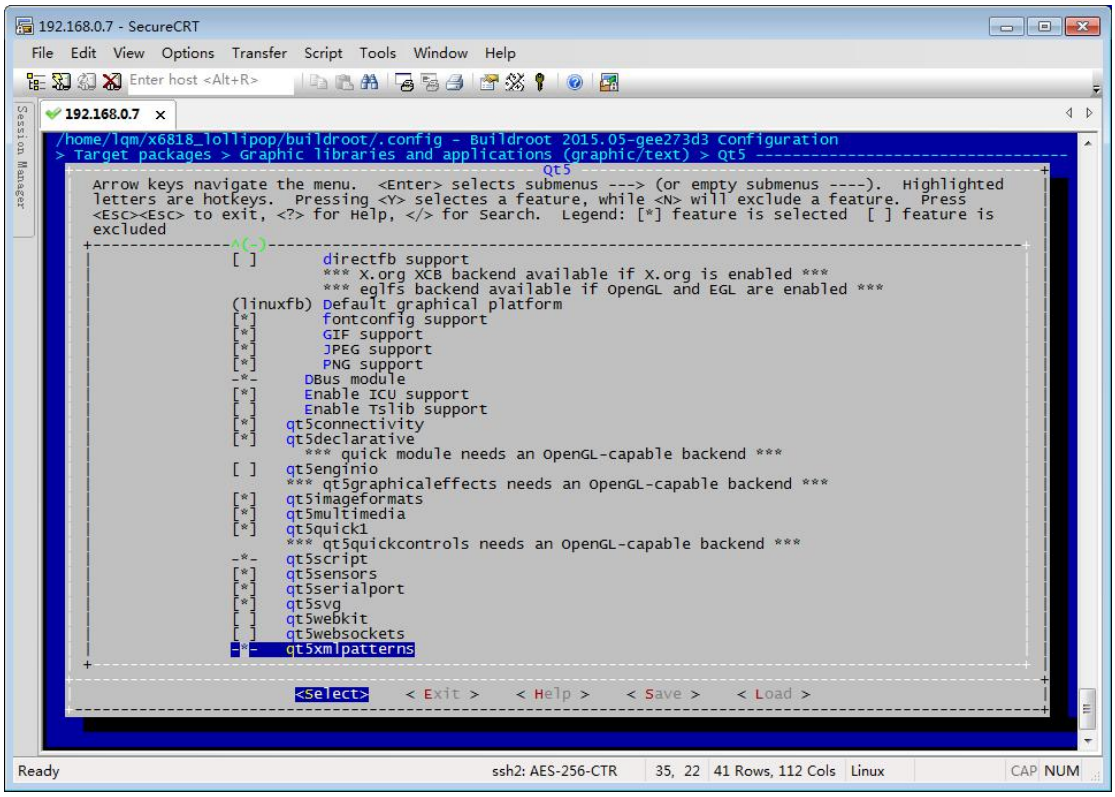


退回上一级，进入 Target packages 目录，再进入 Graphic libraries and applications (graphic/text)目录，选择 Qt5，如下图所示：

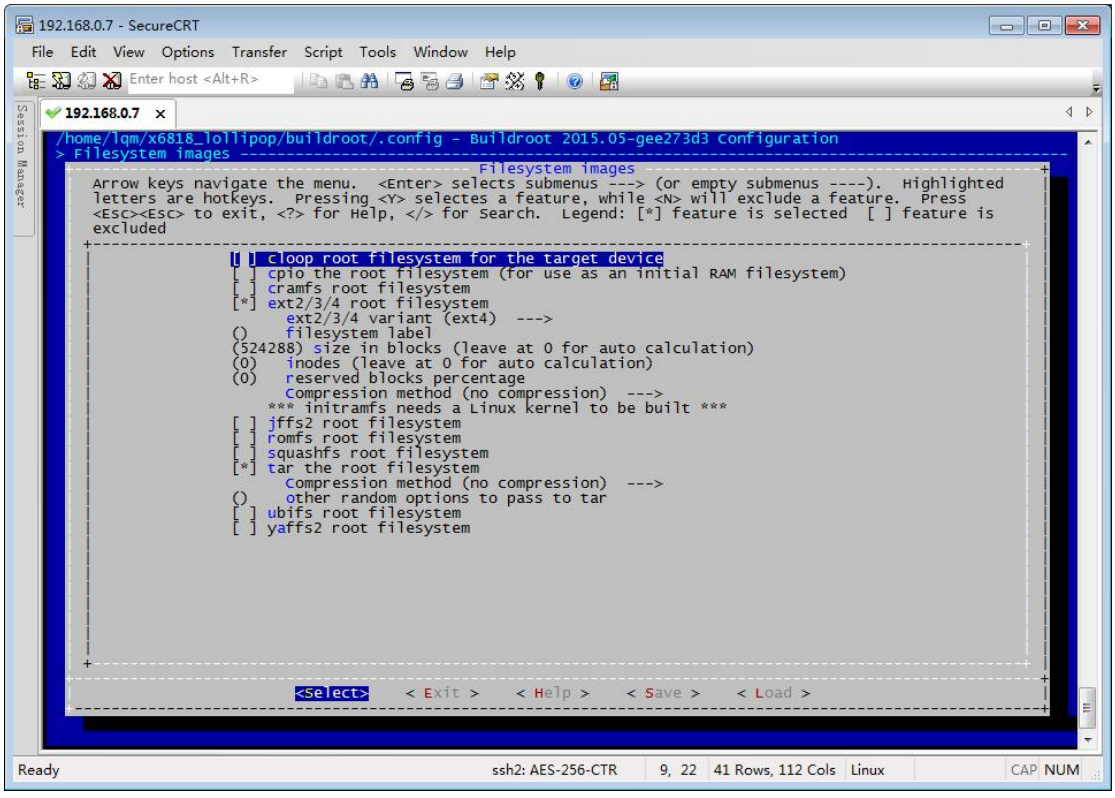


注意不要选 Qt，它对应 QT4.8 版本；进入 Qt5 菜单，按下图配置：





退回 menuconfig 的开始界面，进入 Filesystem images 菜单，作如下配置：



到此，buildroot 配置完成。默认配置保存在 buildroot 根目录的.config 中，我们可以备份该配置文件，以防后续配置出错。执行如下指令备份配置文件：

```
cp  .config  x6818_config
```

4.3 编译 buildroot

配置完成后，执行 make 指令即可编译 buildroot 了。编译 buildroot 会依赖一些第三方插件和库，在 QT5.4 移植章节的编译文件系统小节中，已经给出了需要安装的包，在编译之前需要提前安装，否则会报错。 编译完成后，文件系统映像 rootfs.ext2（实际为 ext4 格式）会生成到 output/images 目录。

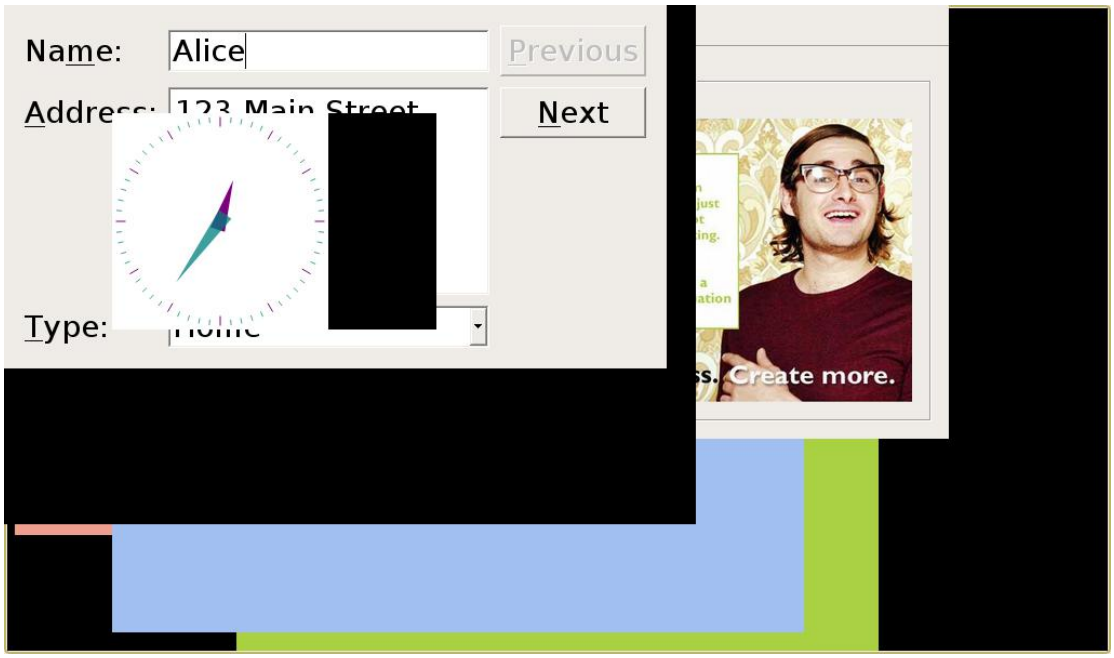
4.4 测试 QT5.4 默认示例

将 uboot，内核，文件系统烧写进开发板，进入 linux 文件系统后，可以进入 /usr/lib/qt/examples 目录测试 QT 示例。

进入 gui/analogclock 目录，执行 analogclock 文件，指令如下：

```
./analogclock &
```

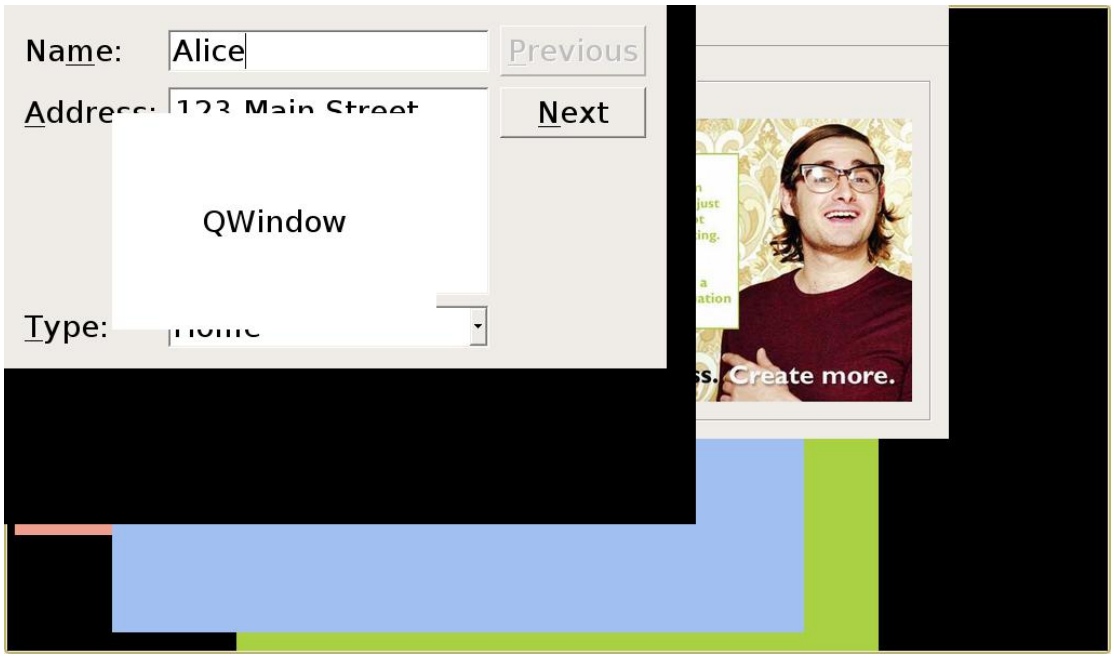
这时，在开发板上可以看到有一个时钟图案被绘制出来，如下图所示：



进入 gui/rasterwindow 目录，执行 rasterwindow 文件，指令如下：

```
./ rasterwindow &
```

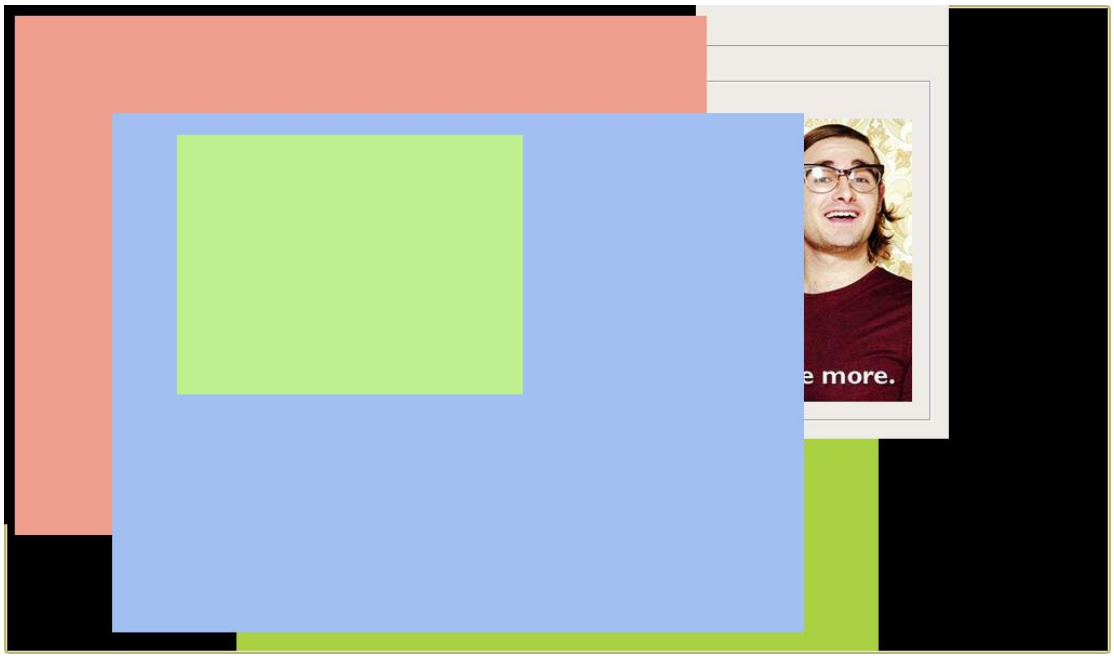
这时，在开发板上可以看到一个标注有 QWindow 的图案被绘制出来，如下图所示：



进入 qpa/windows 目录，执行 windows 文件，指令如下：

```
./windows &
```

这时，在开发板上可以看到有三幅图案被绘制出来，如下图所示：



进入 sql/drilldown 目录，执行如下指令：

```
./drilldown &
```

这时，在开发板上可以看到有四个 QT 画面被绘制出来，如下图所示：



进入 sql/books 目录，执行如下指令：

```
./books &
```

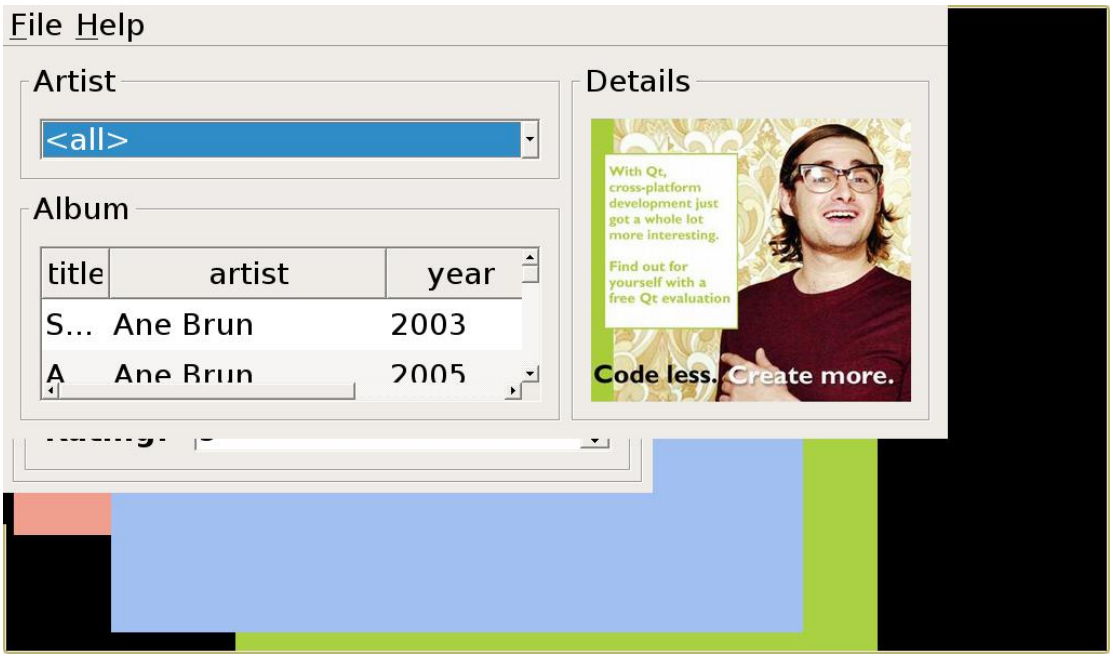
这时，在开发板上可以看到有一个对话框被绘制出来，如下图所示：



进入 sql/masterdetail 目录，执行如下指令：

```
./masterdetail &
```

这时，在开发板上可以看到有一个对话框绘制出来，如下图所示：



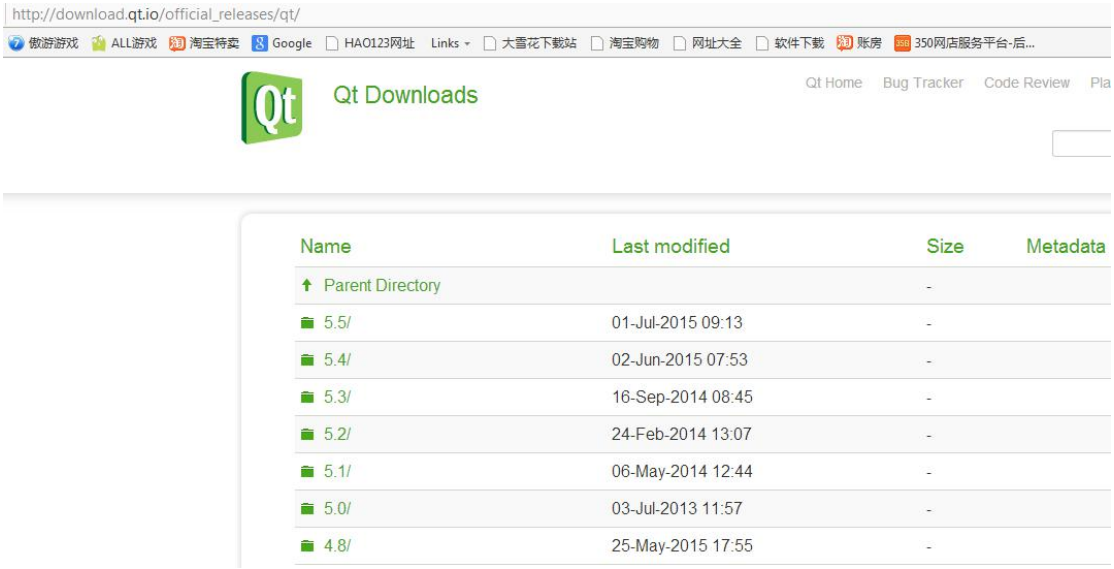
默认 buildroot 编译出了很多示例，这里不带一一列举，有兴趣的读者可以自行尝试。

4.5 安装 QT Creator

通常我们使用 QT Creator 创建基于 QT 的工程。在 QT 官网下载最新的 QT 安装包，下载地址如下：

http://download.qt.io/official_releases/qt/

打开链接页面如下：



点击 5.5，下载最新的安装包，得到名为 qt-opensource-linux-x64-5.5.0-2.run 的文件，将它拷贝到 ubuntu 的用户目录，使用如下指令安装：

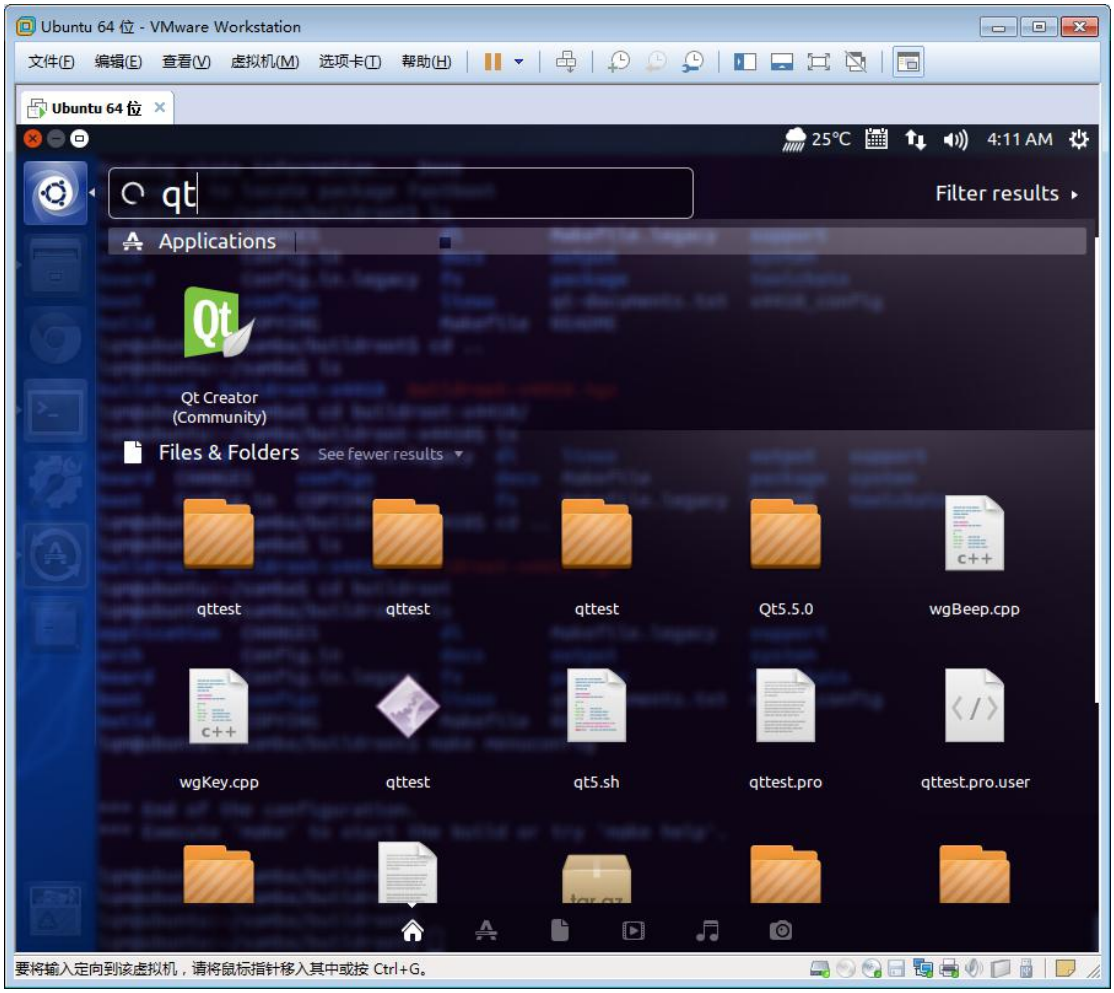
```
./ qt-opensource-linux-x64-5.5.0-2.run
```

安装完成后，QT Creator 也就安装完成了。

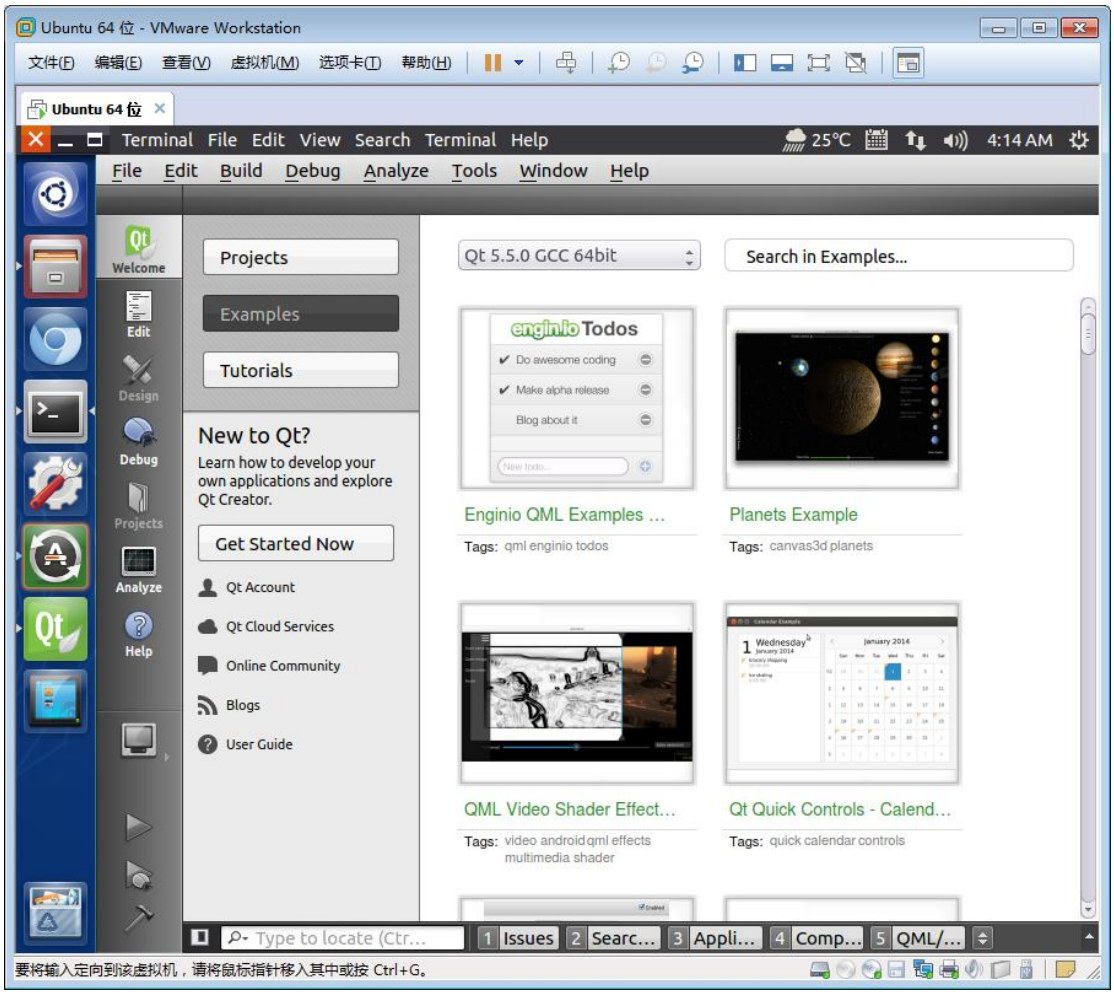
4.6 编译 QT Creator 默认示例

点击 ubuntu 图标，输入 qt，将会查找含有 qt 的文件，同时，安装好的 QT Creator 也会

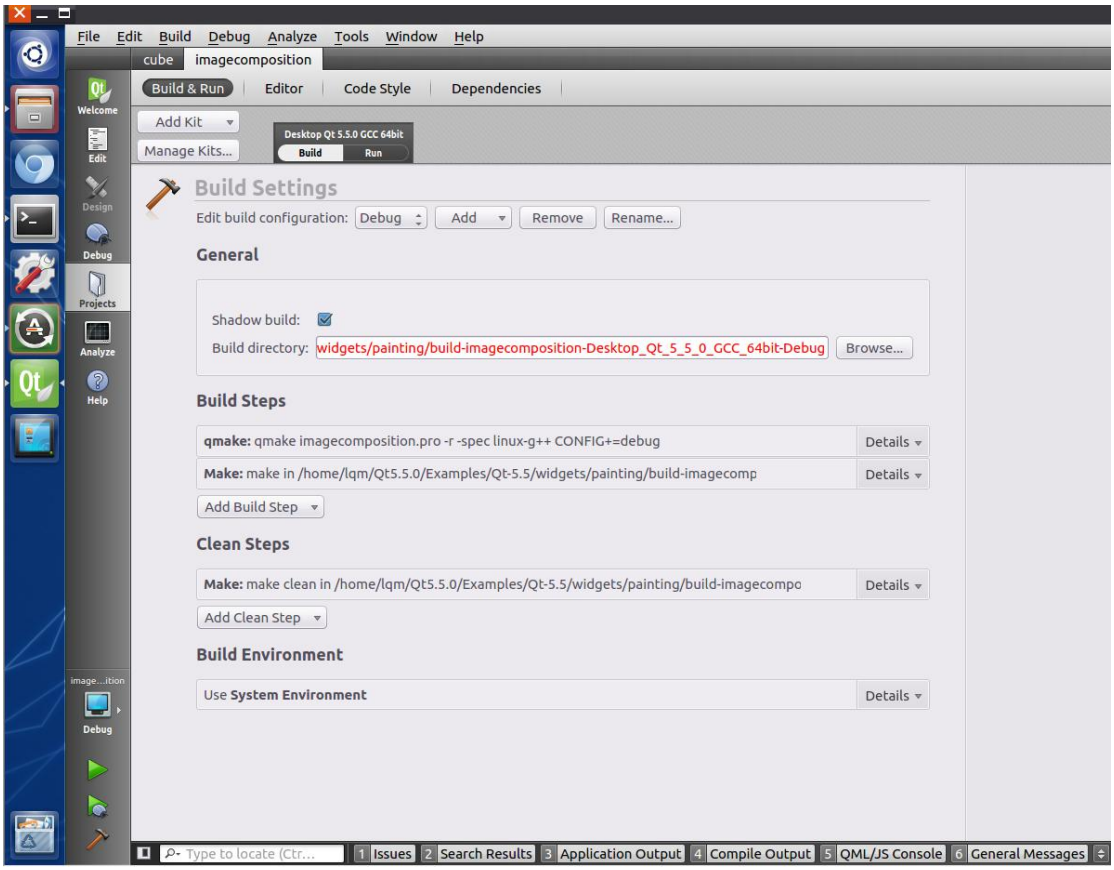
被列出来，如下图所示：



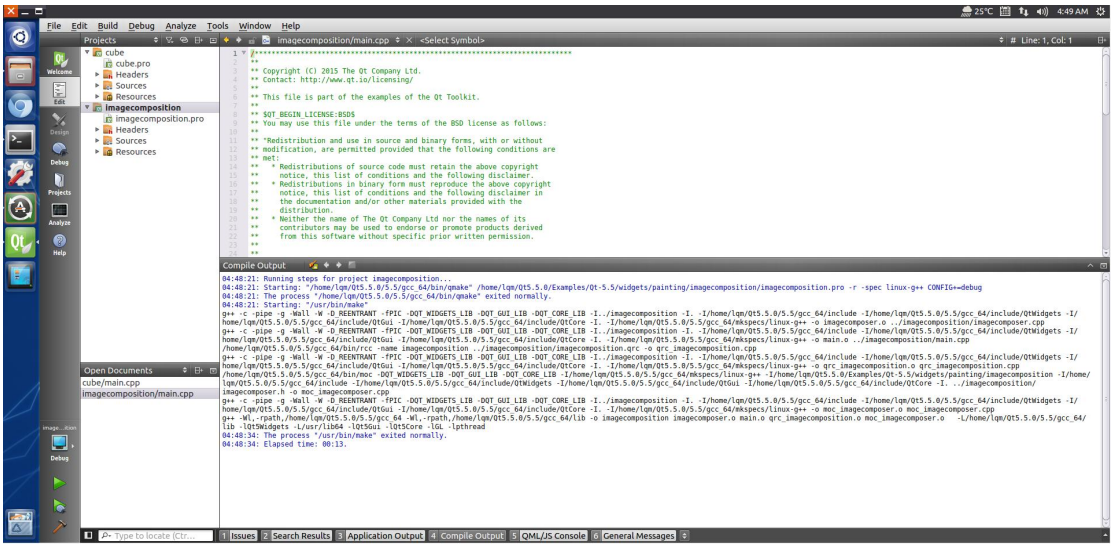
点击 QT 图标，QT Creator 将会运行，如下图所示：



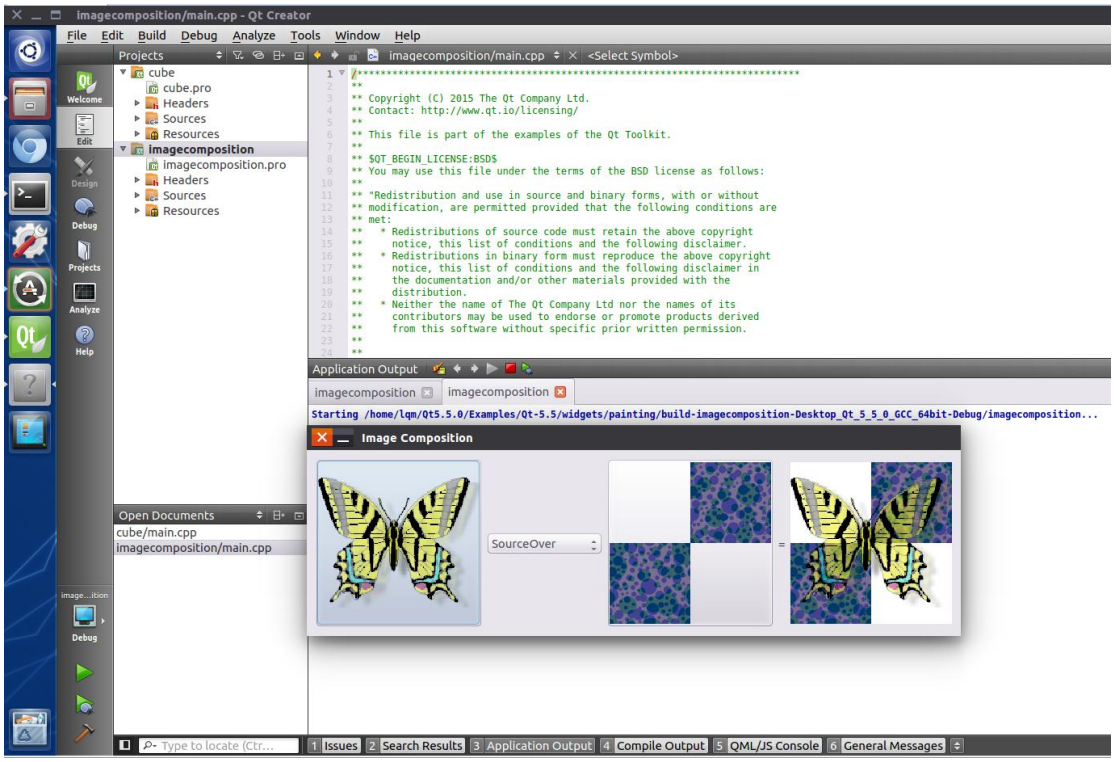
默认 QT 的配置是针对 X86 架构的，这时编译出来的示例只能在 PC 机上运行。我们打开 Image Composition Example 示例，它是一个图片叠加显示的示例，找到该示例，单击即可。打开后的界面如下：



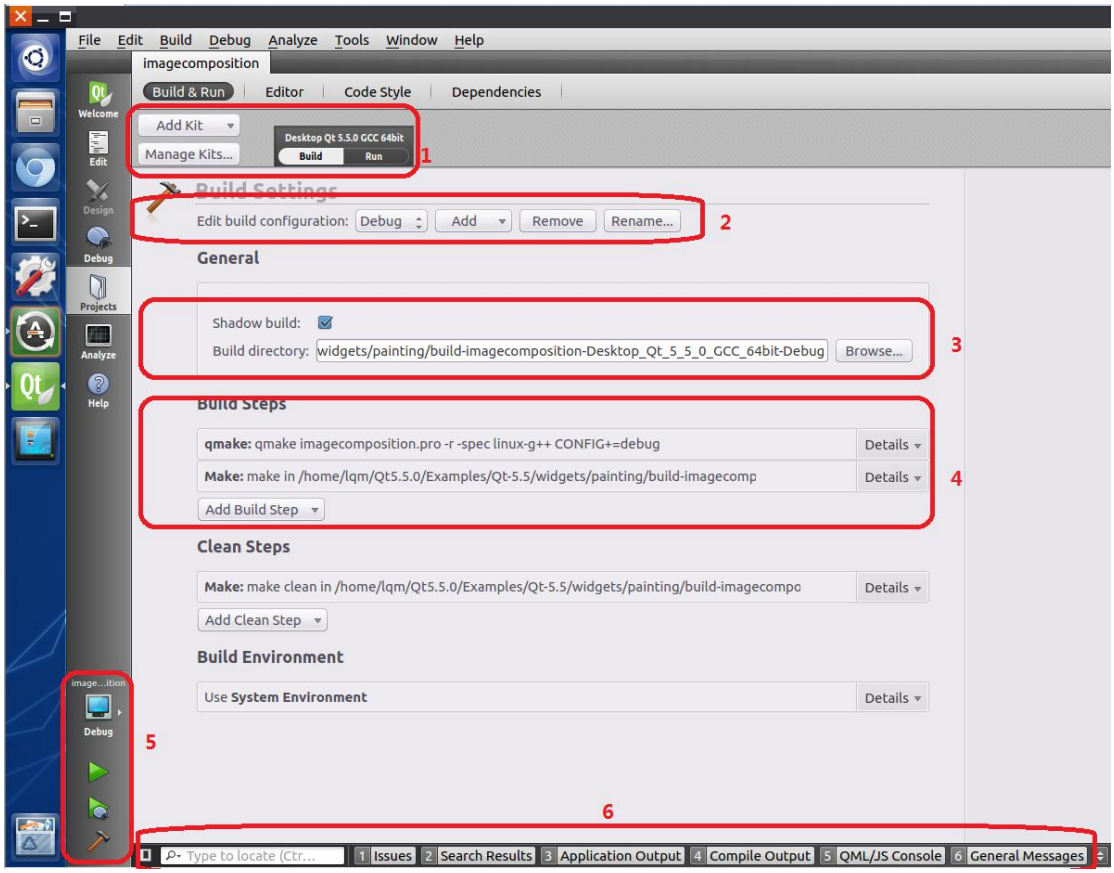
点击左下脚绿色的三脚箭头，开始编译工程。在 Compile Output 栏会显示编译的整个过程。编译完成后，提示如下：



编译完成后，编译出来的映像会自动运行，一个叠加的蝴蝶图像界面显示出来了，界面如下：



下面我们将该示例编译到开发板上运行。使用 QT Creator 打开上面的示例工程，如下图所示：



选择 Projects 一栏，可以看到：

框图 1，显示 Desktop Qt 5.5.0 GCC 64bit，表明它通过 64 位的 GCC 编译，

框图 2 表明在 debug 模式下编译，相对 release 模式，debug 模式下含有大量调试信息，编译出来的映像会比较大。通常发布映像时，我们选择 release 模式。

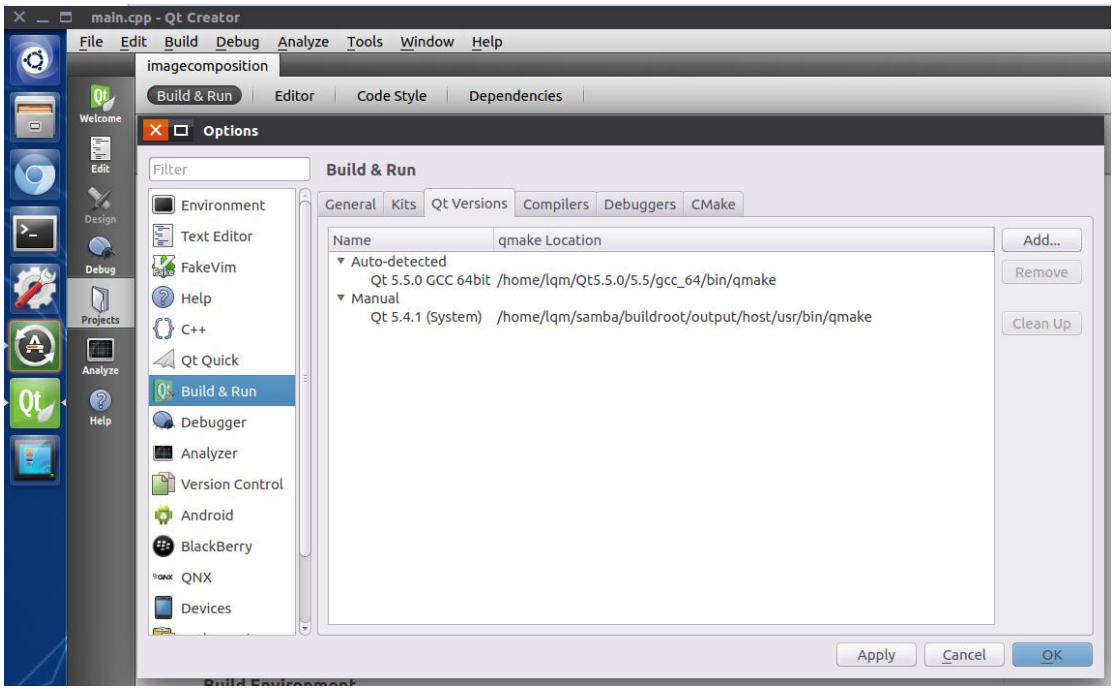
框图 3 指定了编译的路径。

框图 4 指定了 qmake 和交叉编译工具。

框图 5 为调试按键，第一个用于选择编译模式为 debug 或 release，第二个为运行按钮，第三个为单步调试按钮，第四个为编译按钮。

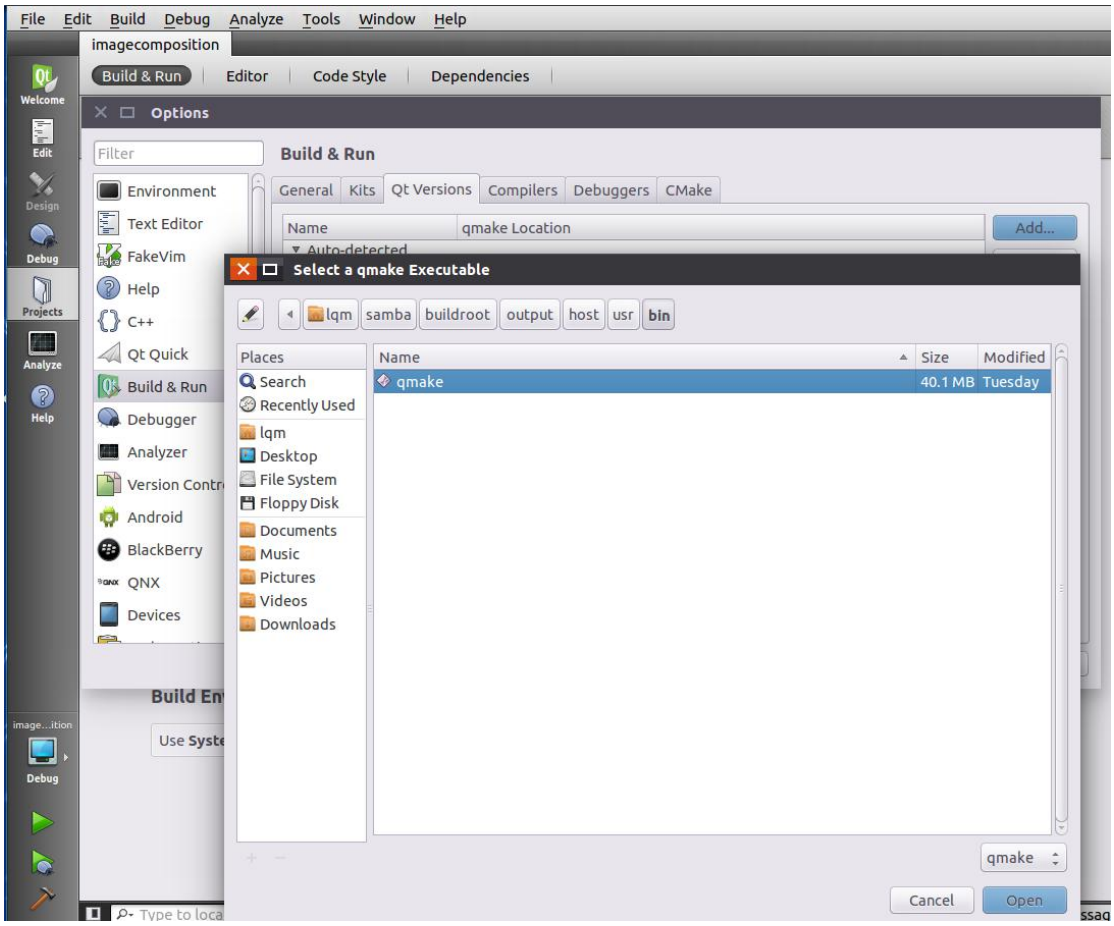
框图 6 为一些输出信息，如 Compile Output，会给出整个编译的信息。

在框图 1 中，点击 Tools->Options，如下图：

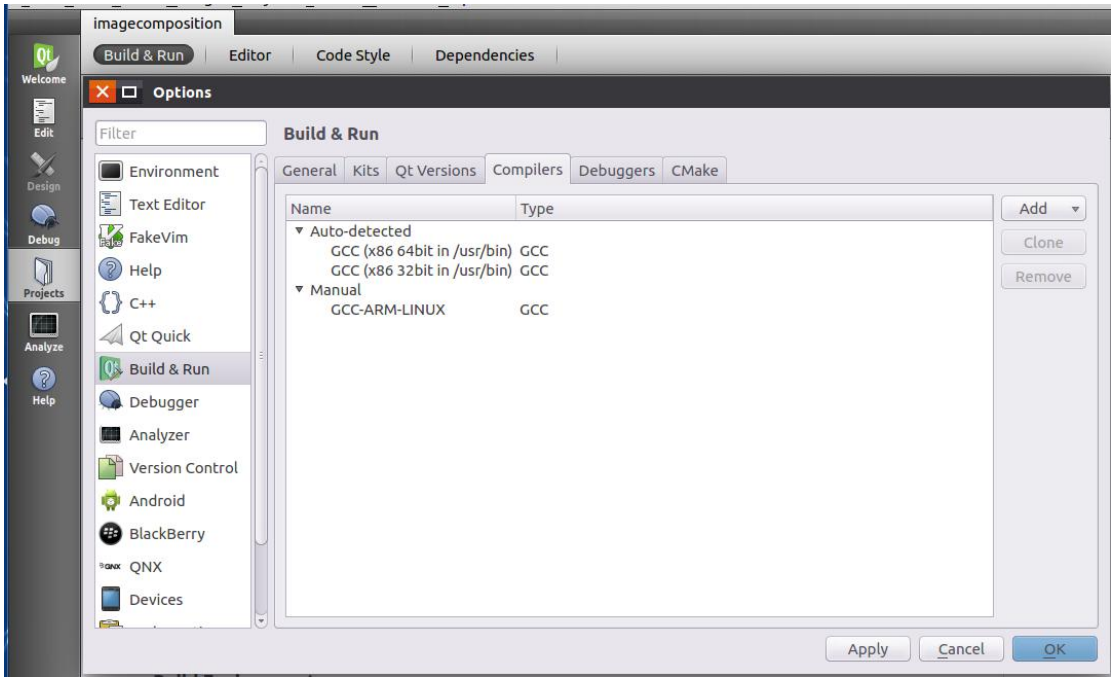


在左测对话框中选择 Build & Run，在后边选择 Qt Versions，默认 Manual 为空，Auto-detected 选择的 QT5.5 默认的 qmake，它将运行在 X86 平台的 linux 系统上，因此我们要手动添加在 ARM 平台上运行的 qmake。

在 buildroot 编译文件系统时，我们选中 QT5 后，将会在 buildroot 的 output/host/usr/bin 目录生成支持 ARM 平台的 qmake，点击 Add，指向该路径的 qmake（buildroot/output/host/usr/bin/qmake）：

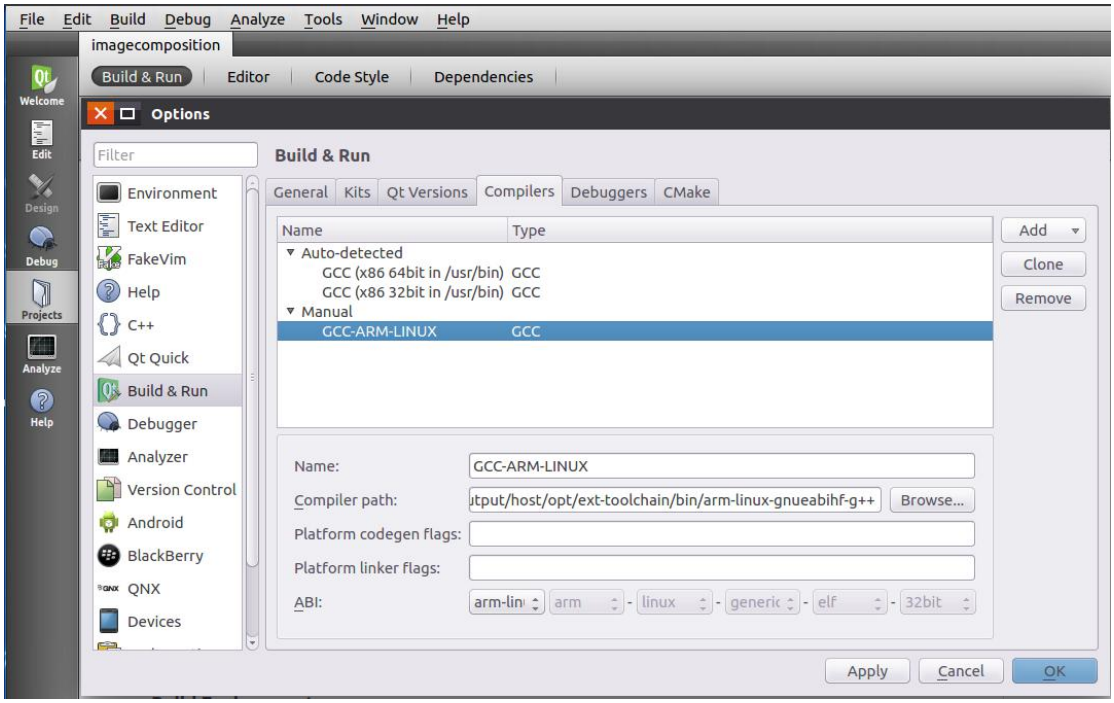


点击 Open，即在 Manual 中添加了支持 ARM 平台的 qmake。再选择 Compilers 一栏，如下图所示：



默认 Manual 为空，Auto-detected 为支持 X86 32 位和 64 位的 GCC，这将直接导致编译出来只能在 PC 机上运行。点击 Add，选择 GCC

（buildroot/output/host/opt/ext-toolchain/bin/arm-linux-gnueabihf-g++），如下图所示：

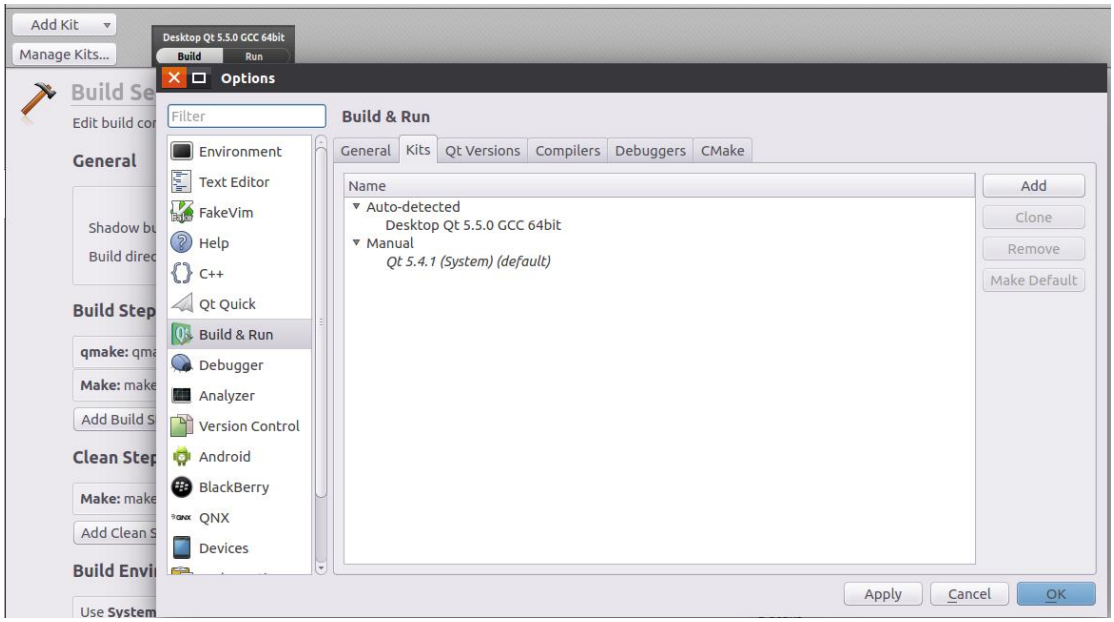


在 Name 中重命名，以区别 ARM 和 PC 平台，如我们命名为 GCC-ARM-LINUX，它将会直接显示在前面 QT 工程界面的框图 1 中，到时我们编译 QT 工程时，能够一目了然，编译出来的到底是 PC 平台还是 ARM 平台。在 Compiler path 中指定交叉编译工具，在 buildroot 中默认已经自动下载并安装了交叉编译工具，我们指定到如下路径即可：

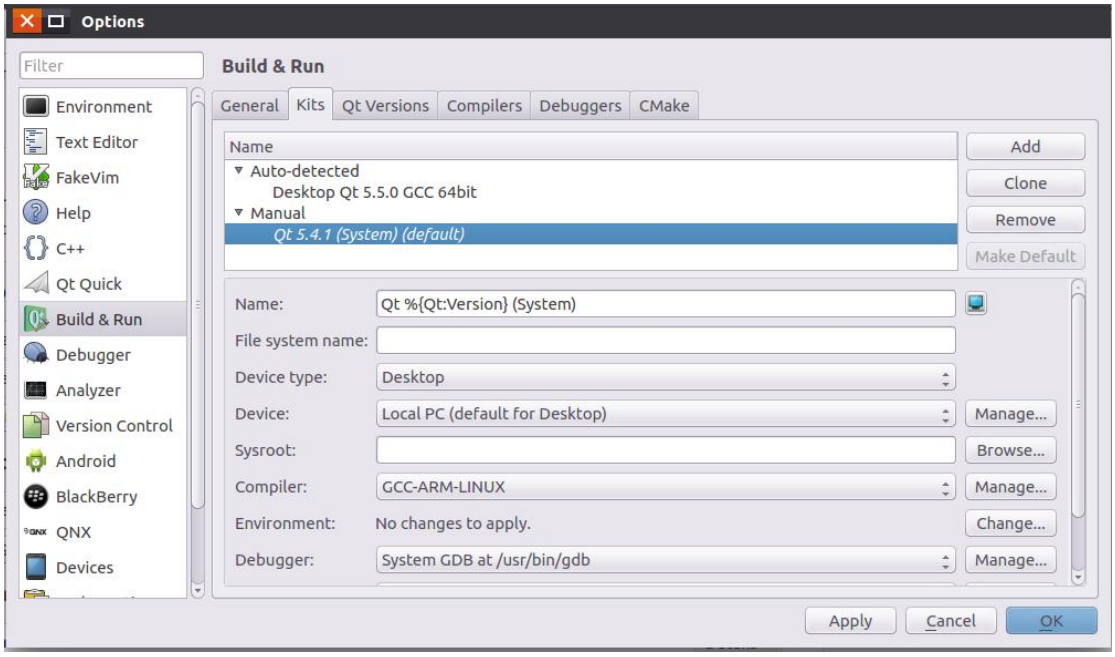
```
buildroot/output/host/opt/ext-toolchain/bin/arm-linux-gnueabihf-g++
```

设置完成后，点击 OK，完成设置。

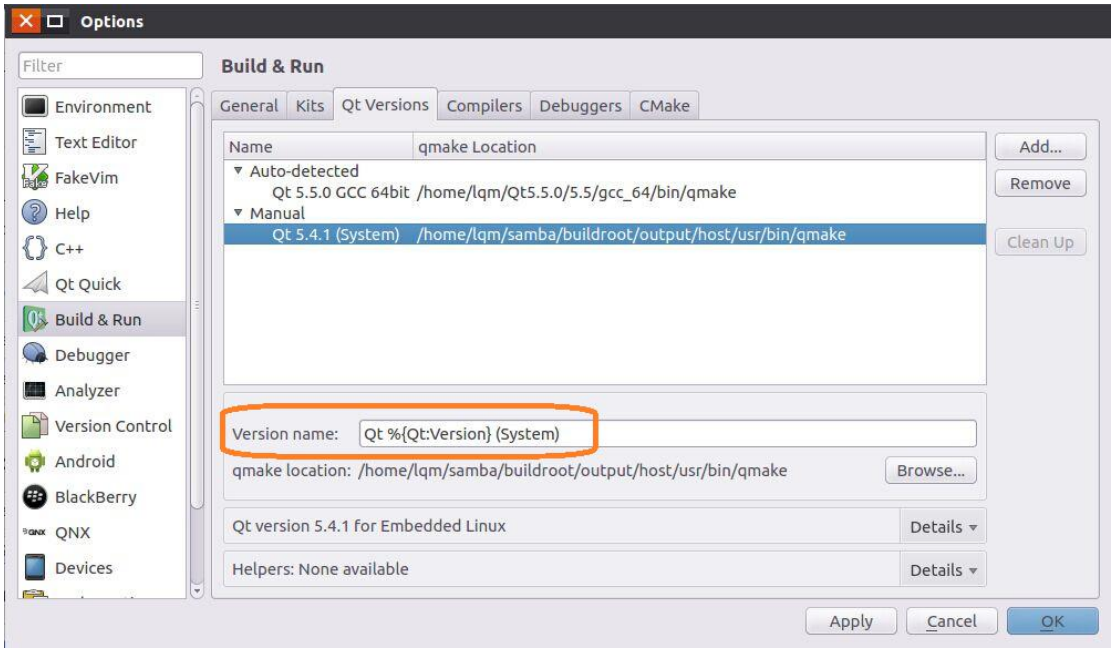
再回到 QT 的工程界面，我们发现框图 1 中的配置仍然针对 PC 机，点击框图 1 中的 Manage Kits，如下图所示：



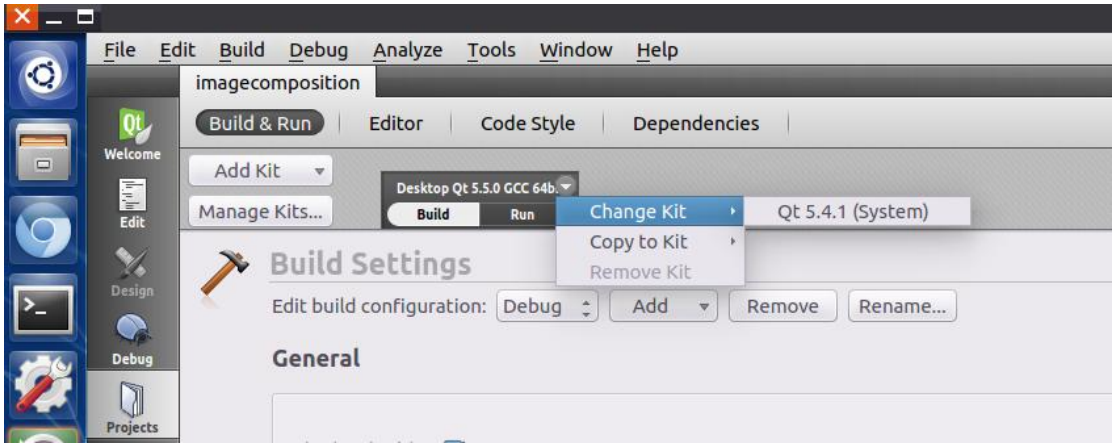
注意，这时默认 Manual 仍然为空，点击 Add，会弹出一个有很多选项的对话框，我们按如下方式配置：



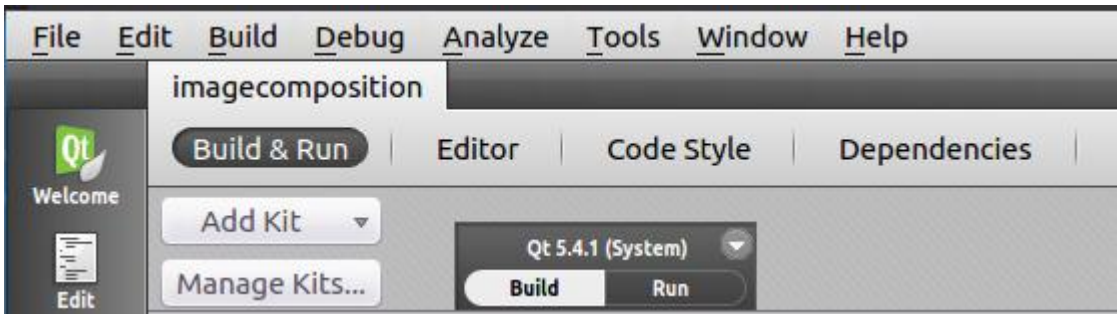
这里的 Name，我们可以点击 Qt Versions，选择 Manual 中的 qmake，下面就会有名称出来，将它拷贝过来即可，如下图所示：



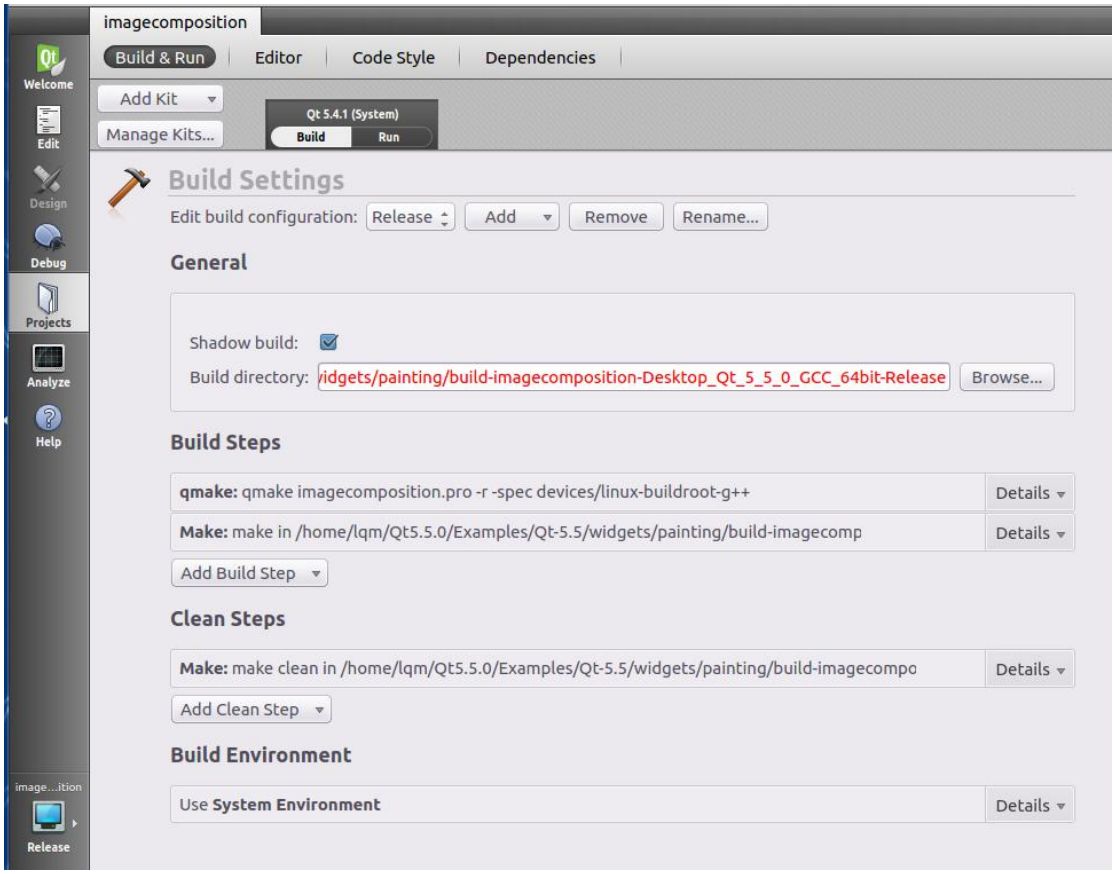
在框图一中，点击设置编译环境的下拉箭头，如下图所示：



选择 Change Kit->Qt5.4.1(System)，更改后的界面如下：

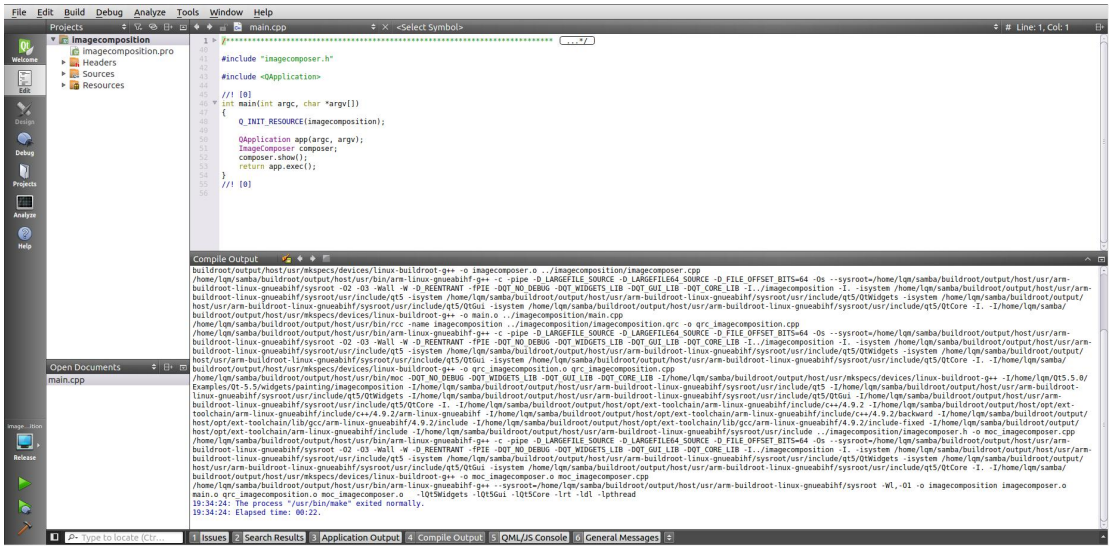


在框图 2 中选择 release，更改后的界面如下：



到此，配置完毕，点击框图 5 中的编译按钮，在 Compile Output 中可以看到编译信息如

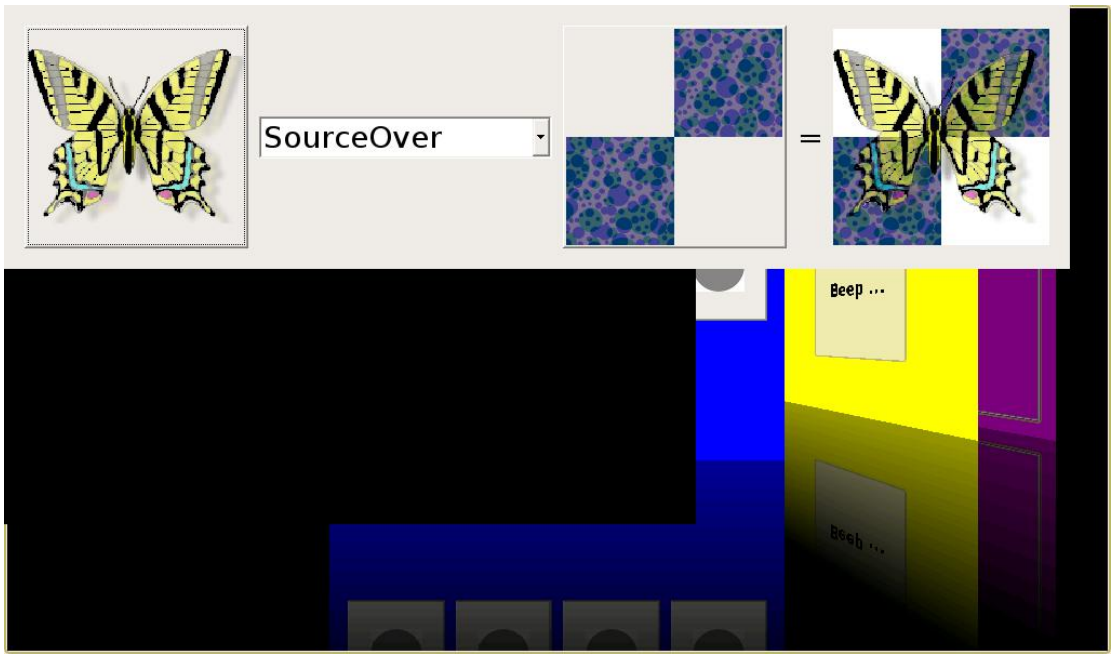
下：



这时，在框图 2 中指定的目录中已经生成了能够在 ARM 平台运行的映像了，如下图所示：

```
lqm@ubuntu:~/Qt5.5.0/Examples/Qt-5.5/widgets/painting/build-imagecomposition-Qt_5_4_1_System-Release$ ls
imagecomposer.o  imagecomposition  main.o  Makefile  moc_imagecomposer.cpp  moc_imagecomposer.o  qrc_imagecomposition.cpp  qrc_imagecomposition.o
lqm@ubuntu:~/Qt5.5.0/Examples/Qt-5.5/widgets/painting/build-imagecomposition-Qt_5_4_1_System-Release$ file imagecomposition
imagecomposition: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 3.1.1, BuildID[sha1]
lqm@ubuntu:~/Qt5.5.0/Examples/Qt-5.5/widgets/painting/build-imagecomposition-Qt_5_4_1_System-Release$
```

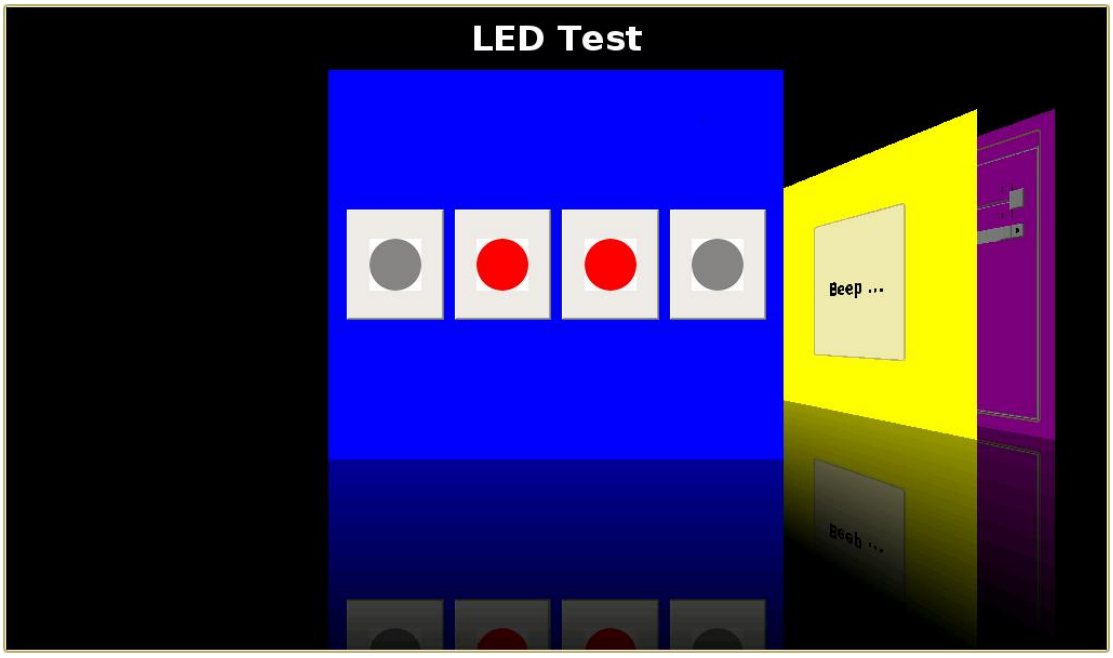
将该文件拷贝到 x6818 开发板上运行，可以看到美丽的蝴蝶图案显示出来了。



第 5 章 qttest 测试程序

5.1 使用 QT_demo 测试 LED

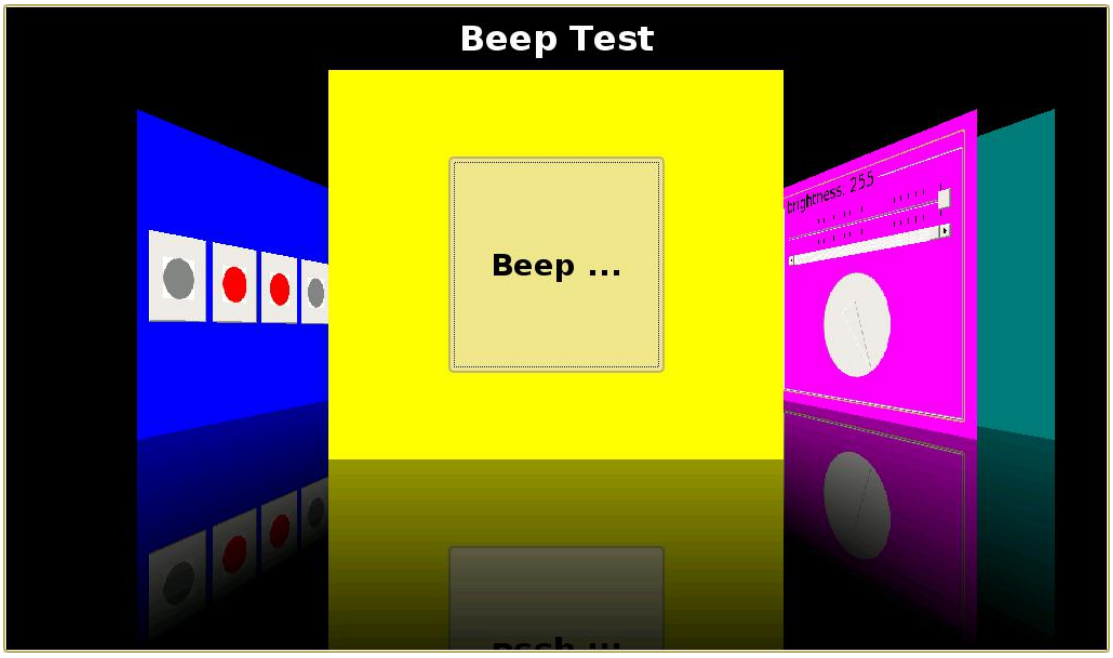
进入 QT5.4 系统后（登录账户 root，123456），默认会运行我们自主编写的测试 demo，进入 LED 测试界面，可以测试开发板的四盏 LED 灯。界面如下：



点击图中任意指示灯，暗色对应开发板上 LED 灯灭，红色对应开发板上 LED 灯亮（注：ibox6818 上 LED 与开发板不同，只有 2 个 LED 可以控制）。

5.2 使用 QT_demo 测试蜂鸣器

测试界面如下：

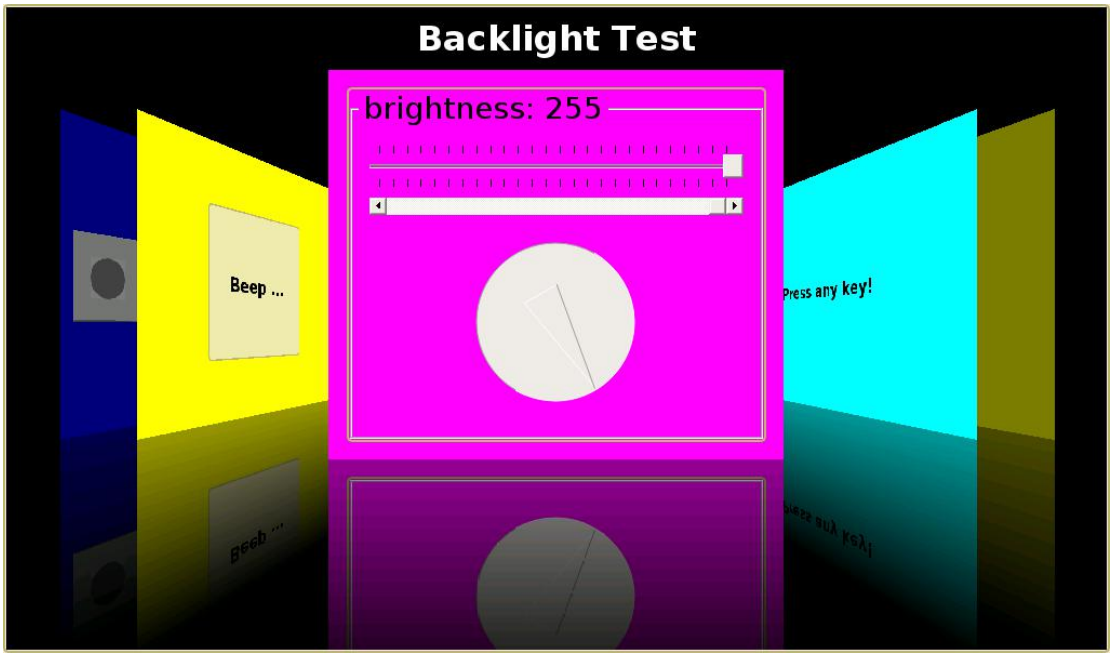


按住 Beep 键时，蜂鸣器鸣叫，松开时，蜂鸣器停止鸣叫。

注：ibox6818 上无蜂鸣器。

5.3 使用 QT_demo 调节背光

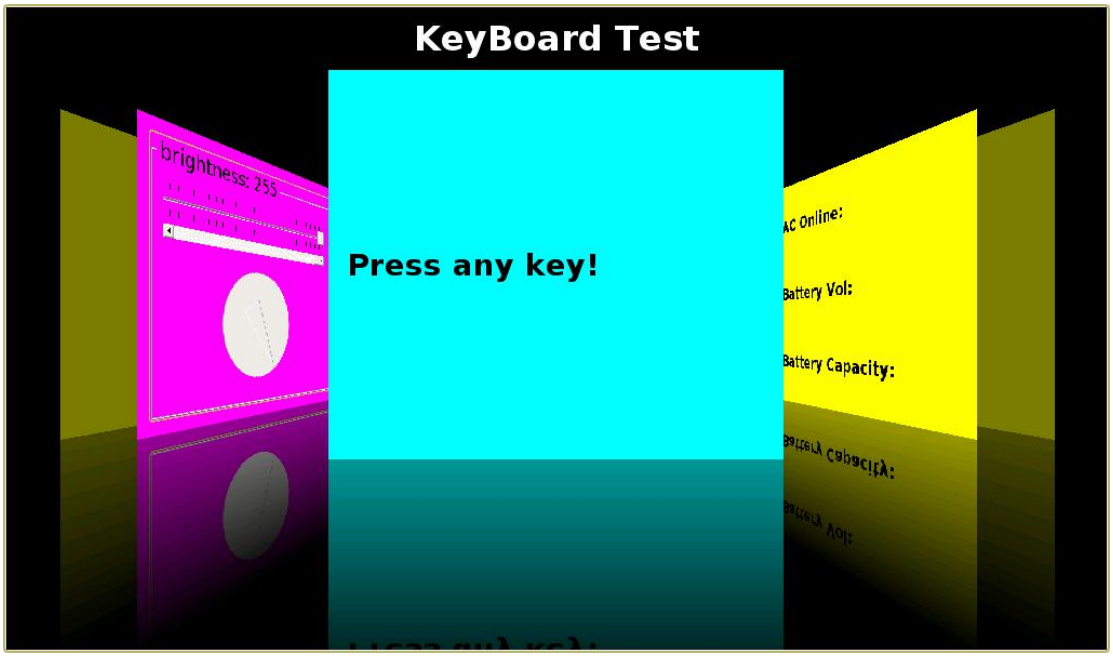
测试界面如下：



滑动圆形滑轮，可对开发板背光进行亮暗调节。

5.4 使用 QT_demo 测试按键

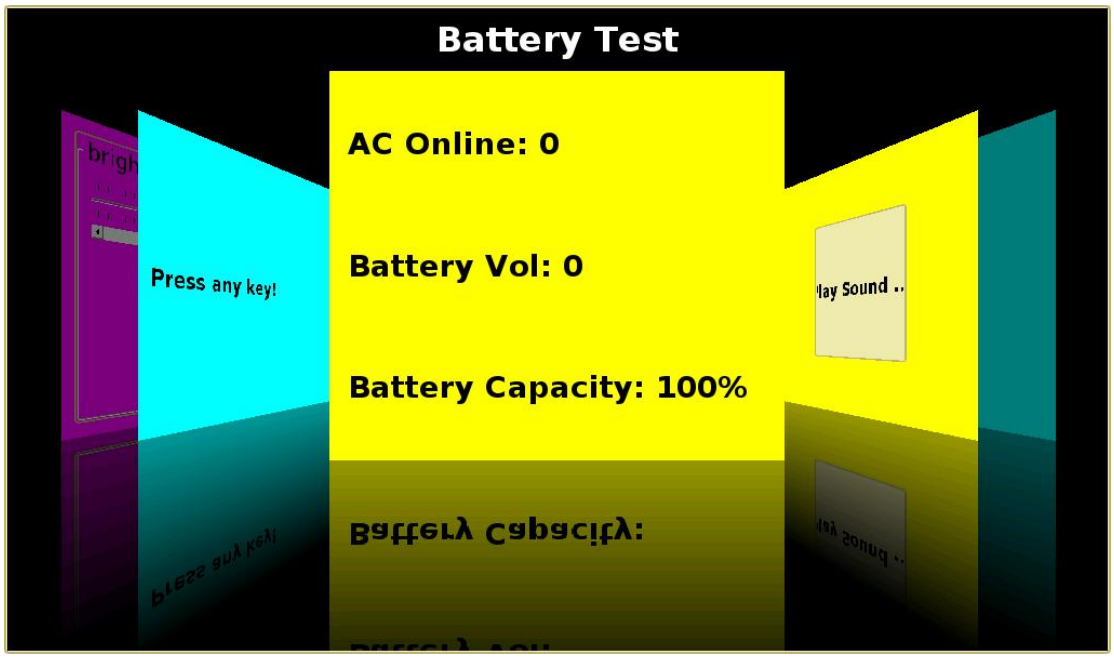
测试界面如下：



按下开发板任一独立按键，图中界面即会显示相应键值，同时，按下时提示[keydown]，抬起时提示[keyup]。目前该功能仅支持音量“+”、“-”键，待调。

5.5 使用 QT_demo 测试 ADC 电压

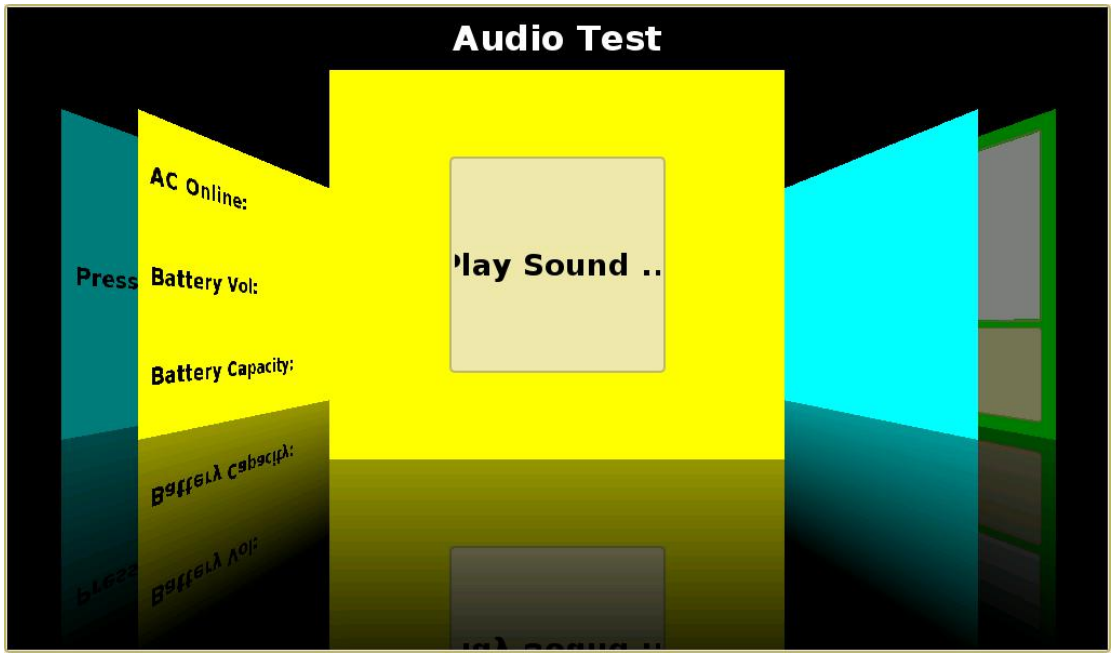
测试界面如下：



使用一字螺丝刀旋转精密电位器上面的旋转按钮，可以看到界面上的 Battery Vol 的值会相应变化，说明这里对电位器上 ADC 电压采样有效。目前该功能异常，待调。

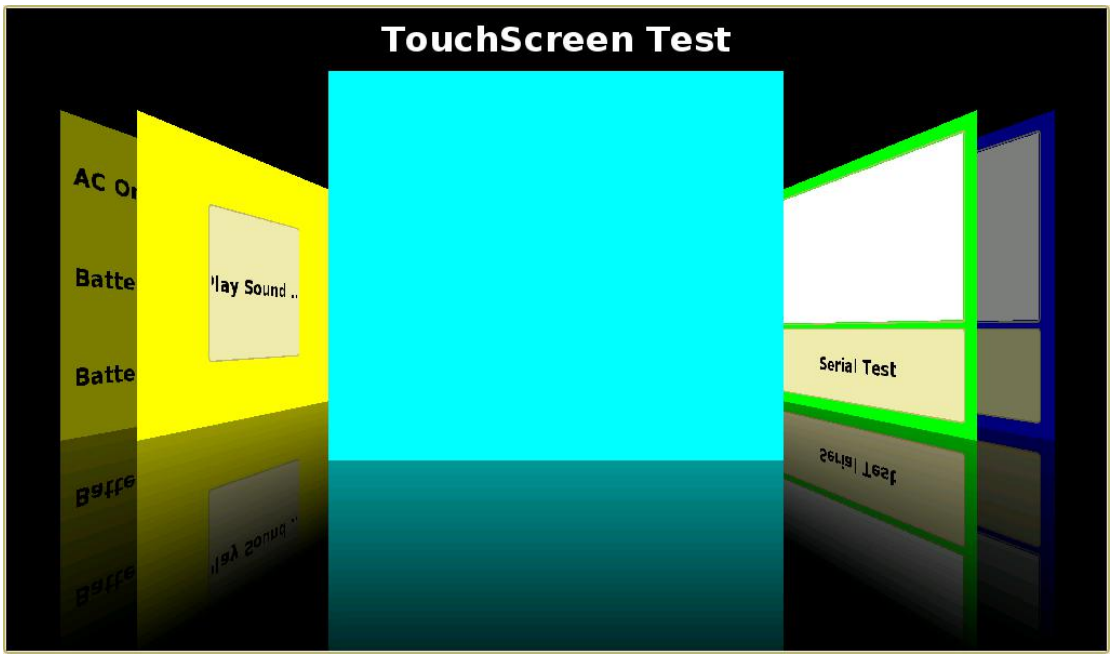
5.6 使用 QT_demo 测试音频

将喇叭或耳机接到开发板的对应接口，点击下图中的 Play Sound 按钮，会播放测试歌曲：

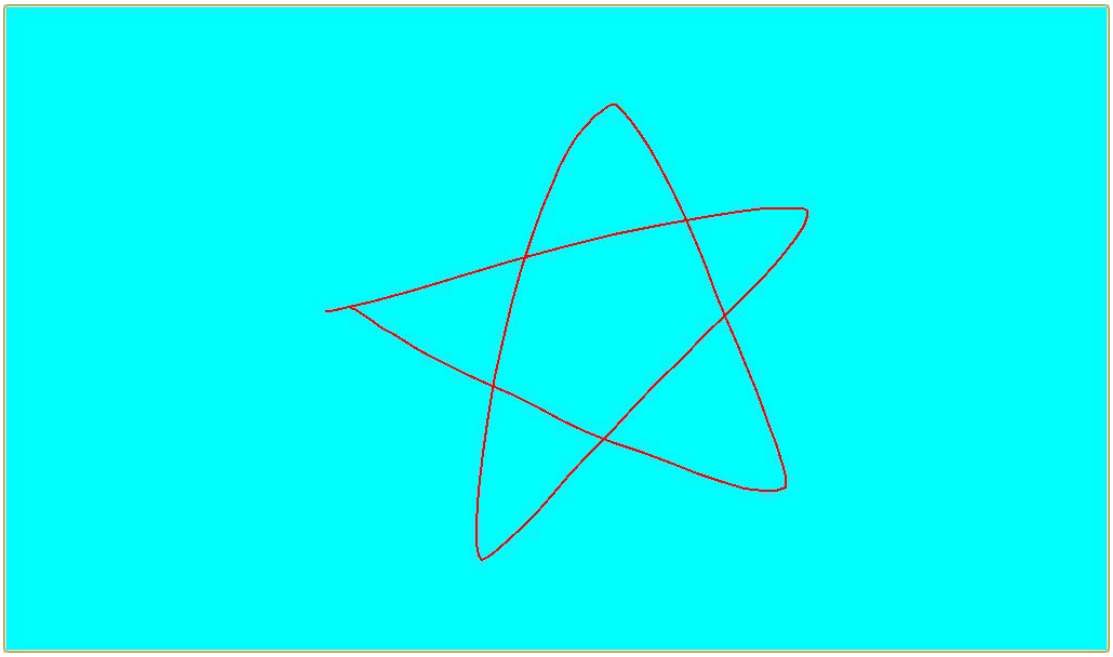


5.7 使用 QT_demo 测试触摸屏

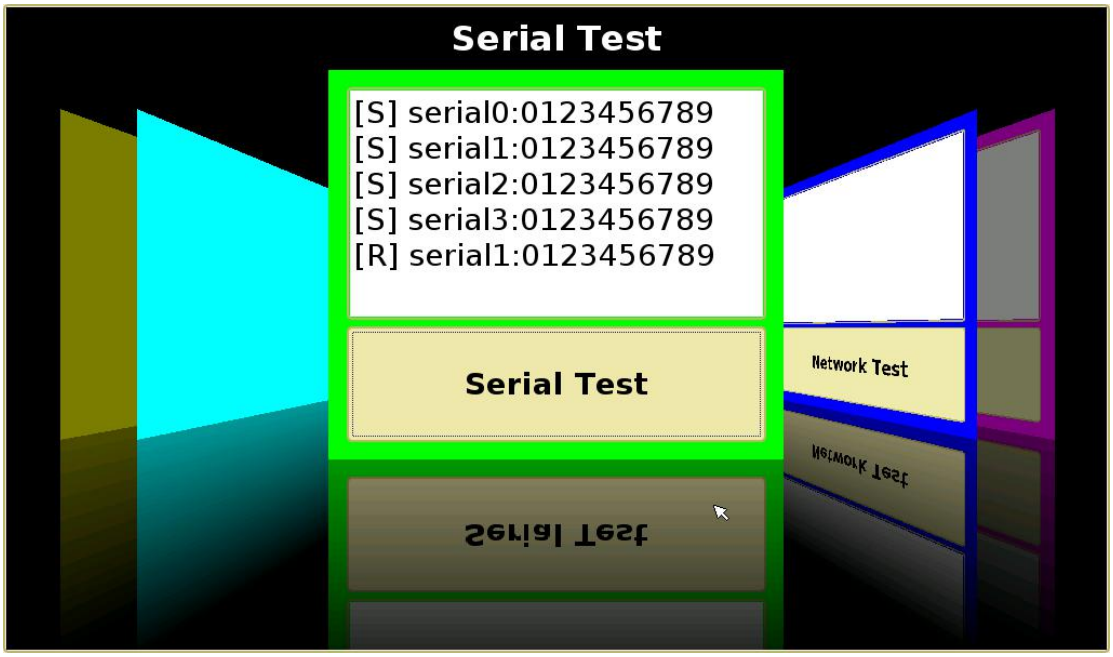
进入如下界面：



单击绿色矩形框，界面会进入全屏模式，这时我们可以任意书写来测试触摸屏了，测试示例图片如下：



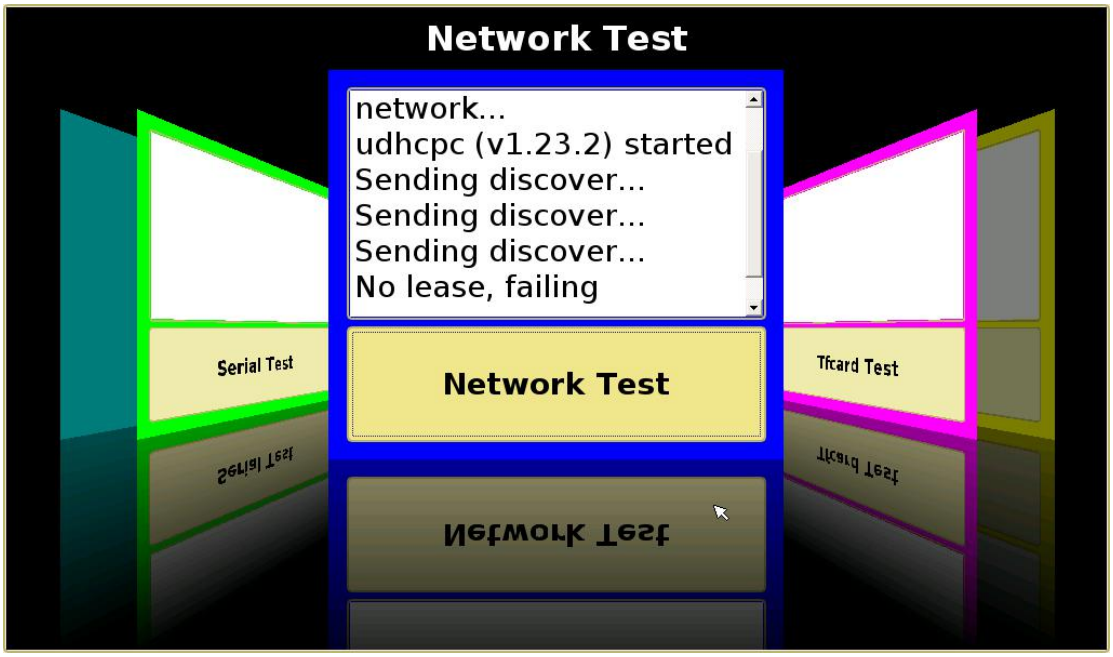
5.8 使用 QT_demo 测试串口



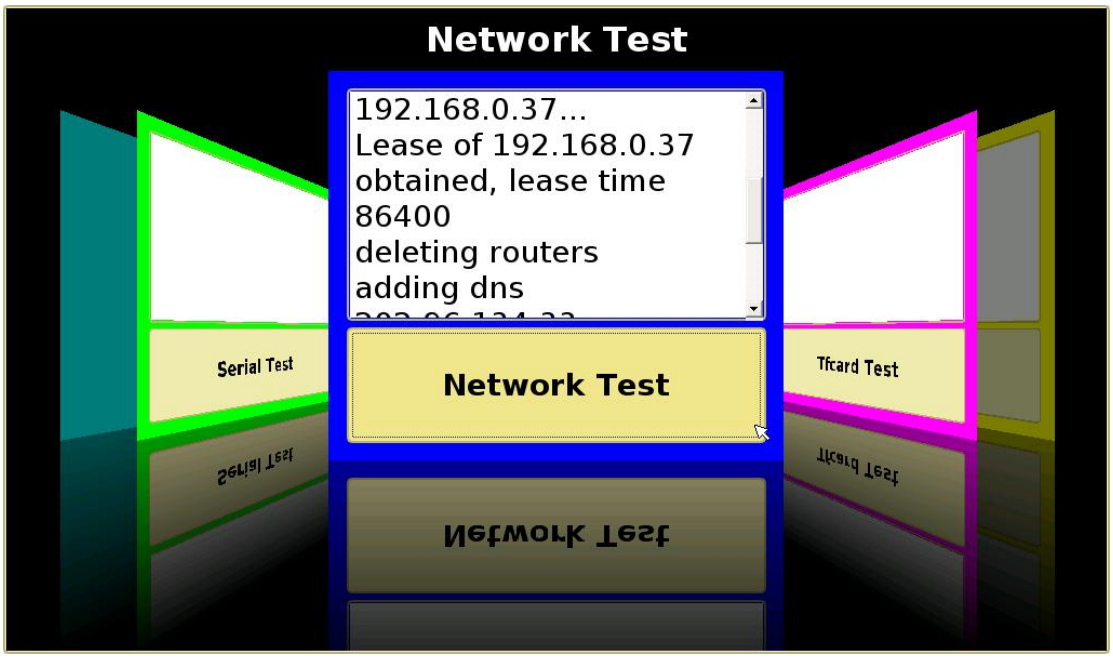
将需要测试的串口的 TXD 和 RXD 短路，再点击开始测试，界面上会提示对应串口会自发自收 0123456789。如果不能自发自收，说明对应的串口硬件有问题。

5.9 使用 QT_demo 测试网络

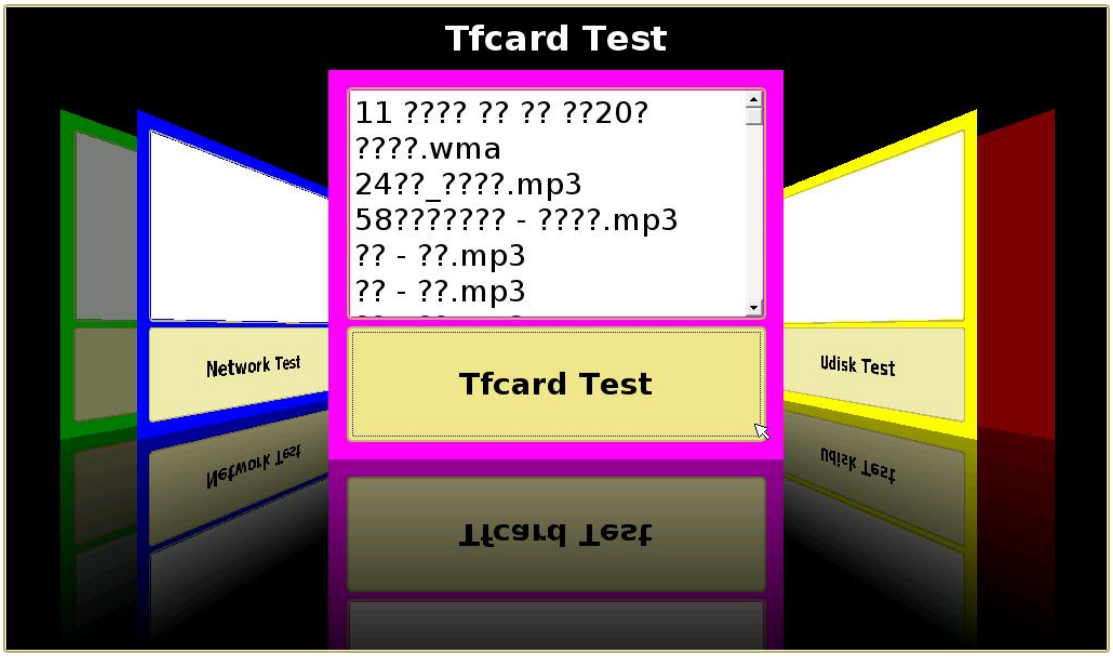
将网线连接开发板的有线以太网接口，点击界面中的 Network Test 按钮，如果网络已经连通，则会添加 DNS，如果没有连通，则会提示相应错误，如下图所示：



连接网线后测试时，会有如下提示：



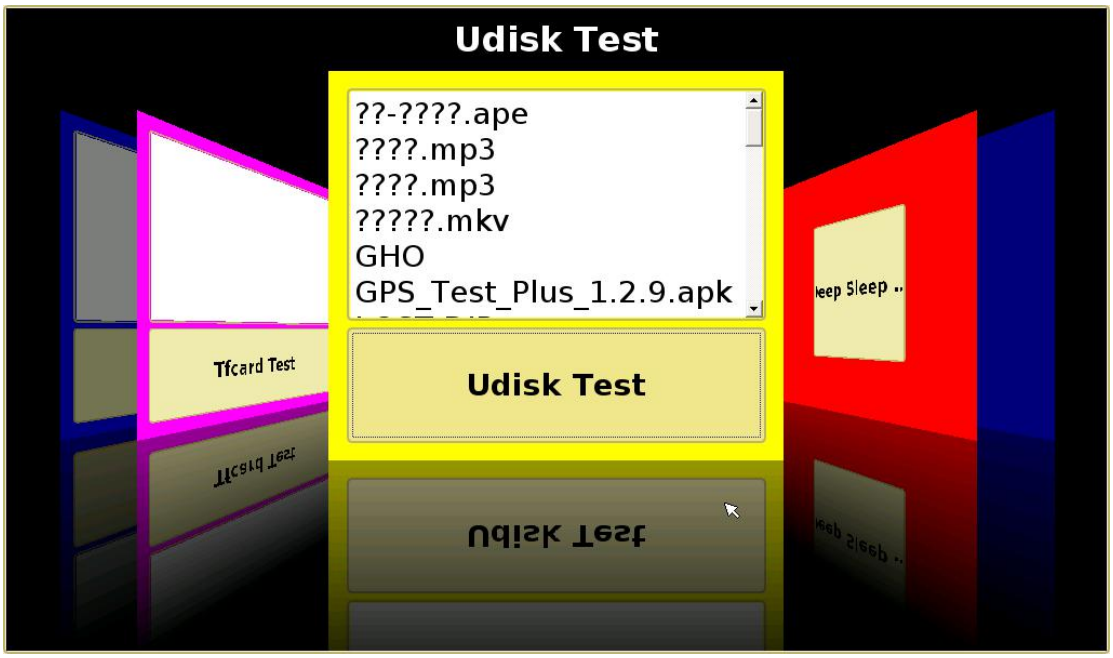
5.10 使用 QT_demo 测试 TF 卡



将 SD 卡插入开发板的 SD 卡槽，点击 Tfcard Test，界面上会列出 SD 卡中的内容。

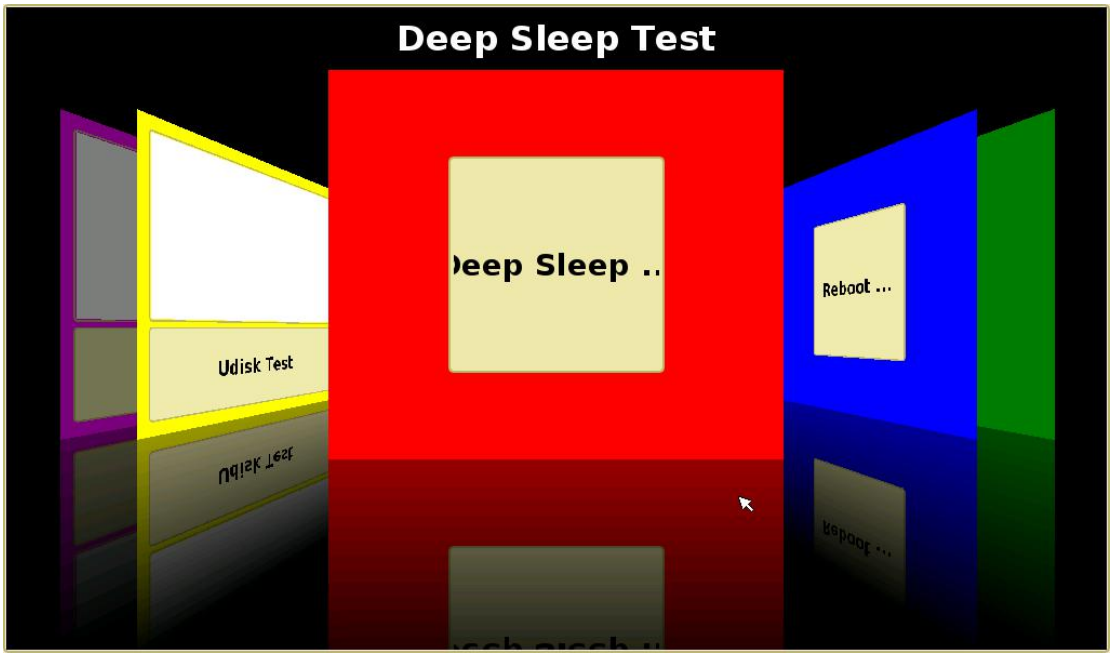
5.11 使用 QT_demo 测试 U 盘

将 U 盘接到开发板的任何一个 USB HOST 接口，点击 Udisk Test，数据框中会列出 U 盘中的数据，如果找不到，则会提示无法 mount U 盘，如图：



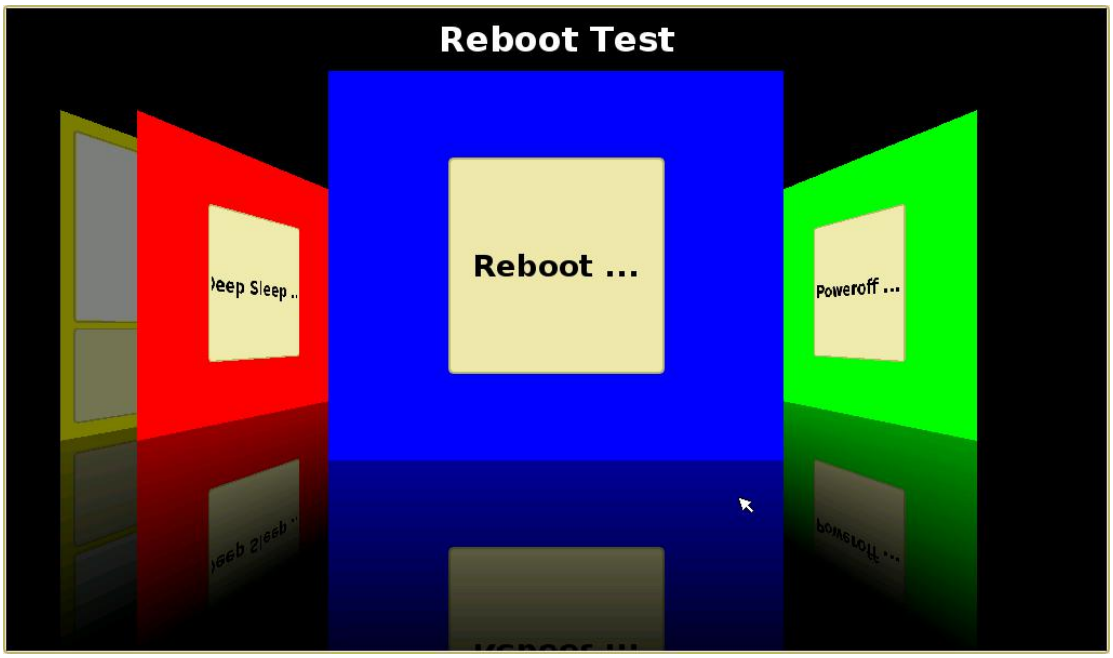
5.12 使用 QT_demo 测试休眠唤醒

点击 Deep Sleep，开发板会进入深度睡眠状态，这时屏幕会全黑，串口终端也将没有任何信息提示，只有按下 POWER 键，方可唤醒开发板。



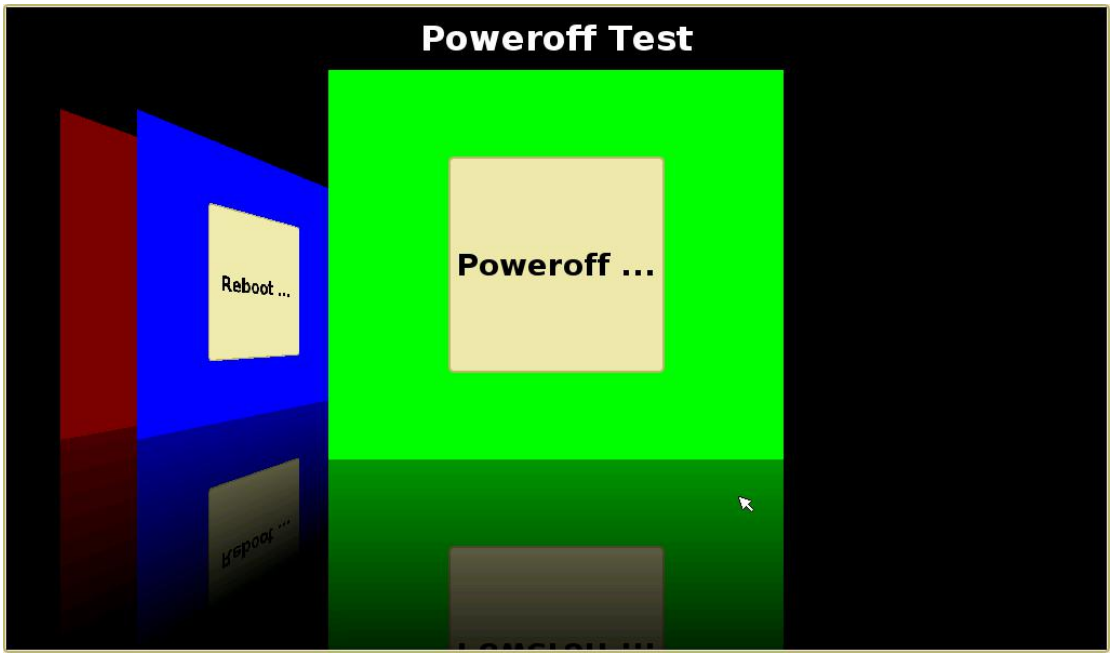
5.13 使用 QT_demo 测试重启

点击 Reboot 按钮，开发板将重启。



5.14 使用 QT_demo 测试关机

点击 Poweroff 按钮，开发板将会关机。



第 6 章 linux 底层开发示例

6.1 播放 mp3

将存放有 mp3 文件的 TF 卡插到开发板的任意卡槽，使用如下命令挂载 TF 卡：

```
cd  /
mkdir  sdcard
mount  /dev/mmcblk1p1  /sdcard
cd  sdcard
```

使用如下命令播放：

```
./mplayer  *.avi
./mplayer  *.mp3
```

连接串口后，可以通过 PC 键盘的 0 或 9 调节音量。也可以使用 madplayer 播放音乐。

6.2 在后台运行程序

在上一节中给出了播放音乐的示例，但是这时候 mplayer 已经占据了终端控制台，在音乐播放完之前，我们无法再使用终端控制台了。又比如我们开发一款产品时，就需要在启动文件系统后运行一个应用程序，如果运行了一个程序，终端控制台就被占用了，那将极大的限制我们的功能。为止，我们可以将程序放在后台运行。使用方法很简单，我们只需在执行的指令后面添加一个”&”即可。如播放音乐时使用如下命令：

```
./mplayer  *.mp3  &
```

6.3 中止程序的运行

中止程序的运行有多种方式，最直接的方式就是直接按 ctrl+c。如前面我们正在播放一段音频文件，我们可以按 ctrl+c 退出程序。但是如果程序在后台运行，那么我们按 ctrl+c 就不管用了。这时我们可以使用 kill 命令。

```
kill+PID
kill+文件名
```

6.4 屏幕抓图

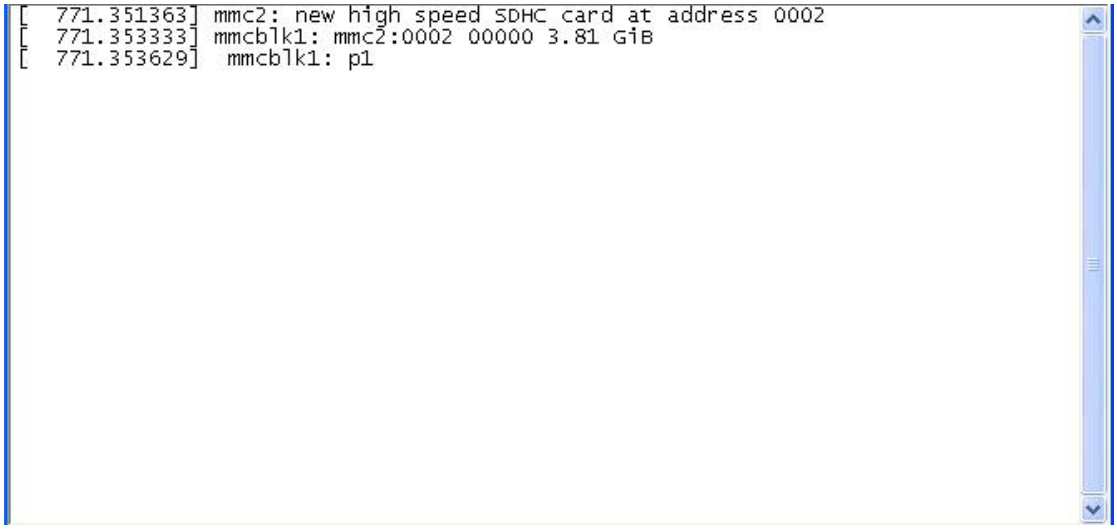
本文档中的各个图片，都是采用 gsnap 这个工具进行抓图的。进入 QT 图形界面后，我们能在 LCD 上看到丰富多彩的人机交互界面。通过 gsnap 可以抓取到图形界面精彩的瞬间。在控制台终端输入如下命令：

```
gsnap  test_pic.jpg  /dev/fb0
```

这时在当前目录将会保存 test_pic.jpg 图像文件。详细的 gsnap 移植步骤在后面会有详细描述。

6.5 挂载 TF 卡

进入 QT 图形界面后，在命令终端会有控制台出现，这时可以通过控制台查看文件系统的内容。将 TF 卡插到开发板的任意卡槽，串口终端会有如下提示：



这时在文件系统的/dev 目录将会自动生成一个名叫 mmcblk1p1 的块设备文件。它就是对应的 TF 卡的设备文件，使用如下命令挂载 TF 卡到/sdcard 目录：

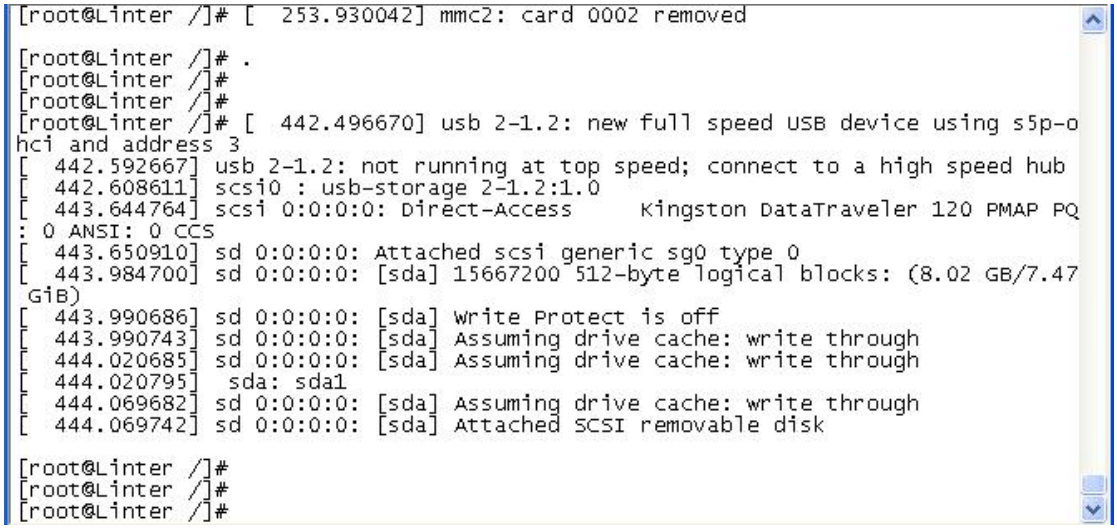
```
mkdir   /sdcard
mount   /dev/mmcblk1p1   /sdcard
```

查看/scard 目录下的内容，即是我们 TF 卡中的内容，如下图所示：



6.6 挂载 U 盘

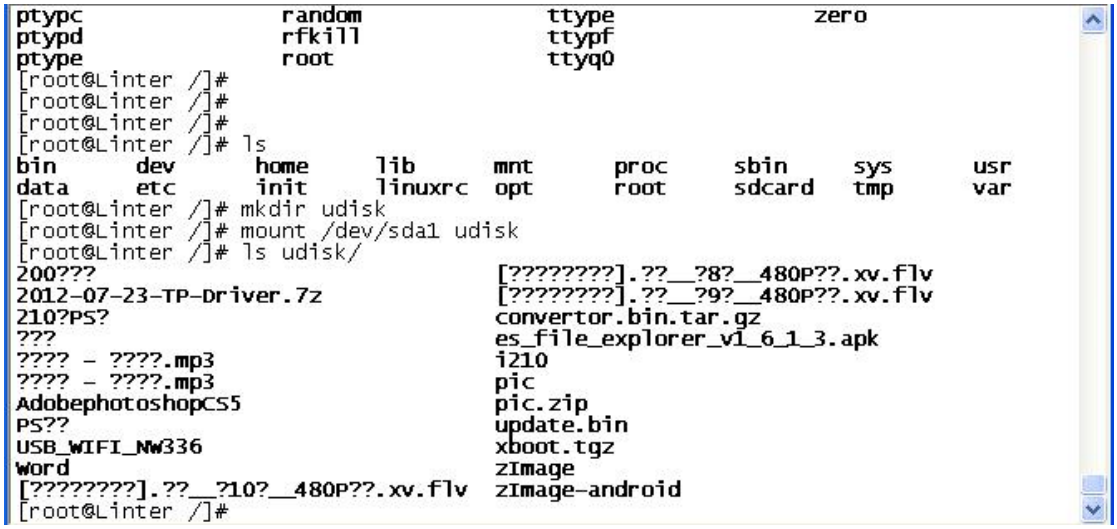
进入 QT 图形界面后，在命令终端会有控制台出现，这时可以通过控制台查看文件系统的内容。插入 U 盘后，串口终端会有如下提示：



这时在文件系统的/dev 目录将会自动生成一个名叫 sda1 的块设备文件。它就是对应的 U 盘设备文件，使用如下命令挂载 U 盘到/udisk 目录：

```
mkdir   /udisk
mount   /dev/sda1   /udisk
```

查看/udisk 目录下的内容，即是我们 U 盘中的内容，如下图所示：



6.7 保存系统时钟

Linux 可以使用 date 指令更改时间日期。例如：

date -s 201104162350 #设置为 2011 年 4 月 16 日 23：50 分

hwclock -w #把刚设置的时间存入 RTC 寄存器

hwclock -s #恢复 linux 系统时钟为 RTC 寄存器值，一般将该指令放在 rcS 中开机自动执行。

6.8 掉电保存数据到 flash

由于本系统采用了 ext4 文件系统，因此可以很方便的保存数据，确保掉电后数据不丢失。如我们从 U 盘中拷备一首歌曲到/root 目录：

```
cp /mnt/muyangqu.mp3 /
```

重启开发板，我们发现在 root 目录仍然存在刚才拷备的这首歌曲，说明掉电后数据并没有丢失。

6.9 设置开机自动运行程序

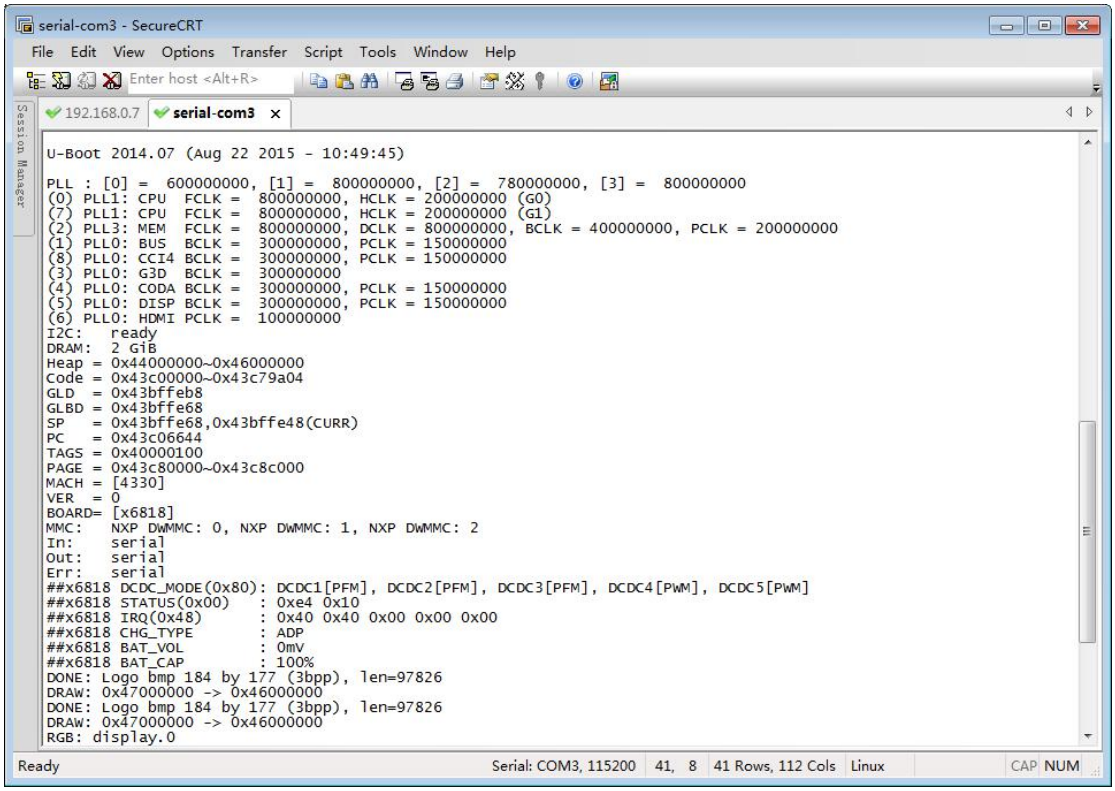
借助启动脚本可以设置各种程序开机后自动运行，这点很类似于 WINDOWS 的 Autobat 自动批处理文件。启动脚本位于/etc/init.d/rcS 中，我们可以将自己想要开机运行的程序或是开机执行的指令放在 rcS 里面。比如我们想制作一个简单的开机音乐，我们就完全可以在 rcS 中添加如下语句：

```
./mplayer  start.mp3  &
```

这时，开机后就会播放名叫 start.mp3 的音乐了。注意 start.mp3 需要在当前执行指令所在目录。

6.10 查看开发板内存信息

X6818 开发板默认配置 1GB DDR3 SDRAM，ibox6818 卡片电脑默认配置 2GB DDR3 在 SDRAM，uboot 启动时，打印信息上会给出 RAM 大小信息：



在进入文件系统后，可以通过 cat 命令查询 Linux 系统分配到的 SDRAM 大小。执行如下命令：

```
cat  /proc/meminfo
```

6.11 uboot 使用 tftp 下载内核

开发板 uboot 已支持网卡驱动，可以使用 ping、tftp 网络命令，这里介绍如何使用 tftp 下载并启动内核。

1.需要搭建好 tftp 服务器，网上有很多相关帖子，这里不再介绍；将编译好的 uImage 拷贝到 tftp 服务器目录下；

2.启动开发板，在 3 秒倒计时时按任意键取消启动进入 uboot 命令行，

3.设置 IP

```
x6818# setenv gatewayip '192.168.0.1'     //网关
x6818# setenv ipaddr '192.168.0.109'      //开发板 ip
x6818# setenv serverip '192.168.0.16'        //tftp 服务器 ip
x6818#save
```

4.设置好后，重启开发板，进入 uboot 命令行输入 tftp 命令：

```
X6818# tftp
Speed: 100, full duplex
Using dwmac.c0060000 device
TFTP from server 192.168.0.16; our IP address is 192.168.0.109
Filename 'uImage'.
Load address: 0x48000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ####################
         926.8 KiB/s
done
Bytes transferred = 5068800 (4d5800 hex)
X6818# bootm 0x48000000
## Booting kernel from Legacy Image at 48000000 ...
   Image Name:     Linux-3.4.39-9tripod
   Image Type:     ARM Linux Kernel Image (uncompressed)
   Data Size:      5068736 Bytes = 4.8 MiB
   Load Address: 40008000
   Entry Point:    40008000
   Verifying Checksum ... OK
   Loading Kernel Image ... OK

Starting kernel ...
```

从上述 log 中看出开发板已经从 tftp 服务器下载 uImage 到 0x48000000 并启动内核。

6.修改环境变量，让重启后开发板自动 tftp 下载并启动内核：

```
setenv bootcmd "tftp;tftp;bootm 0x48000000"
save
```

重启开发板，开发板即可自动执行 tftp 下载，然后 bootm 0x48000000。

6.12 挂载 nfs 网络文件系统

上一节我们介绍了 uboot 如何使用 tftp 下载启动内核，本节继续介绍内核如何挂载 nfs 网络文件系统；与上一节结合在一起直接从网络启动内核与文件系统，十分方便内核与文件系统的开发与调试。

1.内核必须配置支持 NFS 与网络，目前发布的内核已支持，如果用户拿到的内核代码

没有支持，请配置支持并编译；

2.搭建好的 nfs 服务器，网上同样有很多搭建教程，本文不作介绍；

3.将 buildroot/output/target 下所有文件拷贝到 NFS 服务器共享目录；

注意：拷贝完成后删除 etc/init.d/S40network 脚本，否则会导致 nfs 挂载过程出错。

4.在 uboot 命令行中修改环境变量：

```
setenv bootargs "bootargs=lcd=vs070cxn tp=gslx680-linux root=/dev/nfs rw rootfstype=ext4
console=ttyAMA0 nfsroot=192.168.0.22:/home/jason/nfs,proto=tcp,rsize=1024,wsize=1024
ip=192.168.0.109:192.168.0.22:192.168.0.1:255.255.255.0::eth0:off nfsrootdebug init=/linuxrc"
save
```

然后重启开发板即可，如下图 log 打印表明 nfs 挂载成功。

```
[    4.508000] ALSA device list:
[    4.510000]   #0: I2S-alc5623
[    4.513000]   #1: SPDIF-Transciever
[    4.727000] EXT4-fs (mmcblk0p2): recovery complete
[    4.730000] EXT4-fs (mmcblk0p2): mounted filesystem with ordered data mode. Opts: (null)
[    4.735000] VFS: Mounted root (ext4 filesystem) on device 179:2.
[    4.741000] devtmpfs: mounted
[    4.744000] Freeing init memory: 240K
[    4.747000] Write protecting the kernel text section c0008000 - c091a000
[    4.755000] rodata_test: attempting to write to read-only section:
[    4.760000] write to read-only section trapped, success
```

上述环境变量中 IP 设置格式如下：

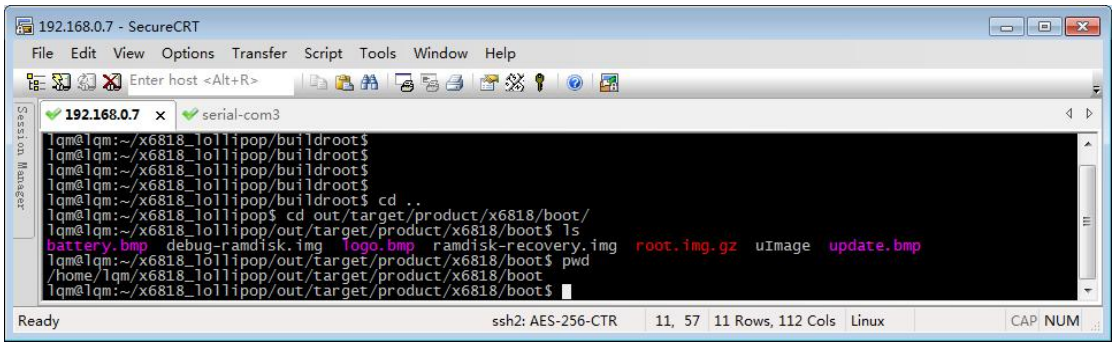
ip 配置格式：nfsroot=[<server-ip>:]<root-dir>[,<nfs-options>]

ip=<client-ip>:<server-ip>:<gw-ip>:<netmask>:<hostname>:<device>:<autoconf>

关于 nfs 文件系统，可以参考内核文档：nfsroot.txt。

第 7 章 使用 ramdisk 文件系统

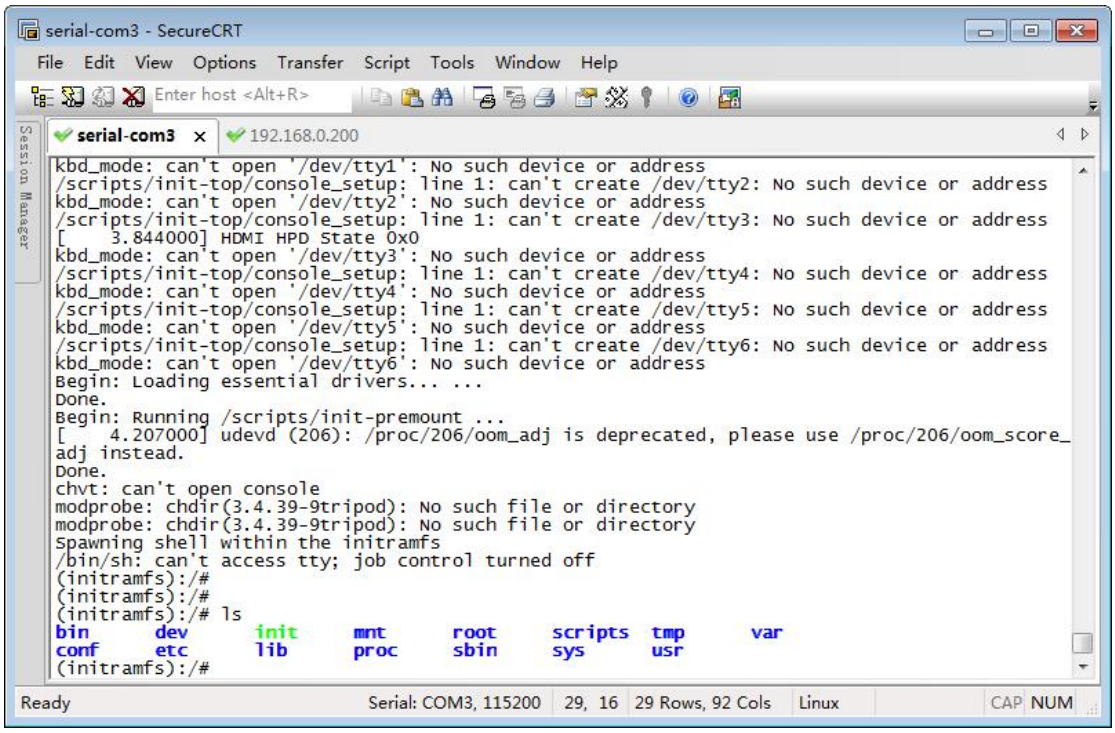
默认我们已经将 ramdisk 文件系统打包到 linux 内核中了，也就是说，我们烧写的 boot.img 映像文件，其实里面已经存在 ramdisk 文件系统了。编译完 android 文件系统后，可以在 out/target/product/x6818/boot 目录下看到，里面存在 debug-ramdisk.img，我们可以通过修改 uboot 启动变量实现挂载 ramdisk。



在 uboot 命令行下，依次输入如下三条指令：

```
ext4load mmc 2:1 0x48000000 uImage;
ext4load mmc 2:1 0x49000000 debug-ramdisk.img;
bootm 0x48000000
```

启动后，将会挂载 ramdisk 文件系统，成功挂载后界面如下：



第 8 章 linux 应用开发示例

本手册给出的所有应用程序全部在九鼎创展 x6818 开发板上运行，这里仅给出了一些比较基础，常用的应用程序，旨在为用户打开 Linux 世界奇妙的大门，用户定能举一反三，编写出属于自己的更加丰富完美的程序。

声明：以下所有应用程序全部为九鼎创展科技有限公司原创作品，所有内容全经我们严格测试，建议用户按照下面步骤动手编译一遍，以增强自己的理解，不推荐直接使用我们提供好的文件。另外，敬请商业人士勿侵犯版权。

8.1 Hello World

第一步：生成可执行文件

在 x6818_lollipop 目录新建 app-ex 目录，在 app-ex 目录新建 hello 目录，然后在 hello 目录下新建 hello.c 和 makefile 两个文件：

```
vim   hello.c
```

```
#include <stdio.h>

int main()
{
        printf("hello,9tripod");
}
```

这是一个最基础的应用程序，如果我们声明了交叉编译工具，可以直接敲入命令进行编译：

```
arm-linux-gcc-4.4.3 -o hello hello.c -static
```

说明：由于开发板文件系统缺少标准函数库，编译时加上-static 选项。

编译完成后，在当前目录会生成 hello 可执行文件，我们可以使用 file 命令查询执行文件是否为 ARM 体系文件：

```
sjx@9tripod-workstation:~/projects/x6818_lollipop_v51$ file hello
hello: ELF 32-bit LSB  executable, ARM, EABI5 version 1 (SYSV), statically linke
d, for GNU/Linux 2.6.32, not stripped
```

第二步：将可执行文件下载到开发板运行

比较常用的方式有以下四种：

1） 通过串口和 sz/rz 工具

2） 复制到存储媒介，如 SD 卡，U 盘等

3） 通过 NFS 挂载文件系统，这时不用将可执行文件拷备到开发板了，推荐调试使用这种方式！

4） 通过 ftp 传输

这里介绍第二种方法，以 TF 卡为例，其他方法请读者自行尝试。

将生成的 hello 文件拷备到 TF 卡，再将 TF 卡插入开发板的 TF 卡接口，将 TF 卡 mount 到/mnt 目录：

```
cd   /
mount   /dev/mmcblk1p1   /mnt
```

进入 mnt 目录，可以看到刚才拷贝的 hello 文件了：

```
[root@X6818:/mnt]# ls
hello
[root@X6818:/mnt]#
```

运行 hello：

```
chmod 777 hello
./hello
```

打印信息如下：

```
[root@X6818:/mnt]# ./hello
hello,9tripod[root@X6818:/mnt]#
```

表明，程序已经成功运行。

前面编译文件我们需要通过手敲命令执行，我们可以通过编写 makefile 来代替手敲的动作。

在 hello 目录下新建 makefile 文件：

```
vim   makefile
```

```
CC=~/toolchain/arm-linux-gcc-4.4.3/arm-linux-gcc-4.4.3/bin/arm-linux-gcc-4.4.3

hello:
        $(CC) -o hello hello.c


clean:
        rm -f hello *.o
```

直接在 hello 目录下敲 make 就可以生成 hello 文件。运行的效果和前面的完全相同。

8.2 LED 测试程序

建立程序编译路径：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    while(1)
    {
        system("echo 0 > /sys/devices/platform/leds-gpio/leds/led2/brightness");
        sleep(1);
        system("echo 1 > /sys/devices/platform/leds-gpio/leds/led2/brightness");
        sleep(1);
    }
    return 0;
}
```

编辑 makefile 文件，内容如下：

```
OBJS=led.o
CC=yourtoolchainpath/bin/arm-linux-gcc-4.4.3
```

```
led:${OBJS}
	${CC} -o $@ $^ -static

clean:
	rm -f led *.o
```

执行 make，将生成的可执行文件 led 下载到开发板上运行，观察开发板上第二盏灯的状态。

8.3 数学函数库调用

建立程序编译路径：

```
mkdir   math
cd   math
vim   math.c
```

编辑如下内容：

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void)
{
	double a=9.0;
	printf("sqrt(%f)=%f\n",a,sqrt(a));

	return 0;
}
```

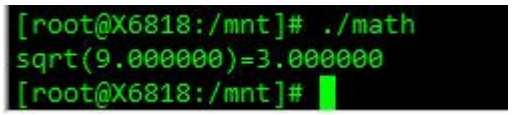
编辑 makefile 文件，内容如下：

```
OBJS=math.o
LDFLAGS = -lm
CC = yourtoolchainpath/bin/arm-linux-gcc-4.4.3

math:${OBJS}
	${CC} -o $@ $^ $(LDFLAGS) -static

clean:
	rm -f math *.o
```

执行 make，将生成的可执行文件 math 下载到开发板上运行，如下图所示：



8.4 多线程编程示例

建立程序编译路径：

```
mkdir   thread
```

```
cd  thread
vim  thread.c
```

编辑如下内容：

```
#include<stddef.h>
#include<stdio.h>
#include<unistd.h>
#include"pthread.h"
void function1(void);
void function2(void);
int func_flag=0;
pthread_mutex_t mutex;

main()
{
    pthread_t reader;
    pthread_mutex_init(&mutex,NULL);
    pthread_create(&reader,NULL,(void*)&function1,NULL);
    function2();
}

void function2(void)
{
    while(1)
    {
        pthread_mutex_lock(&mutex);
        if(func_flag==0)
        {
            printf("excute function2.\n");
            func_flag=1;
        }
        pthread_mutex_unlock(&mutex);
    }
}

void function1(void)
{
    while(1)
    {
        pthread_mutex_lock(&mutex);
        if(func_flag==1)
        {
            printf("excute function1.\n");
            func_flag=0;
```

```
            }
            pthread_mutex_unlock(&mutex);
        }
}
```

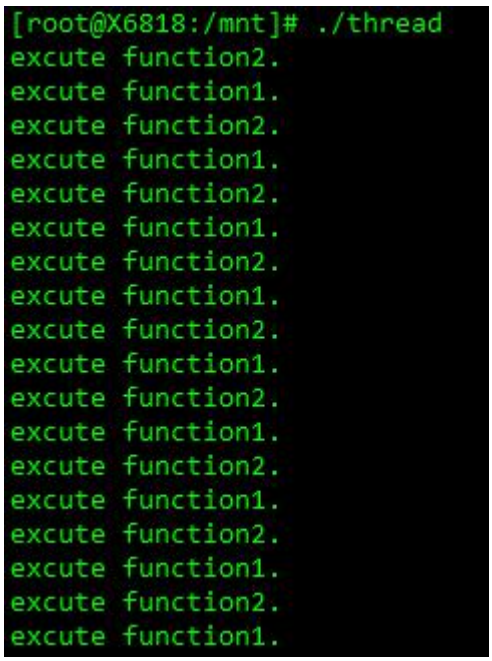
编辑 makefile 文件，内容如下：

```
OBJS=thread.o
LDFLAGS = -lpthread -static
CC = yourtoolchainpath/bin/arm-linux-gcc-4.4.3

thread:${OBJS}
        ${CC} -o $@ $^ $(LDFLAGS)

clean:
        rm -f thread *.o
```

执行 make，将生成的可执行文件 thread 下载到开发板上运行，如下图所示：



8.5 多进程编程示例

在 Linux 下通用调用 fork 函数创建新的进程。调用 fork 时，系统将产生一个与当前进程相同的进程。它与原有的进程具有相同的数据，连接关系和在程序同一处执行时的连续性。通常将原有的进程叫父进程，新创建的进程叫子进程。

fork 调用将分两次返回，从父子进程返回。进程创建语法如下：

```
#include <unistd.h>
pid_t    pid;
pid = fork();
```

如果 pid 返回 0，表示说明从子进程返回，否则从父进程返回，此时返回的是进程的 ID 号。我们可以通过 getpid()函数来获得进程的 ID 号。

首先建立程序编译目录：

```
mkdir    process
cd    process
vim    process.c
```

编辑如下内容：

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>

main()
{
    pid_t pid;
    pid=fork();
    if (pid<0)
    {
        printf("fork is error!\n");
        return 1;
    }
    else if (pid == 0)
    {
        while (1)
        {
            printf("the child process is running now.pid=%d\n",getpid());
            sleep(1);//linux 延时函数,延时 1 秒
        }
    }
    else
    {
        while (1)
        {
            printf("the perent process is running now.pid=%d\n",getpid());
            sleep(1);
        }
    }
    return 0;
}
```

编辑 makefile 文件，内容如下：

```
OBJS=process.o
LDFLAGS = -lpthread -static
CC =yourtoolchainpath/bin/arm-linux-gcc-4.4.3

process:${OBJS}
    ${CC} -o $@ $^ $(LDFLAGS)
```

```
clean:
	rm -f process *.o
```

执行 make 编译，将生成的可执行文件下载到开发板运行，仔细观察串口监控信息：

```
[root@X6818:/mnt]# ./process
the perent process is running now.pid=517
the child process is running now.pid=518
the perent process is running now.pid=517
the child process is running now.pid=518
the perent process is running now.pid=517
the child process is running now.pid=518
the perent process is running now.pid=517
the child process is running now.pid=518
the perent process is running now.pid=517
the child process is running now.pid=518
```

8.6 makefile 编程示例

在上面的很多测试程序实例中，我们都编写了一些简单的 makefile 文件。下面我们介绍 makefile 的基本的语法。

makefile 就好比批处理文件，里面写了一系列集合，当运行 make 编译时，便会按 makefile 提供的命令及顺序完成编译。

这里我们给出三个文件：main.c，func.c，func.h。主程序在 main.c 中，在 main.c 中程序会调用 func.c 中的函数，func.c 中的函数又会用到 func.h 中定义的变量。

main.c 文件内容如下：

```
#include "func.h"

extern int fd;

int main(int argc,char **argv)
{
	fd = open(DEVICE_NAME,0);//打开设备
	if(fd == -1)
	{
		printf("open device %s error \n",DEVICE_NAME);
		return 0;
	}
	else
	{
		printf("open device %s ok! \n",DEVICE_NAME);
	}

	while(1)
	{
		glint_led();
	}
```

```
    close(fd);
    return 0;
}
```

该文件会调用 glint_led ()函数，这个函数在 func.c 中。func.c 的内容如下：

```
#include "func.h"

void glint_led(void)
{
    ioctl(fd,LED_ON);
    sleep(1);
    ioctl(fd,LED_OFF);
    sleep(1);
}
```

这里仅仅是一个读取按键的函数，供 main 函数调用。该函数需要用到了一些变量，另外还需要一些头文件支持，这些都存放在 func.h 中，其内容如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>

#define DEVICE_NAME     "/dev/vib"
#define LED_ON 0x11
#define LED_OFF 0x22

int fd;
```

很明显，这是基于前面的 LED 测试程序，人为的分成的三个文件。我们的目的不在于分离代码，而在于学习 makefile 的编写方法。

当不使用 makefile 时，我们使用如下指令编译：

```
arm-linux-gcc-4.4.3 -o mkfile main.c func.c
```

编译完成后，将会生成可执行文件 mkfile。将它下载到开发板上运行，和前面的按键测试完全相同。现在我们尝试编写第一个属于自己的 makefile:

```
mkfile:main.o func.o
    arm-linux-gcc-4.4.3 -o mkfilemain.o func.o
main.o:main.c
    arm-linux-gcc-4.4.3 -c main.c -o main.o
func.o:func.c func.h
    arm-linux-gcc-4.4.3 -c func.c -o func.o

clean:
    rm -f mkfile *.o
```

执行 make 后，编译器会依次编译 main.c 和 func.c 文件，生成 main.o 和 func.o 文件，最后将这两个.o 文件打包到可执行文件 mkfile 中。这时将 mkfile 文件下载到开发板运行，

效果和前面的是一样的。我们可以执行 make clean 指令清除生成的.o 文件和可执行文件。

makefile 具有很强大的推理功能，我们完全可以简化上面的代码。优化后的代码如下：

```
OBJS=main.o func.o
CC=arm-linux-gcc-4.4.3

mkfile:${OBJS}
    ${CC} -o $@ $^
main.o:
func.o:func.h

clean:
    rm -f mkfile *.o
```

可见，这次比上面的完整版要简化多了。前面通过变量 OBJS 定义了要编译的源文件，变量 CC 给出了交叉编译工具。$@ 表示目标文件的全称，即 mkfile，$^表示所有被依赖的文件，并以空格分开，即 main.o func.o。后面的 clean 为清除指令，执行 make clean 后会执行 clean 后面的指令。需要注意的是，rm 指令后面千万不要使用$@符号来表征我们要删除的目标文件，因为这时候$@已经不再表示 mkfile 了，而表示 clean。同样，使用 make 指令编译，一样能够生成我们需要的目录文件 mkfile。

上面的 makefile 使用了变量以及预定义变量。第一句即定义了变量 OBJS，将它赋值为 main.o func.o，第二句定义了变量 CC，将它赋值为一个交叉编译工具定义。引用变量时，通过${*}表示，这里*表示前面定义的变量。

上面使用了$@和$^两个预定义变量，GNU make 主要有以下七种预定义变量：

$*	不包含扩展名的目标文件名称
$+	所有的依赖文件，以空格分开，以出现的先后为序，可能包含重复的依赖文件
$<	第一个依赖文件的名称
$?	所有的依赖文件，以空格分开，这些依赖文件的修改日期比目标的创建日期晚
$@	目标的完整名称
$^	所有的依赖文件，以空格分开，不包含重复的依赖文件
$%	如果目标是归档成员，则该变量表示目标的归档成员名称。例如，如果目标名称为 mytarget.so(image.o)，则$@ 为 mytarget.so，而$% 为 image.o。

对比以上几种编译方式，我们不然发现，其实最简的还是第一种，因为它就一句话就搞定了。那么在 makefile 中，我们是否也可以精简到只有一句话呢？答案是肯定的。我们继续利用 makefile 强大的推理功能进行简化，得到如下 makefile 代码：

```
OBJS=main.o func.o
CC=arm-none-linux-gnueabi-gcc

mkfile:${OBJS}
    ${CC} -o $@ $^

clean:
    rm -f mkfile *.o
```

这次，makefile 真正编译的代码，就只有上面红色部分一条指令了。和前面比较，不难发现，单独对 main.c 和 func.c 两个文件编译的指令已经去掉了。前面我们提到，makefile

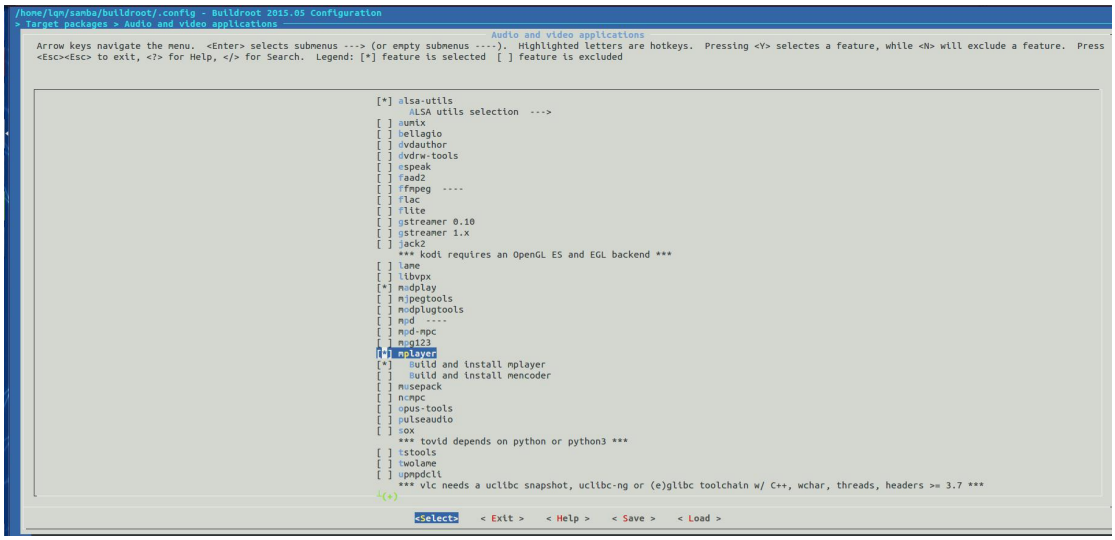
具有强大的推理功能，我们在生成目标文件 mkfile 时，makefile 会推理出它需要 main.c 和 func.c 两个文件，因此它首先就会去编译这两个文件，最后再执行目标文件的生成。因此我们完全可以将它们省去。

这里只是 makefile 的一点基础，读者可以借助于其他书籍对 makefile 作更深一层的了解。

8.7 mplayer 移植

有两种移植 mplayer，第一种方法就是直接下载 mplayer 源码包，手动配置编译环境，再通过交叉编译环境编译。该方法比较传统，我们有更方便的方法，直接在 buildroot 中选中 mplayer，再 make 即可，buildroot 会自动下载 mplayer 源码包，自动配置编译环境并编译。

在 buildroot 的 menuconfig 配置选项中，进入 Target packages-> Audio and video applications 选项，选中 mplayer，如下图所示：



保存退出，在 buildroot 下直接 make，生成文件系统后更新到开发板，mplayer 就打包进文件系统了。

8.8 madplayer 移植

madplayer 的移植方法和 mplayer 的移植方法完全雷同，直接在 buildroot 中选中 madplayer 的选项，再 make 即可。

8.9 屏幕抓图工具 gsnap 移植

说明：以下步骤基于 4.3.3 的交叉编译工具，读者需修改成 x6818 对应的交叉编译工具，其他步骤雷同。

屏幕抓图的方法有多种，有不少爱好者自己动手写抓图小程序。这里我们使用 JPEG 库来处理。具体用到了 jpegsrc.v6b.tar.gz 和 gsnap.tar.gz 两个源码包。

一：安装 libjpeg

解压 jpeg 库源码包，进入根目录：

```
tar zxf jpegsrc.v6b.tar.gz
cd jpeg-6b
```

二：配置编译环境：

```
./configure --prefix=/usr/local/arm/4.3.3/arm-none-linux-gnueabi
--exec-prefix=/usr/local/arm/4.3.3/arm-none-linux-gnueabi --enable-shared --enable-static
```

三：修改 makefile

CC = gcc 修改为 **CC = arm-none-linux-gnueabi-gcc**

AR = ar ac 修改为 **AR = arm-none-linux-gnueabi-ar ac**

AR2=ranlib 修改为 **AR2= arm-none-linux-gnueabi-ranlib**

确保—exec-prefix 已经设置为/usr/local/arm/4.3.3/arm-none-linux-gnueabi，如果没有，手动设置。

四：在/usr/local/arm/4.3.3/arm-none-linux-gnueabi 下建立 man/man1 目录：

```
cd  /usr/local/arm/4.3.3/arm-none-linux-gnueabi
mkdir  -p  man/man1
```

五：编译，安装

```
make
make  install
```

这时，在/usr/local/arm/4.3.3/arm-none-linux-gnueabi/man/man1 目录下将会生成以下文件：

```
cjpeg.1  djpeg.1  jpegtran.1  rdjpgcom.1  wrjpgcom.1
```

在/usr/local/arm/4.3.3/arm-none-linux-gnueabi/lib 目录下生成以下文件：

```
libjpeg.a  libjpeg.la  libjpeg.so  libjpeg.so.62  libjpeg.so.62.0.0
```

六：解压 gsnap

```
tar  zxf  gsnap.tar.gz
cd  gsnap
```

七：修改 makefile

```
all:
    arm-none-linux-gnueabi-gcc -g gsnap.c -ljpeg -o gsnap
clean:
    rm -f gsnap
```

八：编译，得到可执行文件 gsnap

```
make
```

九：将 jpeg 库文件复制到文件系统的 lib 目录，注意保持文件的链接属性

```
cp  -a  libjpeg.s*  “文件系统路径”/lib
```

十：将可执行文件 gsnap 复制到文件系统的 sbin 目录

```
cp  gsnap  “文件系统路径”/sbin
```

十一：重新制作文件系统，下载到开发板上，使用如下命令即可截获图形界面：

```
gsnap  1.jpg  /dev/fb0
```

同样可以将图片保持为 bmp,png 等其他格式。

第 9 章 安装 ubuntu12.04 系统

x6818 开发平台支持 ubuntu12.04 操作系统，它的 bootloader 和内核与 linux 系统完全相同，我们只需要修改文件系统即可。

有三种方法可以启动 ubuntu 系统，具体如下表：

	uboot	内核	文件系统
方法一	存在 emmc 中	存在 emmc 中	存在 SD 卡中
方法二	存在 emmc 中	存在 emmc 中	存在 emmc 中
方法三	存在 SD 卡中	存在 SD 卡中	存在 SD 卡中

这里我们先介绍方法一和方法二，方法三读者可以自行尝试。

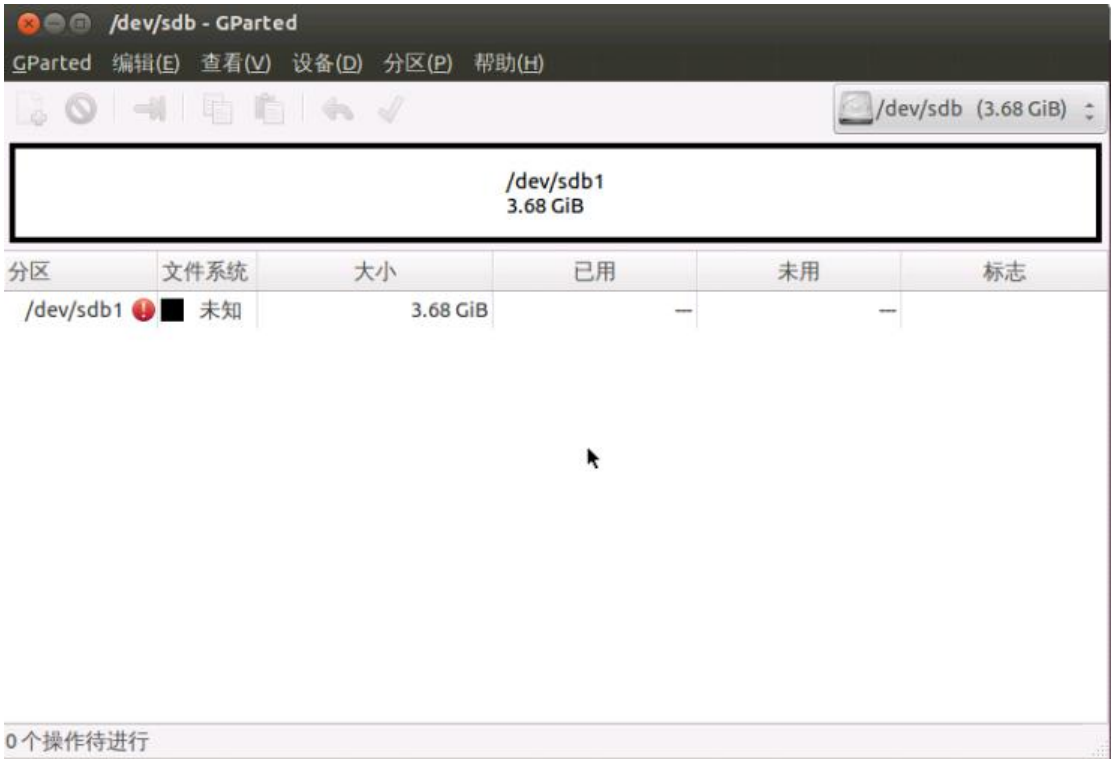
9.1 ubuntu 烧写方法一

9.1.1 将 uboot 和 boot.img 烧写到 emmc

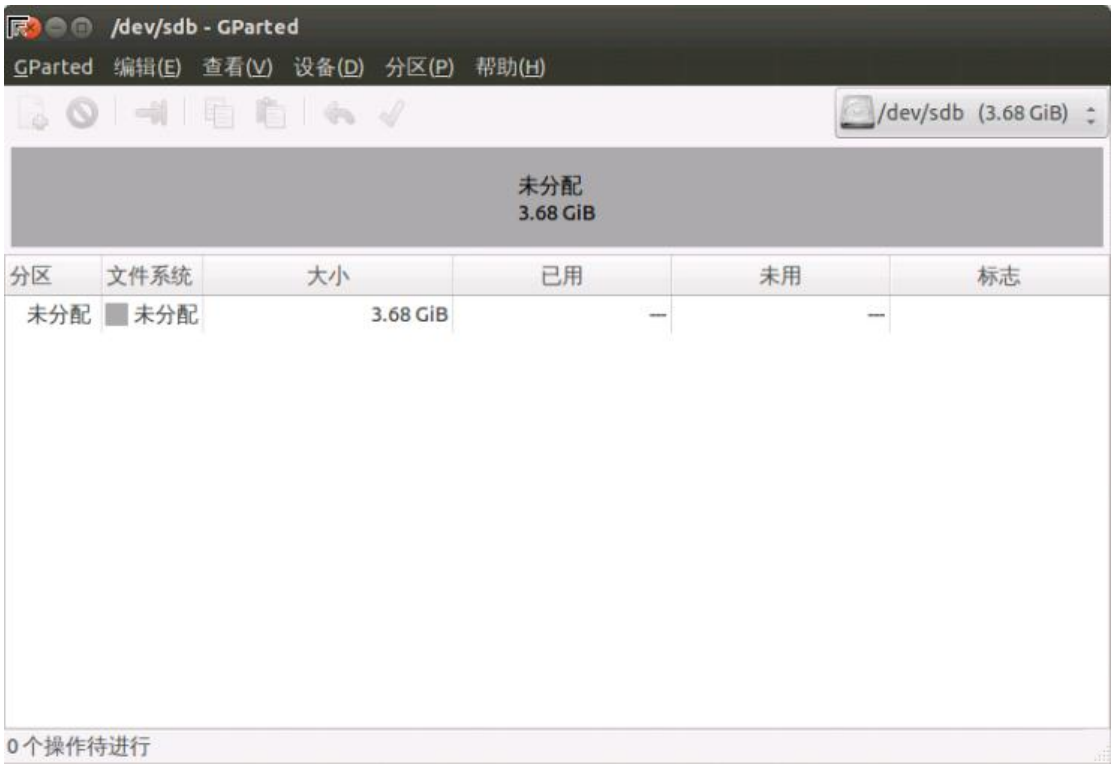
方法完全雷同于前文，参考本文第三章，这里不再赘述。

9.1.2 将 ubuntu 文件系统烧到到 TF 卡

第一步：准备一张 TF 卡，使用 gparted 工具删除里面所有分区。将 SD 卡通过读卡器插到运行 ubuntu 系统的 PC 机上，在命令终端敲入 sudo gparted /dev/sdb，如下图所示：



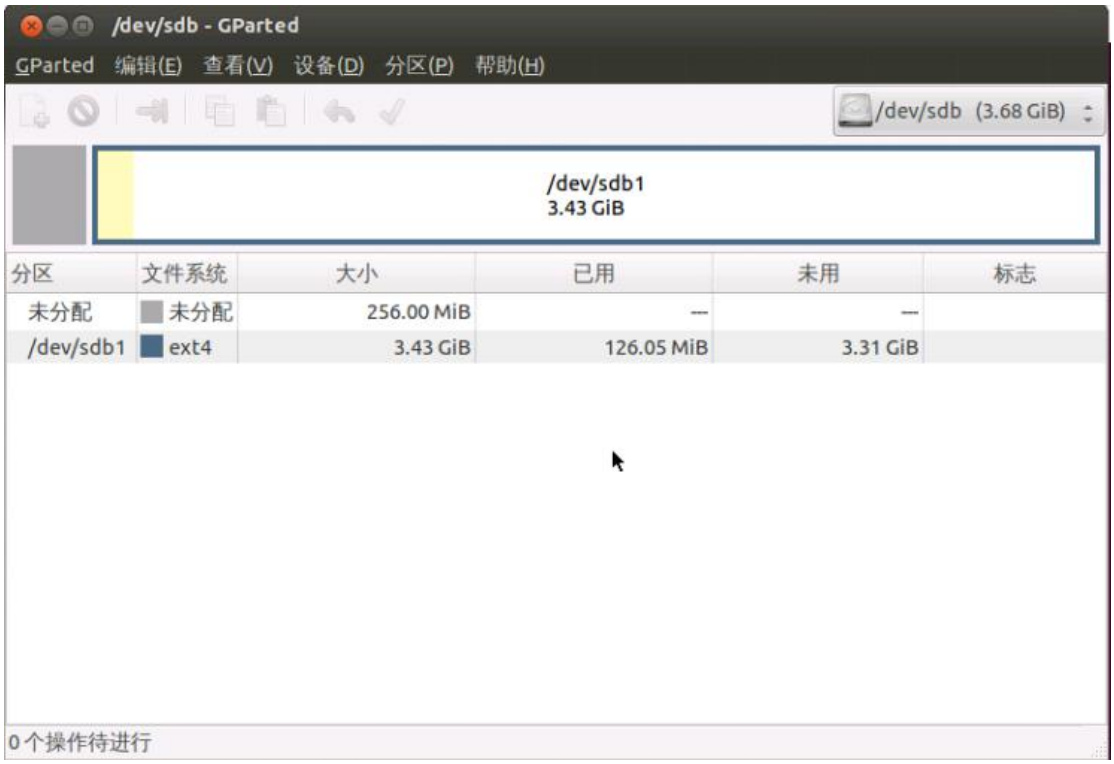
选中列表中的磁盘分区，右键删除，再点击菜单上的勾，应用刚才的操作。删除完后界面如下：



第二步：新建分区。选中列表中未分配的磁盘，再点击分区->新建，在弹出的对话框中，在**文件系统中选择 ext4，在之前的空余空间中输入 256，预留 256M 空间**，可用于后面烧写 uboot，方便后续 SD 卡更新。



点击添加，再点击菜单栏的勾，完成新的分区创建。



第三步：将我们制作好的 ubuntu 源码包 x6818_A/Image/x6818-ubuntu/ubuntu-rootfs.tar.bz2 解压到前面新建的 SD 卡分区：

```
cd ~
mkdir sdcard
sudo mount /dev/sdb1 sdcard
sudo mv yourubuntudir/ubuntu-rootfs.tar.bz2 sdcard
cd sdcard
sudo tar xvf ubuntu-rootfs.tar.bz2
rm ubuntu-rootfs.tar.bz2
```

解压完成之后，可以将 ubuntu-rootfs.tar.bz2 删除，到此，ubuntu 的 TF 卡制作完成。

9.1.3 设置 uboot 环境变量

连接调试串口，启动开发板，在串口打印 3 秒倒计时时按任意键取消倒计时，使用如下命令设置开发板的 uboot 环境变量：

```
setenv bootcmd "ext4load mmc 2:1 0x48000000 uImage;bootm 0x48000000"
setenv bootargs " lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p1 rw rootfstype=ext4"
save
```

设置完环境变量，即可启动 ubuntu 操作系统了。

针对 android，linux，ubuntu 三种不同性质的操作系统，环境变量会有所不同，它们表现在 bootargs 和 bootcmd 两个变量上。针对同一种操作系统，如 android，或 linux，或 ubuntu，如果对应不同的外设，如 LCD，触摸屏等，则它只表现在 bootargs 这个环境变量上。

以下设置为触摸屏是思立微 gsl1680，显示屏不同的设置参数。

标清屏（800x480）：

```
setenv bootargs "lcd= at070tn92 tp=gslx680-linux root=/dev/mmcblk0p1 rw rootfstype=ext4"
```

```
save
```

高清屏（1024x600）：

```
setenv bootargs "lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p1 rw rootfstype=ext4"
save
```

MIPI 屏（1024x600）：

```
setenv bootargs "lcd=wy070ml tp=gslx680-linux root=/dev/mmcblk0p1 rw rootfstype=ext4"
save
```

VGA-1024x768:

```
setenv bootargs "lcd=vga-1024x768 tp=gslx680-linux root=/dev/mmcblk0p1 rw rootfstype=ext4"
save
```

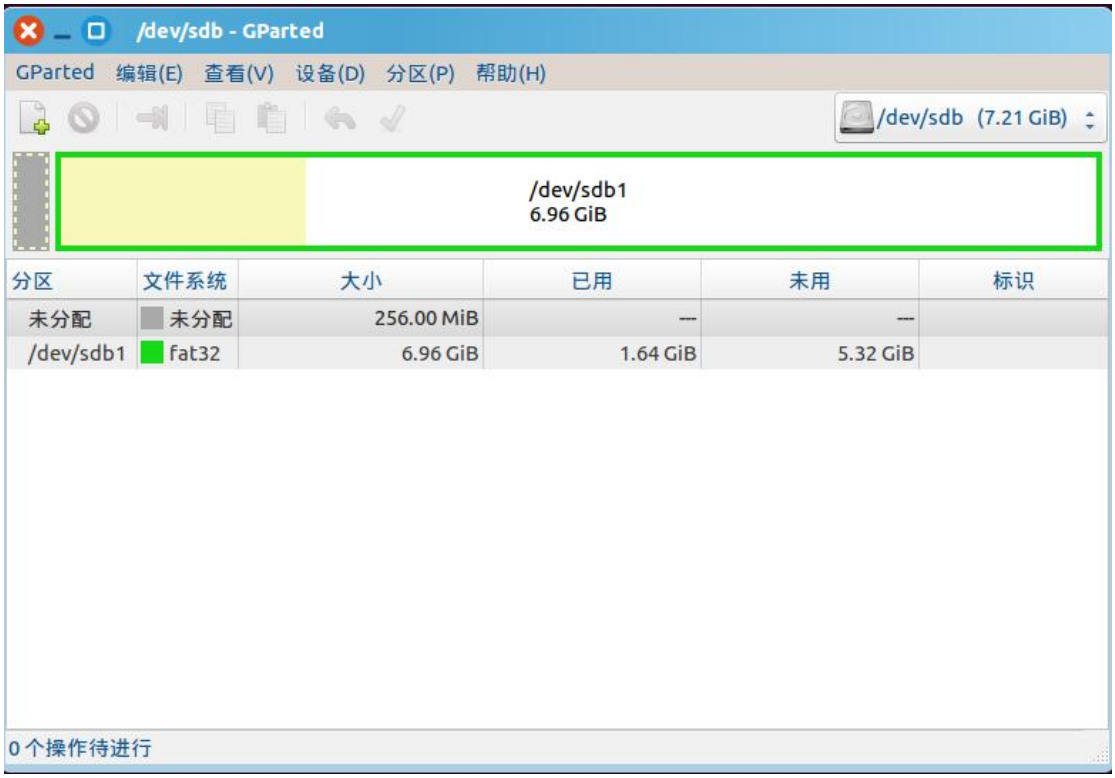
VGA-1440x900:

```
setenv bootargs "lcd=vga-1440x900 tp=gslx680-linux root=/dev/mmcblk0p1 rw rootfstype=ext4"
save
```

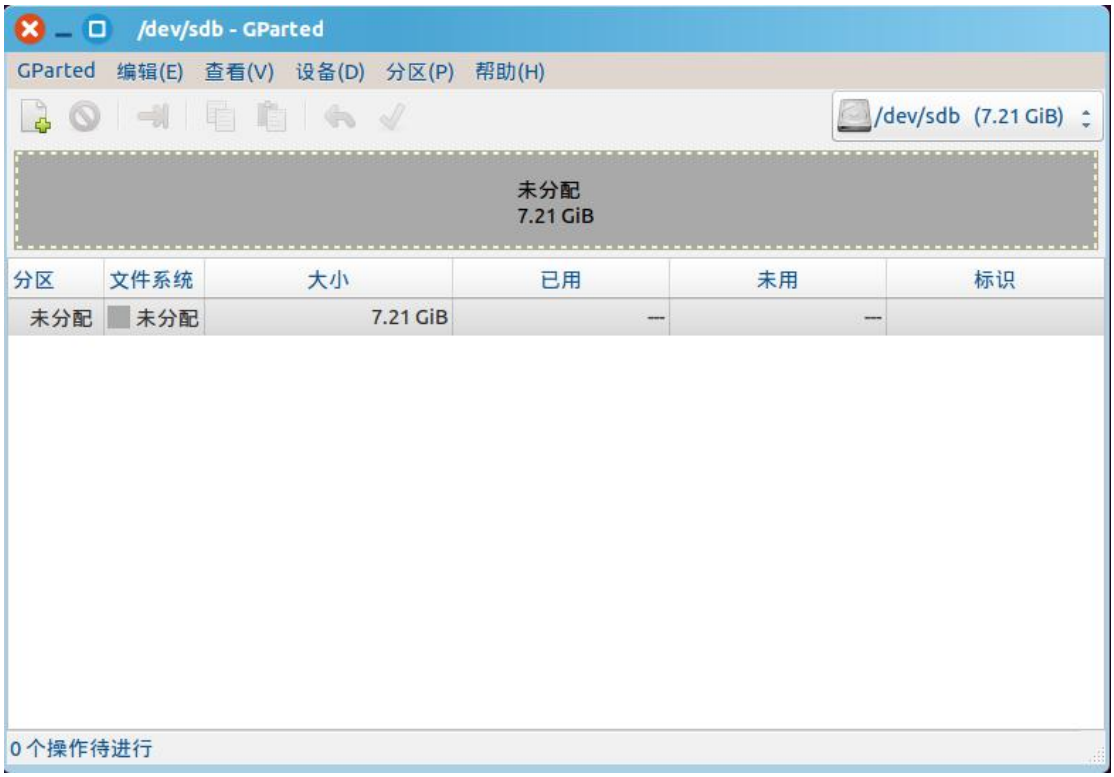
9.2 ubuntu 烧写方法二

9.2.1 制作 ubuntu 升级卡

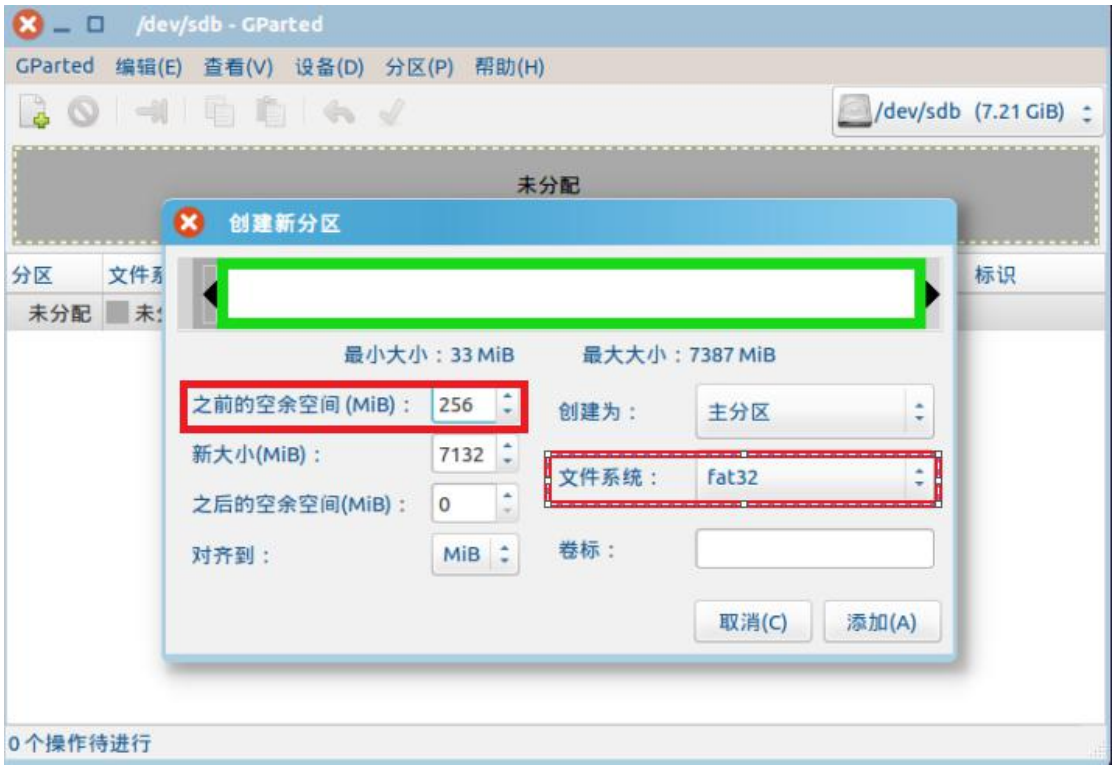
第一步：准备一张不小于 4GB 的 TF 卡，使用 gparted 工具删除里面所有分区。将 TF 卡通过读卡器插到运行 ubuntu 系统的 PC 机上，在命令终端敲入 sudo gparted /dev/sdb，如下图所示：



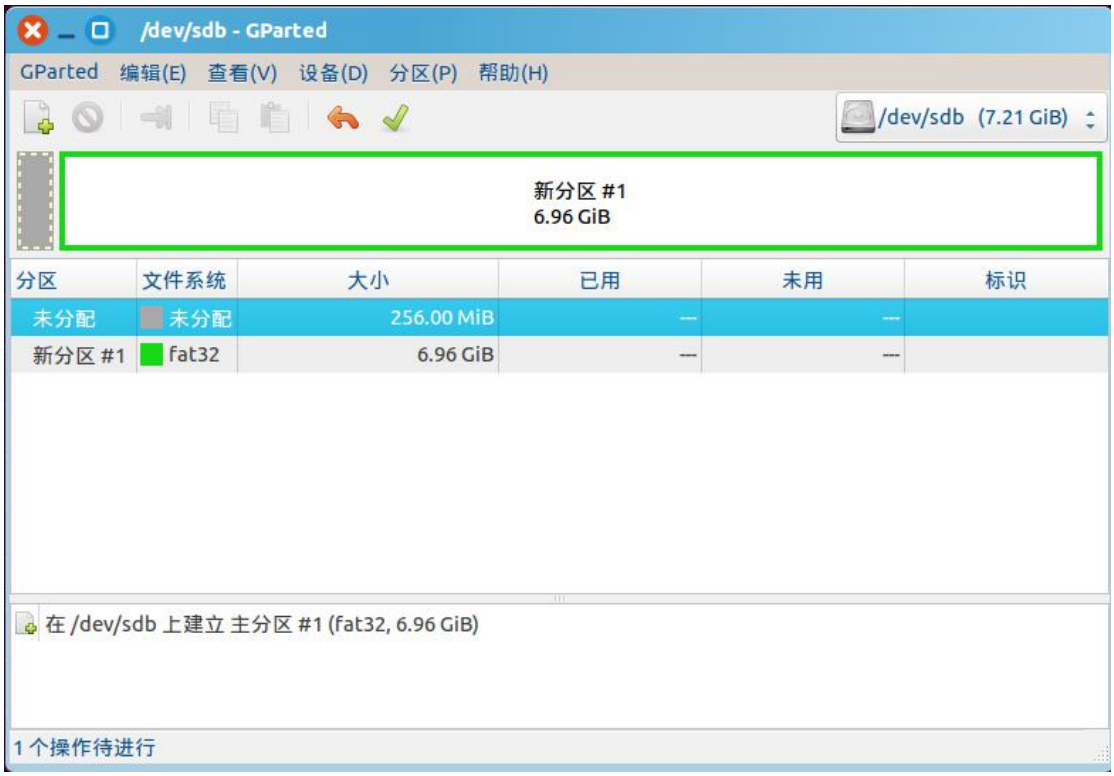
选中列表中的磁盘分区，右键删除，再点击菜单上的勾，应用刚才的操作。删除完后界面如下：



第二步：新建分区。选中列表中未分配的磁盘，再点击分区->新建，在弹出的对话框中，在文件系统中选择 fat32，**在之前的空余空间中输入 256，预留 256M 空间**，可用于后面烧写 uboot，方便后续 TF 卡更新。



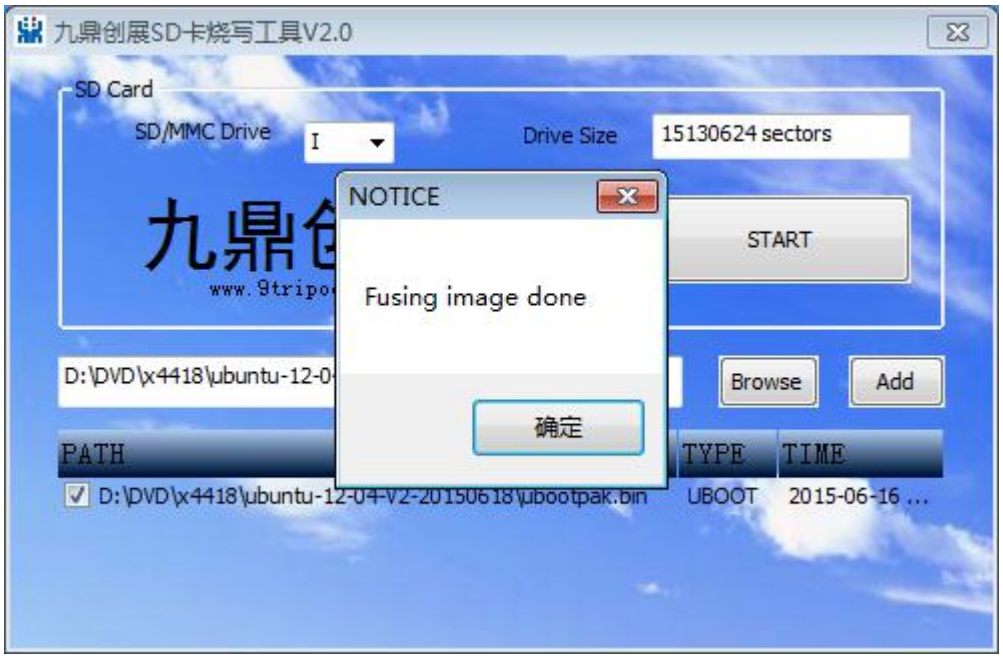
点击添加，再点击菜单栏的勾，完成新的分区创建。



第三步：将 TF 卡通过读卡器插到装有 WINDOWS 系统的 PC 机上，使用 iRom_Fusing_Tool 工具将能够通过 TF 卡更新 ubuntu 的 uboot 映像烧写到 TF 卡。



点击 Browse，找到需要的 uboot 映像，点击 Add，然后点击 START，即可将 uboot 烧写到 TF 卡。烧写成功后会弹出如下对话框：



第五步：将 ubootpak.bin，boot.img 以及 ubuntu-rootfs.tar.bz2 拷贝到 TF 卡的 x6818-ubuntu 目录，同时在 x6818-ubuntu 目录下新建一个名为 env.txt 的文件，编辑内容如下：

```
************************ ubuntu12.04 ******************************
bootargs=lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p7 rw rootfstype=ext4
bootcmd=ext4load mmc 2:1 0x48000000 uImage;bootm 0x48000000
********************** www.9tripod.com ***************************
```

保证 TF 卡的根目录不存在 x6818-android 及 x6818-qt 目录，避免更新成其他系统。

注意：x6818 开发板和 ibox6818 卡片电脑的 ubootpak.bin，boot.img 有区别(详细说明请看前文 2.2 节)，请拷贝对应的映像，否则升级失败！

9.2.2 使用升级卡一键升级 ubuntu 系统

将 TF 卡插到开发板右侧的 TF 卡槽，按住返回键(开发板左边自下而上第二个按键)后上电开机，将会自动升级。由于 ubuntu 系统比较庞大，约 15 分钟后将会自动更新完成，并进入 ubuntu 系统。如果不想按住返回按键实现强制自动升级，只需在进入 uboot 命令行后，执行如下指令，再重启即可。

```
env default –a -f
save
```

9.2.3 设置 uboot 环境变量

注意，将 ubuntu 烧写到 EMMC 中时，所挂载的文件系统的设备节点会有所变化（即方法一与方法二环境变量有所不同）。

启动开发板，设置开发板的 uboot 环境变量，参考示例如下：

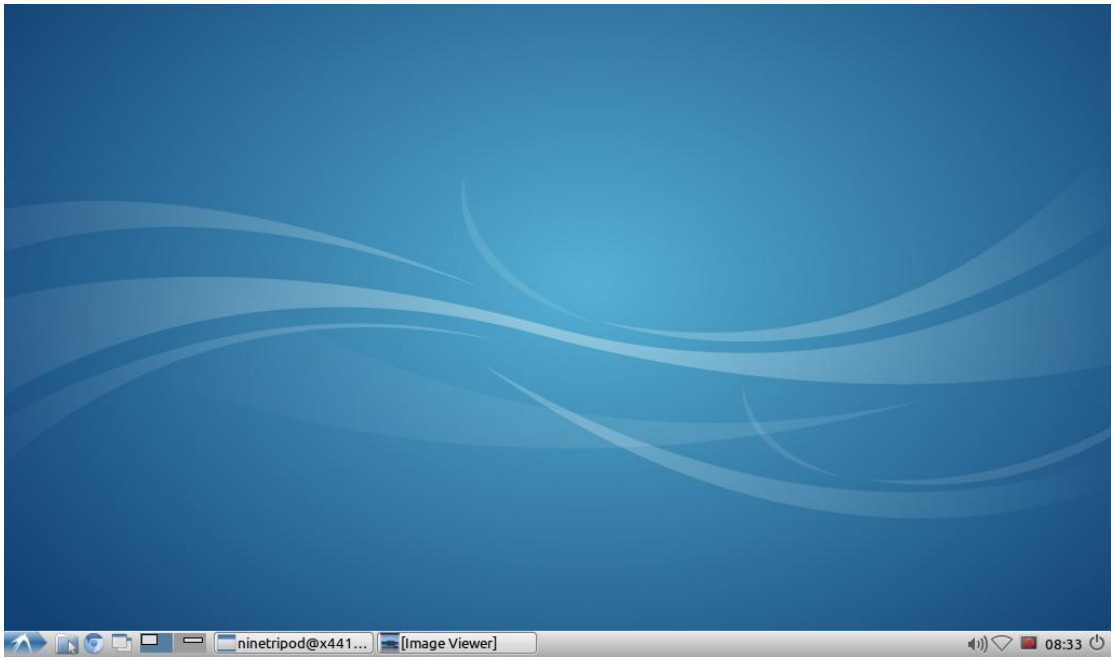
```
setenv bootcmd "ext4load mmc 2:1 0x48000000 uImage;bootm 0x48000000"
setenv bootargs " lcd=vs070cxn tp=gslx680-linux root=/dev/mmcblk0p7 rw rootfstype=ext4"
save
```

设置完环境变量，即可启动 ubuntu 操作系统了。关于环境变量的更多说明，请查看前文 9.1.3 节。

第 10 章 体验 ubuntu12.04 系统

10.1 ubuntu 系统界面

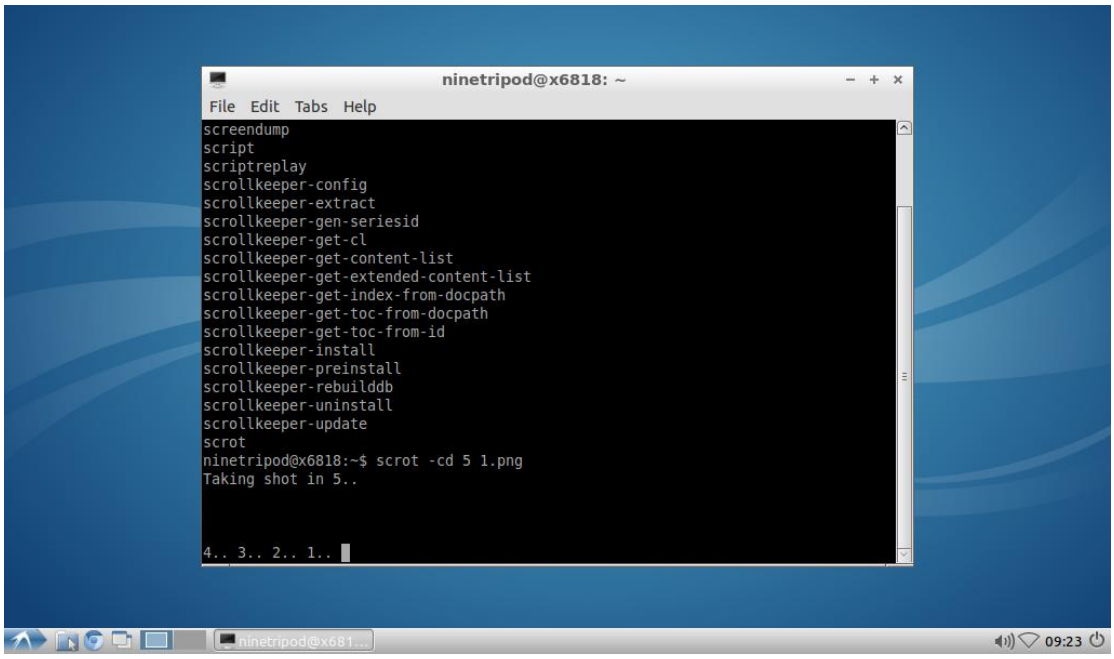
账户：root、ninetripod 密码：123456



10.2 使用 USB 鼠标键盘

启动开发板进入 ubuntu 系统，插上 USB 鼠标键盘即可操作了。

10.3 使用超级终端



10.4 使用 scrot 工具抓图

在命令终端输入 scrot 命令可以实现抓图。如：

```
scrot -cd 5 1.png
```

表示延时 5 秒后将桌面的图像保存为 1.png。

10.5 使用有线上网

将网线接到开发板的以太网接口即可实现上网。



10.6 使用 WIFI 上网

连接 USB WIFI 后，输入密码即可上网。



10.7 使用 3G 上网

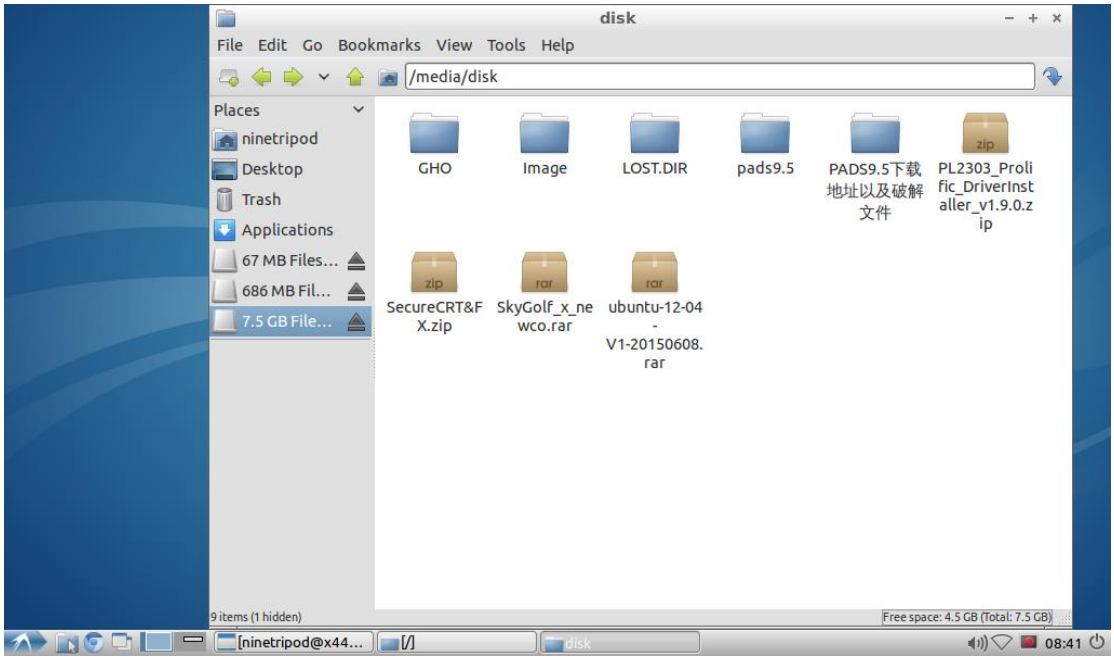
将指定的 3G 模块插到 x6818 开发板的 USB 接口即可上网。

10.8 使用蓝牙传输数据

x6818 开发板支持 realtek 的 RT8723BU 模块，在 ubuntu 下可直接上网，并通过蓝牙传输数据。

10.9 使用 U 盘

将 U 盘插到 USB 口即可使用 U 盘。



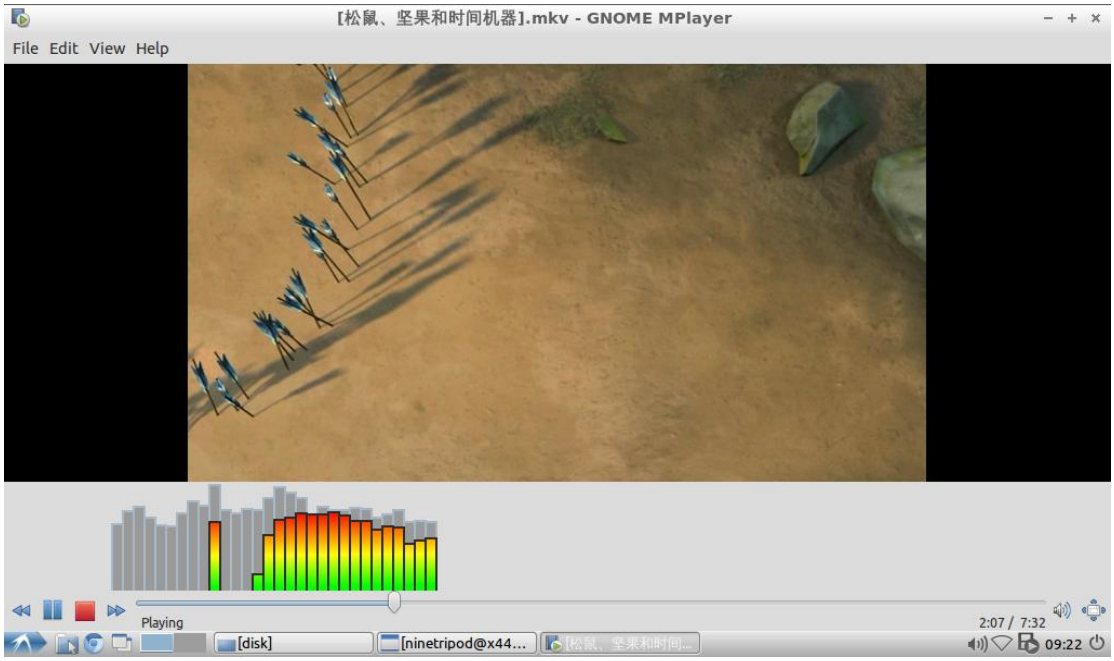
10.10 播放音乐

将 U 盘拷贝到 U 盘上，插上 U 盘后，找到音乐文件，双击即可播放。



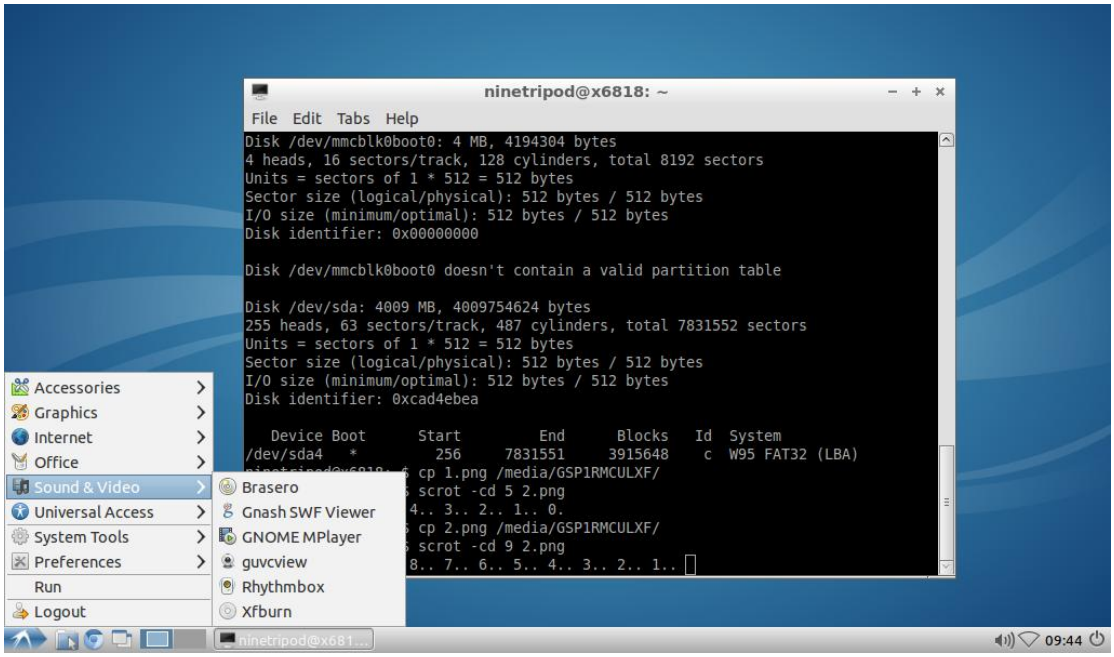
10.11 播放视频

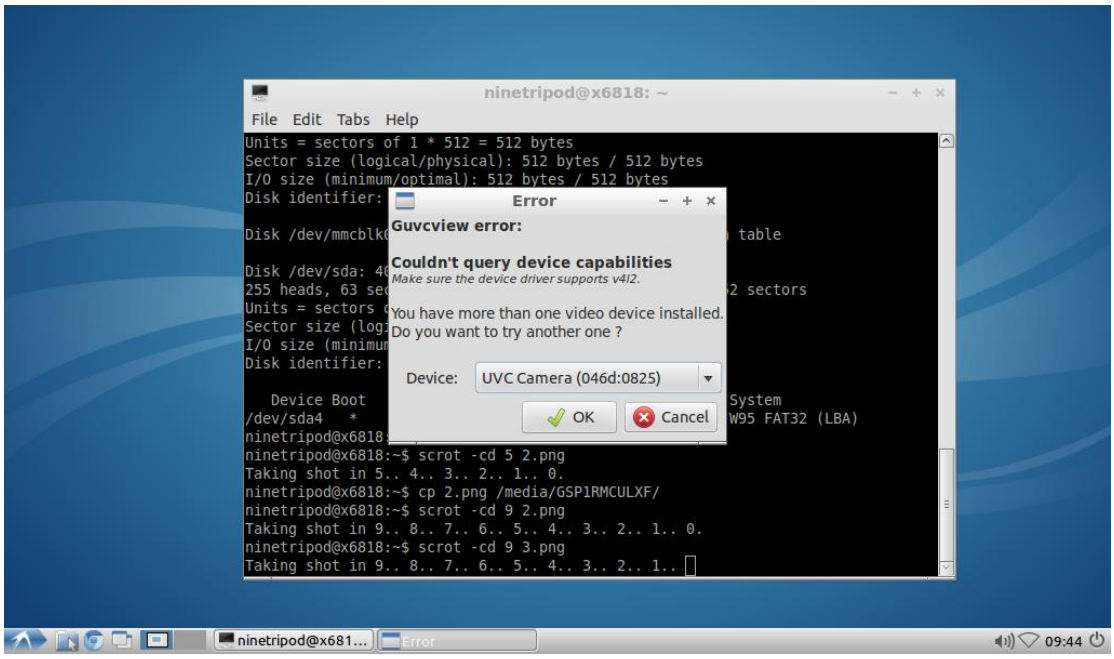
将 U 盘拷贝到 U 盘上，插上 U 盘后，找到视频文件，双击即可播放。以下是播放 1080P 视频的截图，实测没有一点卡顿现象，非常的流畅。

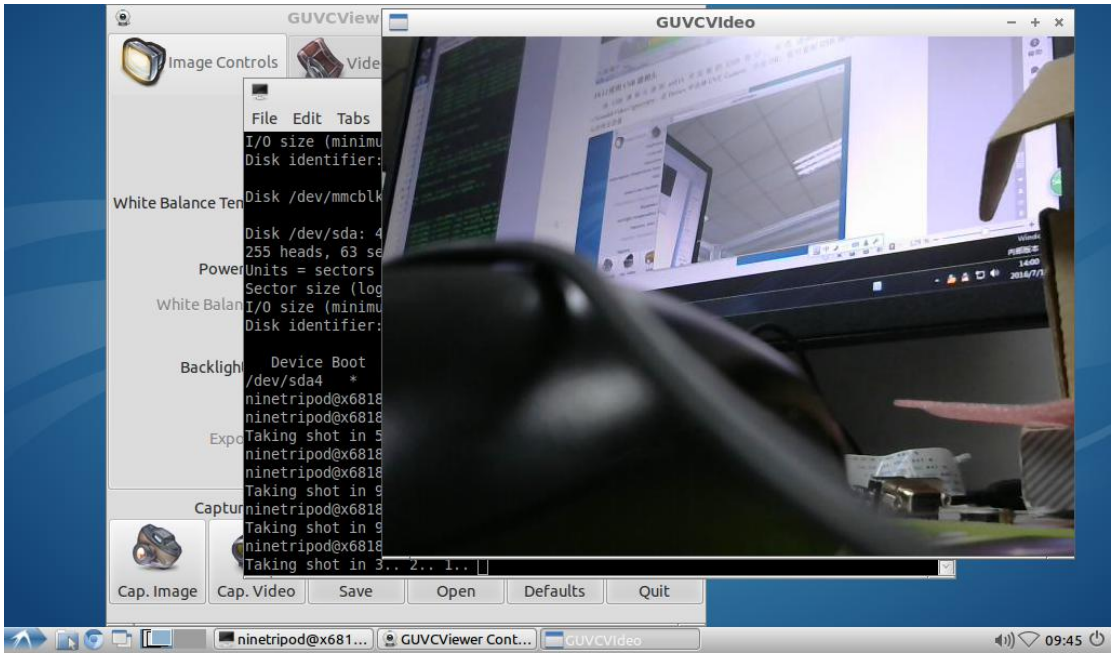


10.12 使用 USB 摄相头

将 USB 摄相头插到 x6818 开发板的 USB 接口，点击 ubuntu 的开始 ->Sound&Video->guvcview，在 Device 中选择 UVC Camera，点击 OK，即可看到 USB 摄相头的预览图像。




ninetripod@x6818: ~
File Edit Tabs Help
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier:
Error
Guvcview error:
Couldn't query device capabilities
Make sure the device driver supports v4l2.
You have more than one video device installed.
Do you want to try another one ?
Device: UVC Camera (046d:0825)
OK
Cancel
ninetripod@x6818:~$ scrot -cd 5 2.png
Taking shot in 5.. 4.. 3.. 2.. 1.. 0.
ninetripod@x6818:~$ cp 2.png /media/GSP1RMCULXF/
ninetripod@x6818:~$ scrot -cd 9 2.png
Taking shot in 9.. 8.. 7.. 6.. 5.. 4.. 3.. 2.. 1.. 0.
ninetripod@x6818:~$ scrot -cd 9 3.png
Taking shot in 9.. 8.. 7.. 6.. 5.. 4.. 3.. 2.. 1..
09:44


GUVCVideo
Image Controls
File Edit Tabs
Cap. Image
Cap. Video
Save
Open
Defaults
Quit
09:45

第 11 章 其他产品介绍

11.1 核心板系列

X6410CV10
X210CV3
X210CV4
G210CV10
I210CV10
X4412CV2
X4418CV3
X6818CV3
X3288CV3

11.2 开发板系列

x6410 开发板
x210 开发板
g210 开发板
i210 开发板
x4412 开发板
x4418 开发板
x6818 开发板
x3288 开发板

11.3 卡片电脑系列

ibox4412 卡片电脑
ibox4418 卡片电脑
ibox6818 卡片电脑

说明：产品详细规格，以及更多其他产品请关注九鼎创展官方网站和论坛。